Generative Models

Prof. Anil Singh Parihar

Department of Computer Science & Engineering

Delhi Technological University, Delhi

Supervised vs Unsupervised Learning

Supervised Learning

Data: (x, y)

x is data, y is label

Goal: Learn a *function* to map x → y

Examples: Classification, regression, object detection, semantic segmentation, image captioning, etc.

Classification

Cat

Supervised vs Unsupervised Learning

Supervised Learning

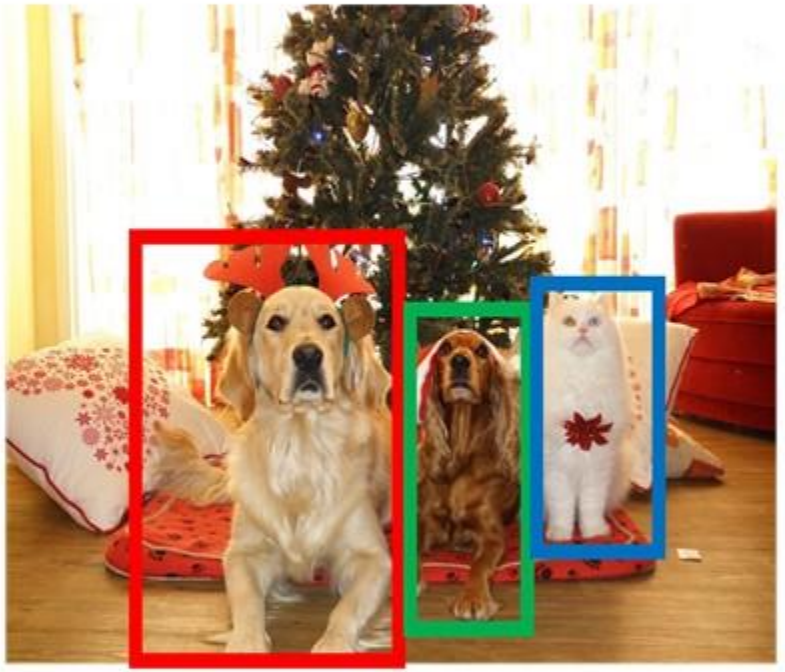
Data: (x, y)

x is data, y is label

Goal: Learn a *function* to map x → y

Examples: Classification, regression, object detection, semantic segmentation, image captioning, etc.

Object Detection



DOG, **DOG**, **CAT**

Supervised vs Unsupervised Learning

Supervised Learning

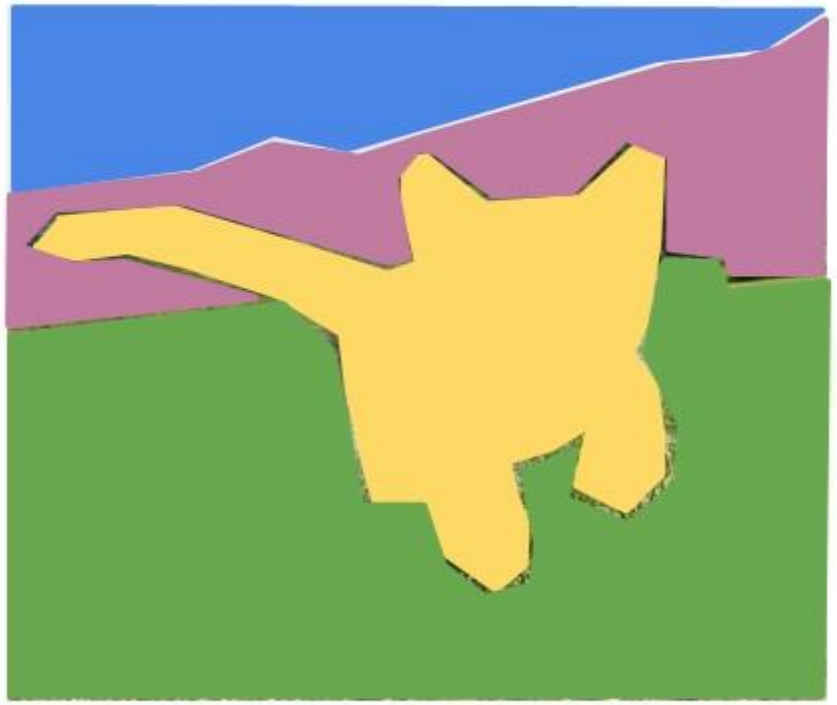
Data: (x, y)

x is data, y is label

Goal: Learn a *function* to map x → y

Examples: Classification, regression, object detection, semantic segmentation, image captioning, etc.

Semantic Segmentation



GRASS, CAT, TREE, SKY

Supervised vs Unsupervised Learning

Supervised Learning

Data: (x, y)

x is data, y is label

Goal: Learn a *function* to map x → y

Examples: Classification, regression, object detection, semantic segmentation, image captioning, etc.

Image captioning

A cat sitting on a suitcase on the floor

Caption generated using neuraltalk2
Image is CC0 Public domain

Supervised vs Unsupervised Learning

Supervised Learning

Data: (x, y)

x is data, y is label

Goal: Learn a *function* to map x → y

Examples: Classification, regression, object detection, semantic segmentation, image captioning, etc.

Unsupervised Learning

Data: x

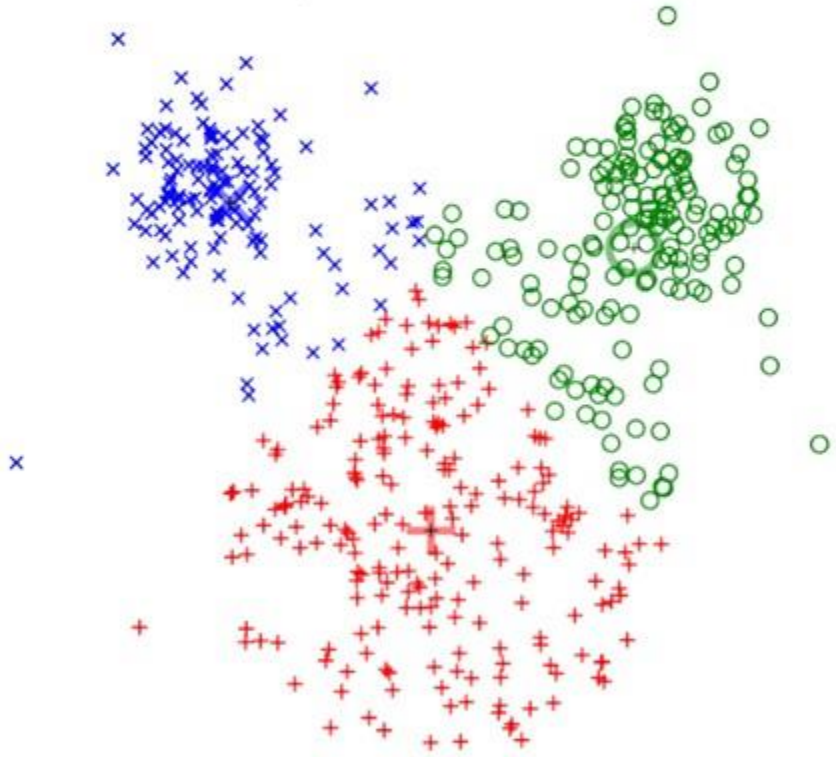
Just data, no labels!

Goal: Learn some underlying hidden *structure* of the data

Examples: Clustering, dimensionality reduction, feature learning, density estimation, etc.

Supervised vs Unsupervised Learning

Clustering
(e.g. K-Means)



Unsupervised Learning

Data: x

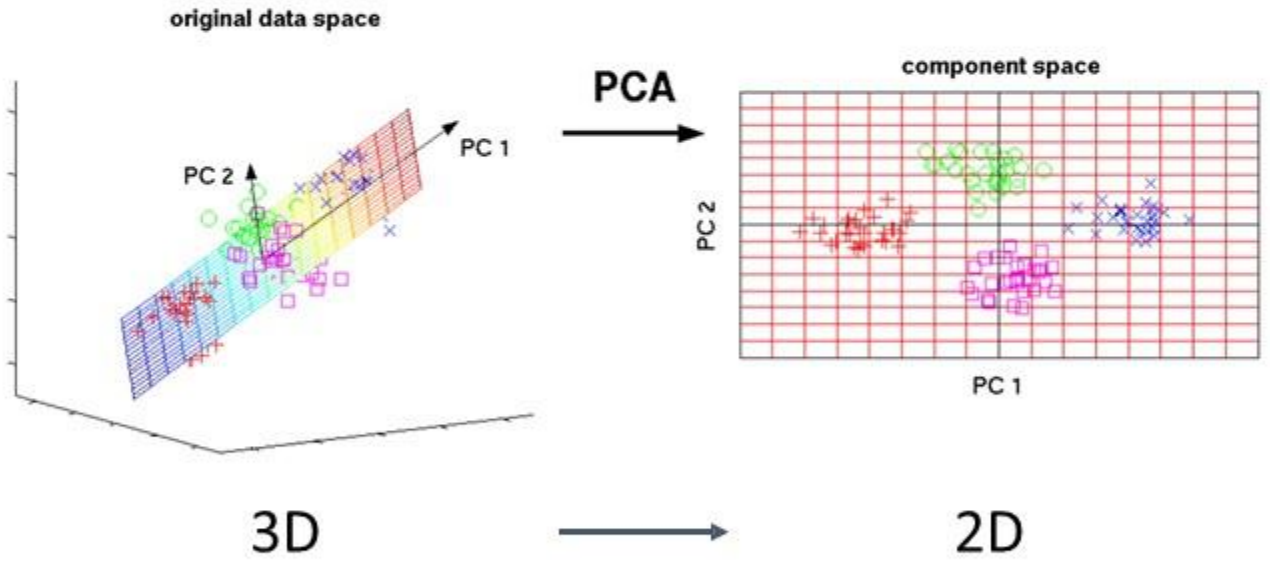
Just data, no labels!

Goal: Learn some underlying hidden *structure* of the data

Examples: Clustering, dimensionality reduction, feature learning, density estimation, etc.

Supervised vs Unsupervised Learning

Dimensionality Reduction (e.g. Principal Components Analysis)



Unsupervised Learning

Data: x

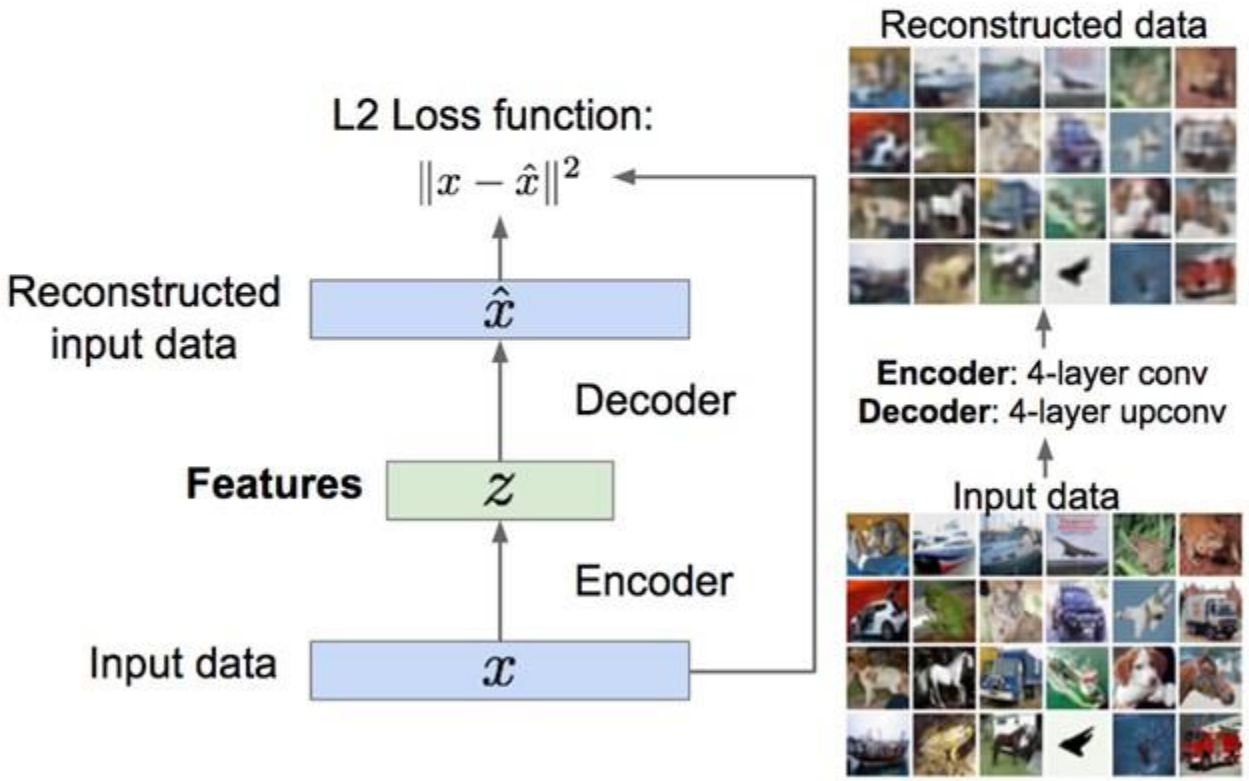
Just data, no labels!

Goal: Learn some underlying hidden *structure* of the data

Examples: Clustering, dimensionality reduction, feature learning, density estimation, etc.

Supervised vs Unsupervised Learning

Feature Learning
(e.g. autoencoders)



Unsupervised Learning

Data: x

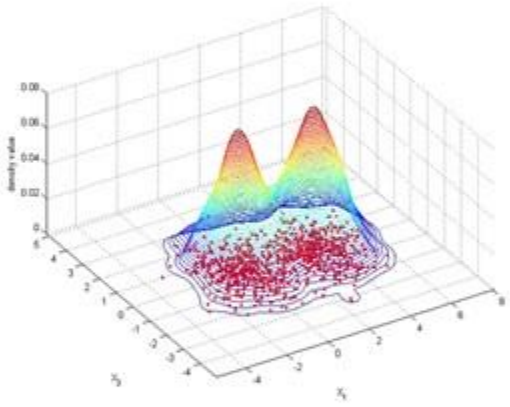
Just data, no labels!

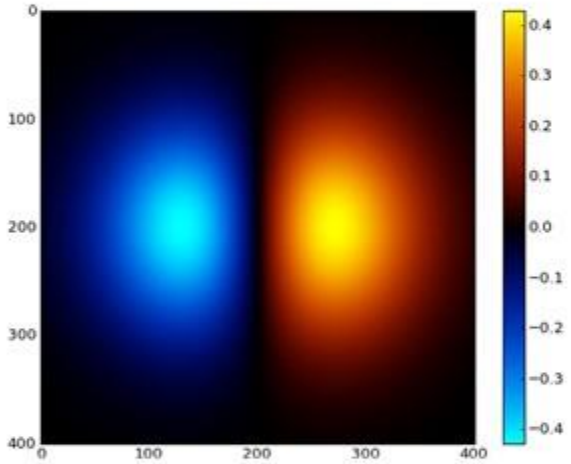
Goal: Learn some underlying hidden *structure* of the data

Examples: Clustering, dimensionality reduction, feature learning, density estimation, etc.

Supervised vs Unsupervised Learning

Density Estimation





Unsupervised Learning

Data: x

Just data, no labels!

Goal: Learn some underlying hidden *structure* of the data

Examples: Clustering, dimensionality reduction, feature learning, density estimation, etc.

Supervised vs Unsupervised Learning

Supervised Learning

Data: (x, y)

x is data, y is label

Goal: Learn a *function* to map x → y

Examples: Classification, regression, object detection, semantic segmentation, image captioning, etc.

Unsupervised Learning

Data: x

Just data, no labels!

Goal: Learn some underlying hidden *structure* of the data

Examples: Clustering, dimensionality reduction, feature learning, density estimation, etc.

Discriminative vs Generative Models

Discriminative Model: Learn a probability distribution p(y|x)

Generative Model: Learn a probability distribution p(x)

Conditional Generative Model: Learn p(x|y)

Data: x

Label: y

Cat

Discriminative vs Generative Models

Discriminative Model: Learn a probability distribution p(y|x)

Generative Model: Learn a probability distribution p(x)

Conditional Generative Model: Learn p(x|y)

Data: x

Label: y

Cat

Probability Recap:

Density Function

p(x) assigns a positive number to each possible x; higher numbers mean x is more likely

Density functions are **normalized**:

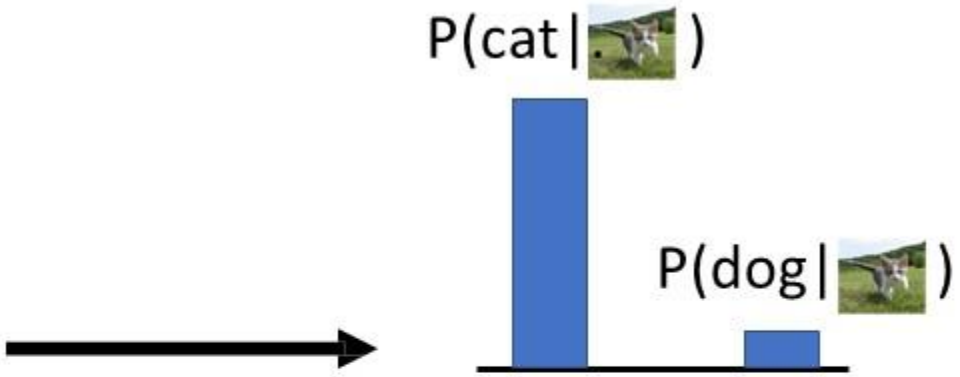
$$\int_X p(x)dx = 1$$

Different values of x **compete** for density

Discriminative vs Generative Models

Discriminative Model: Learn a probability distribution p(y|x)

Data: x



Generative Model: Learn a probability distribution p(x)

Density Function
p(x) assigns a positive number to each possible x; higher numbers mean x is more likely

Density functions are **normalized**:

$$\int_X p(x)dx = 1$$

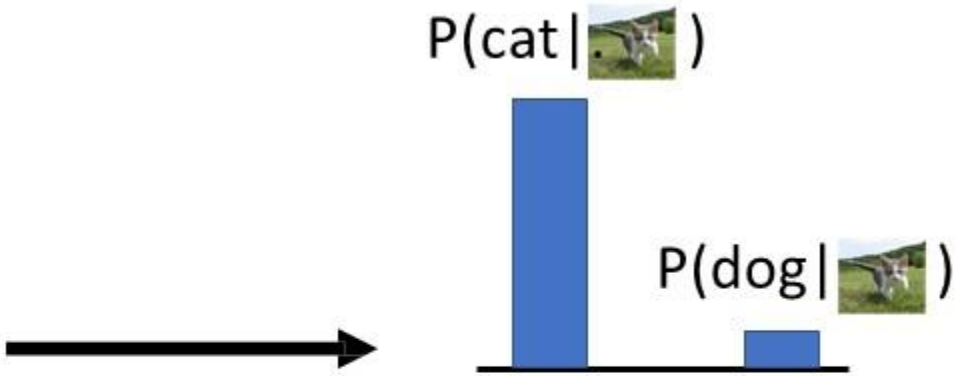
Different values of x **compete** for density

Conditional Generative Model: Learn p(x|y)

Discriminative vs Generative Models

Discriminative Model: Learn a probability distribution p(y|x)

Data: x



Generative Model: Learn a probability distribution p(x)

- For example, if a discriminative model is classifying animals in images and the labels are "dog", and "cat",
- The model assigns a probability to each of these labels such that the sum of probabilities for "dog", and "cat",for a single image equals 1.

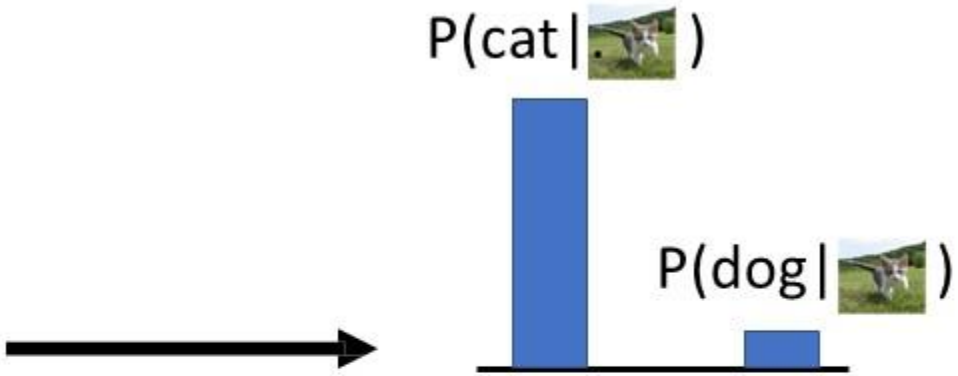
Conditional Generative Model: Learn p(x|y)

- Here, you can think of it as these labels competing against each other for higher probability based on the input image features.

Discriminative vs Generative Models

Discriminative Model: Learn a probability distribution p(y|x)

Data: x



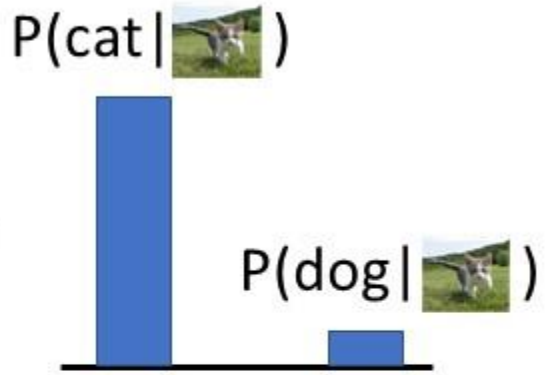
Generative Model: Learn a probability distribution p(x)

- However, the probability assignments for one image do not affect the probability assignments for another image.
- Each classification decision is independent of the others.
- For example, the probability that image A is classified as a "dog" doesn't influence or compete with the probability of image B being classified as a "cat".

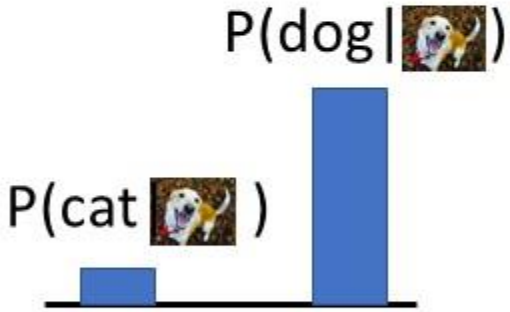
Conditional Generative Model: Learn p(x|y)

Discriminative vs Generative Models

Discriminative Model: Learn a probability distribution p(y|x)



Generative Model: Learn a probability distribution p(x)

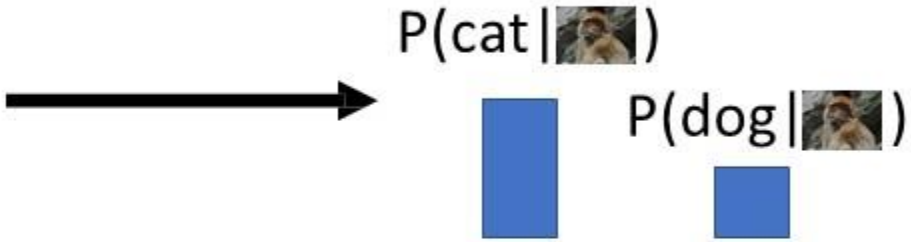


Conditional Generative Model: Learn p(x|y)

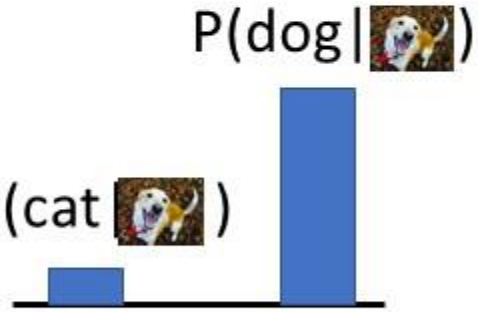
Discriminative model: the possible labels for each input "compete" for probability mass for given Image.
But no competition between **images**

Discriminative vs Generative Models

Discriminative Model: Learn a probability distribution p(y|x)



Generative Model: Learn a probability distribution p(x)

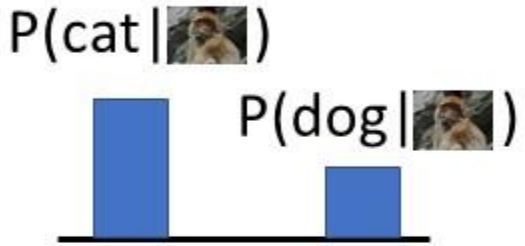


Conditional Generative Model: Learn p(x|y)

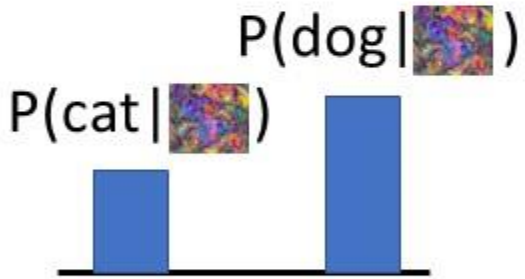
Discriminative model: No way for the model to handle inputs for which model is not trained;
it must give label distributions for all images (whether trained on it or not)

Discriminative vs Generative Models

Discriminative Model: Learn a probability distribution p(y|x)



Generative Model: Learn a probability distribution p(x)



Conditional Generative Model: Learn p(x|y)

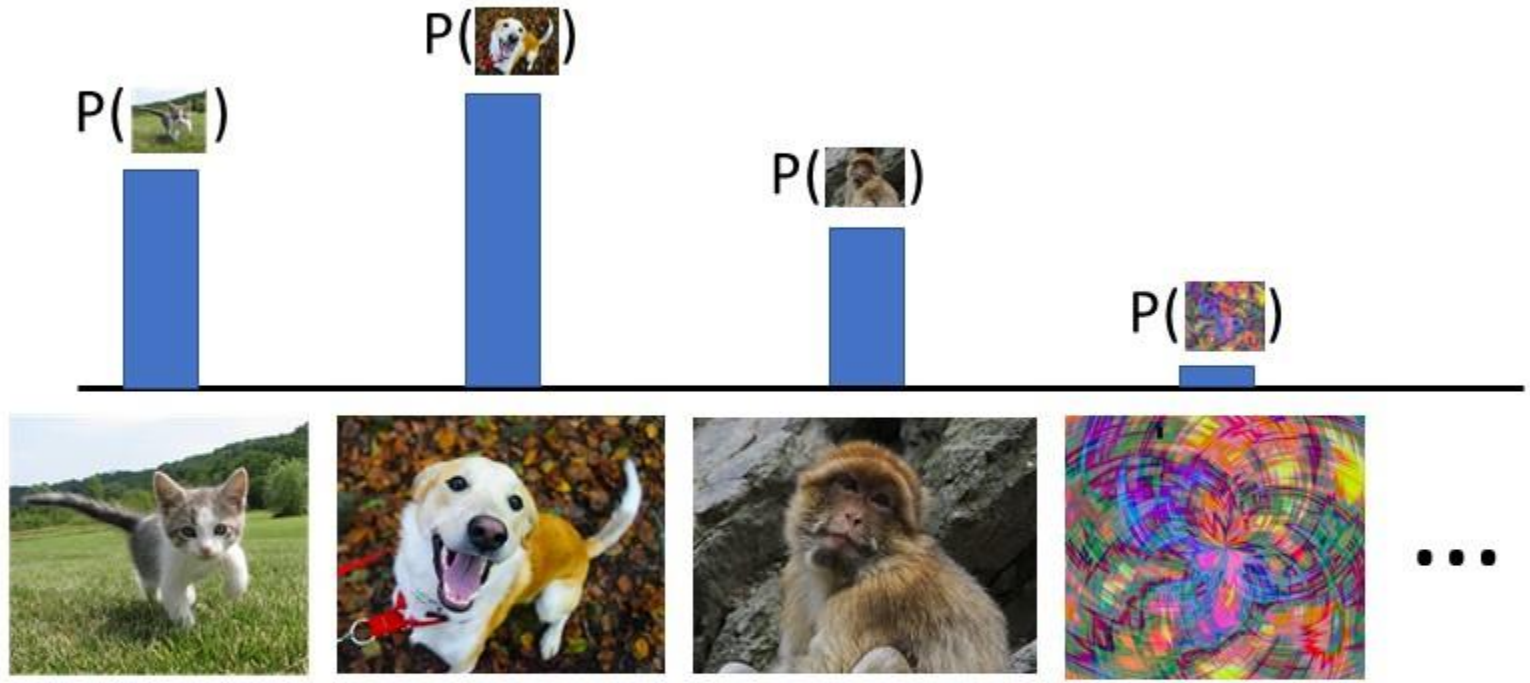
Discriminative model: No way for the model to handle unreasonable inputs; it must give label distributions for all images

Discriminative vs Generative Models

Discriminative Model: Learn a probability distribution p(y|x)

Generative Model: Learn a probability distribution p(x)

Conditional Generative Model: Learn p(x|y)



Generative model: learn the underlying probability distribution of a dataset p(x), where x represents the data points in the dataset.

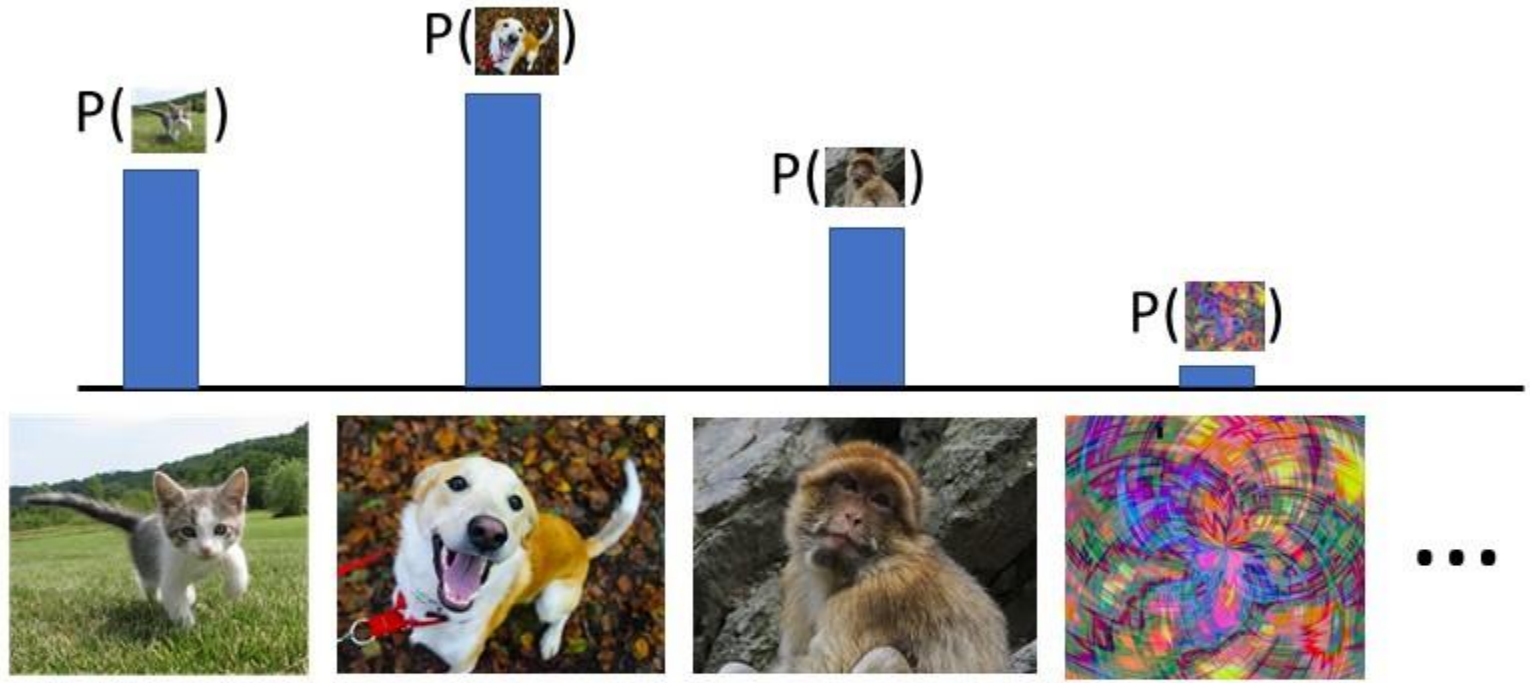
Incase of image dataset x is an image

Discriminative vs Generative Models

Discriminative Model: Learn a probability distribution p(y|x)

Generative Model: Learn a probability distribution p(x)

Conditional Generative Model: Learn p(x|y)



Generative model: learn the underlying probability distribution p(x), means

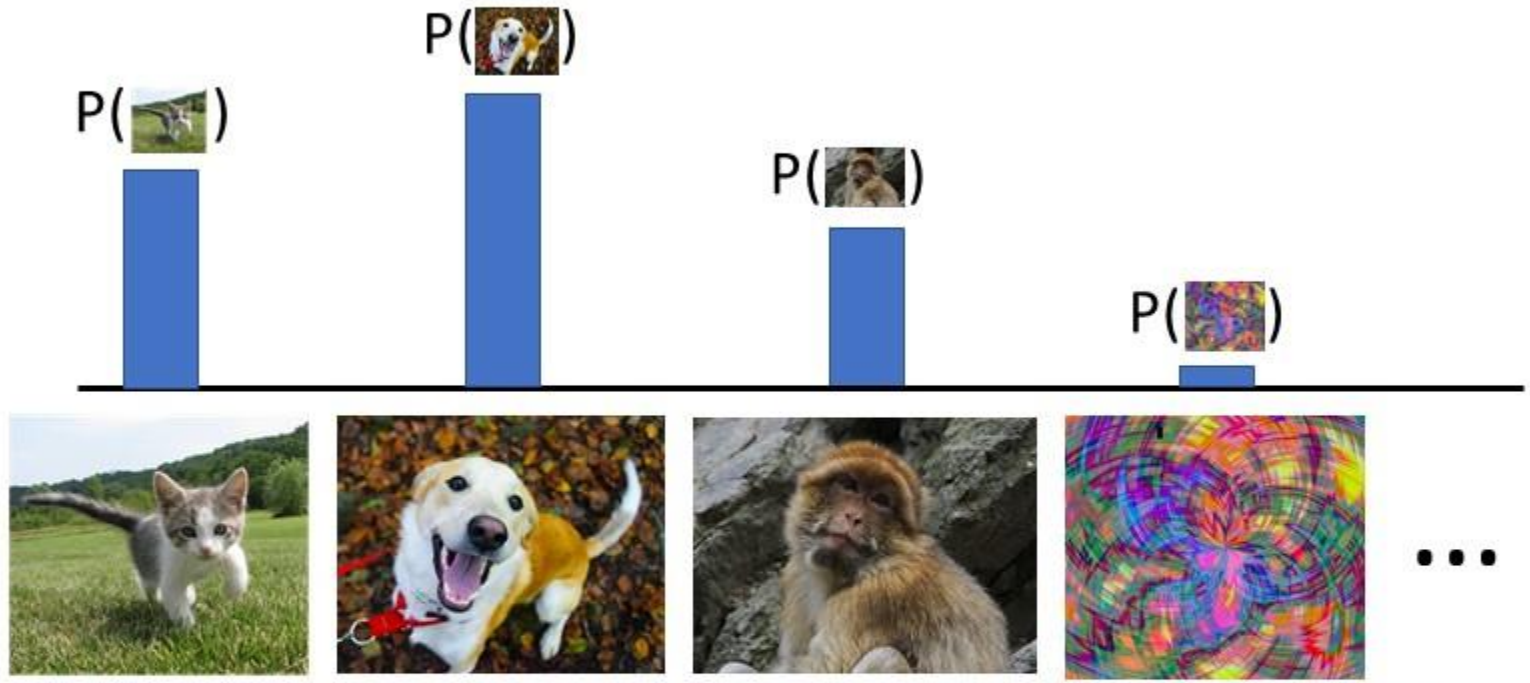
the model is **trained to understand and replicate** the complex probability structure of the input data.

Discriminative vs Generative Models

Discriminative Model: Learn a probability distribution p(y|x)

Generative Model: Learn a probability distribution p(x)

Conditional Generative Model: Learn p(x|y)



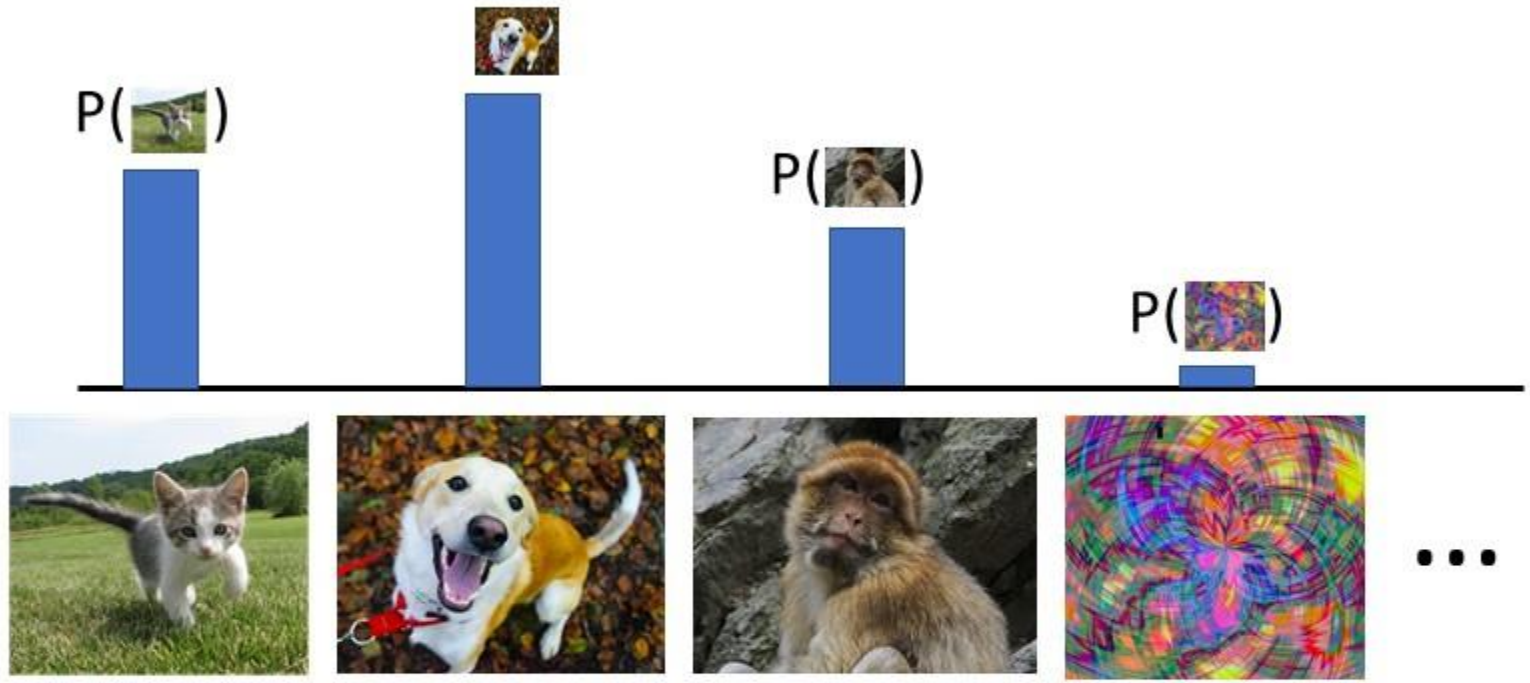
Generative model: By learning probability distribution p(x), these models can **simulate or generate new data points** that have similar statistical properties as the observed data..

Discriminative vs Generative Models

Discriminative Model: Learn a probability distribution p(y|x)

Generative Model: Learn a probability distribution p(x)

Conditional Generative Model: Learn p(x|y)



Generative model: All possible images compete with each other for probability mass

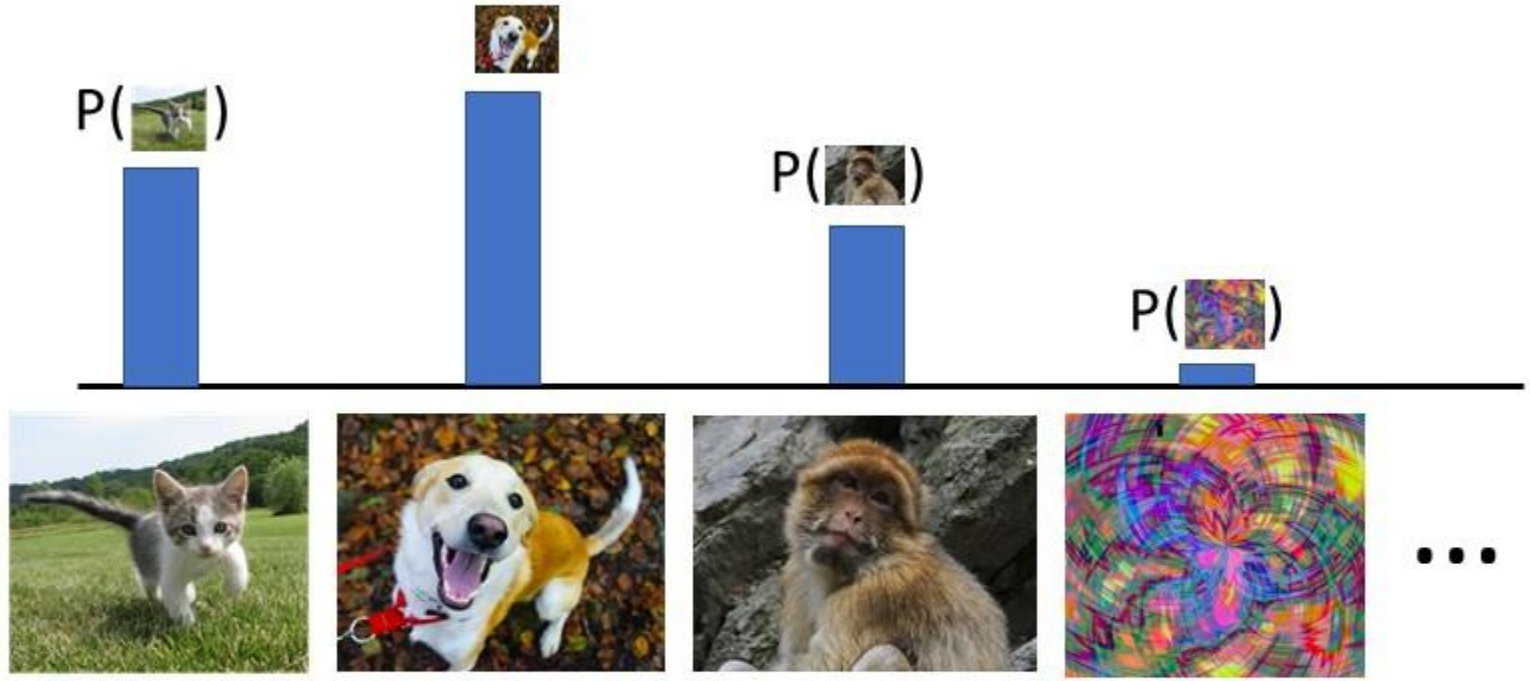
Requires deep image understanding! Is a dog more likely to sit or stand? How about 3-legged dog vs 3-armed monkey?

Discriminative vs Generative Models

Discriminative Model: Learn a probability distribution p(y|x)

Generative Model: Learn a probability distribution p(x)

Conditional Generative Model: Learn p(x|y)



Generative model: All possible images compete with each other for probability mass

Model can “reject” unreasonable inputs by assigning them small values

Discriminative vs Generative Models

Discriminative Model: Learn a probability distribution p(y|x)

Generative Model: Learn a probability distribution p(x)

Conditional Generative Model: Learn p(x|y)

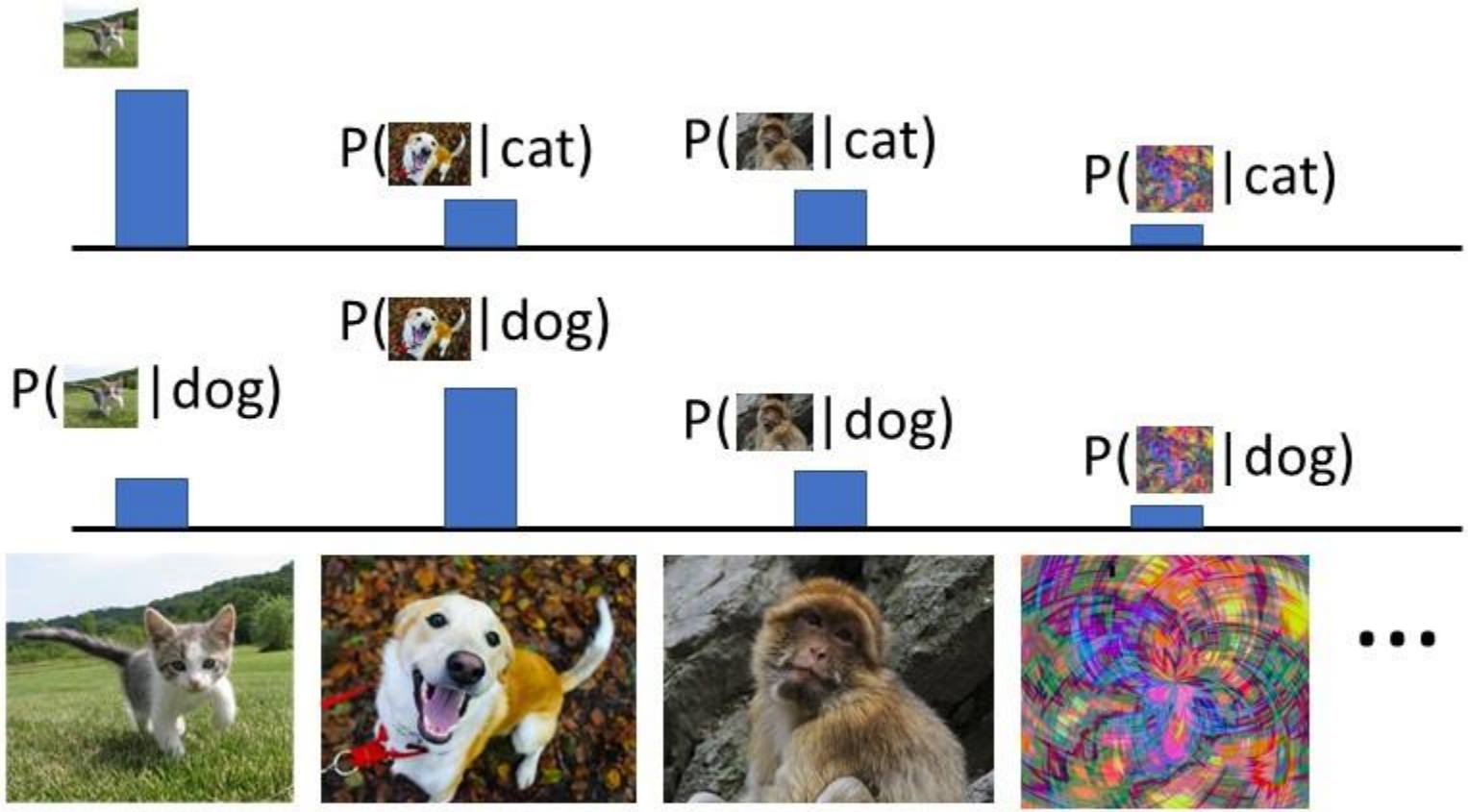
- The model learns the conditional probability distribution p(x|y), where x is the data and y is the condition or label.
- In these models, learning involves understanding how the characteristics of data change with different conditions or labels.
- This could be anything from a simple category label (like generating images of a particular class) to more complex data types (like text descriptions). .

Discriminative vs Generative Models

Discriminative Model: Learn a probability distribution p(y|x)

Generative Model: Learn a probability distribution p(x)

Conditional Generative Model: Learn p(x|y)



Conditional Generative Model: Each possible label induces a different competition among all images (different distribution)

Discriminative vs Generative Models

Discriminative Model: Learn a probability distribution p(y|x)

Generative Model: Learn a probability distribution p(x)

Conditional Generative Model: Learn p(x|y)

Recall **Bayes' Rule:**

$$P(x \mid y) = \frac{P(y \mid x)}{P(y)} P(x)$$

Discriminative vs Generative Models

Discriminative Model: Learn a probability distribution p(y|x)

Generative Model: Learn a probability distribution p(x)

Conditional Generative Model: Learn p(x|y)

Recall **Bayes' Rule:**

$$P(x \mid y) = \frac{P(y \mid x)}{P(y)} P(x)$$

Conditional Generative Model

Discriminative Model

Prior over labels

(Unconditional) Generative Model

We can build a conditional generative model from other components!

What can we do with a discriminative model?

Discriminative Model:
Learn a probability distribution p(y|x)

→ Assign labels to data
Feature learning (with labels)

Generative Model:
Learn a probability distribution p(x)

Conditional Generative Model: Learn p(x|y)

What can we do with a discriminative model?

Discriminative Model: Learn a probability distribution p(y|x) → Assign labels to data
Feature learning (with labels)

Generative Model: Learn a probability distribution p(x) → Generative models aim to learn the overall probability distribution of the dataset and it detects outliers (has a very low probability under the model's distribution) easily enables

Conditional Generative Model: Learn p(x|y)

What can we do with a discriminative model?

Discriminative Model:
Learn a probability distribution p(y|x)

→ Assign labels to data
Feature learning (with labels)

Generative Model:
Learn a probability distribution p(x)

→ Detect outliers
Feature learning (without labels)
Sample to **generate** new data

Conditional Generative Model: Learn p(x|y)

What can we do with a discriminative model?

Discriminative Model: Learn a probability distribution p(y|x) → Assign labels to data; Feature learning (with labels)

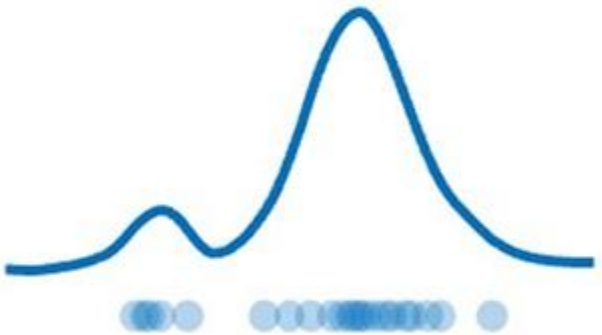
Generative Model: Learn a probability distribution p(x) → Detect outliers; Feature learning (without labels); Sample to **generate** new data

Conditional Generative Model: Learn p(x|y) → Assign labels, while rejecting outliers!; Generate new data conditioned on input labels

Generative Modeling

Goal: Take samples x from some distribution (unknown) $p_{data}(x)$ as training input learn a model (estimated) $p_{model}(x)$ that represents that distribution

Density Estimation



Sample Generation

Input samples

Training data ~ $P_{data}(x)$

Generated samples

Generated ~ $P_{model}(x)$

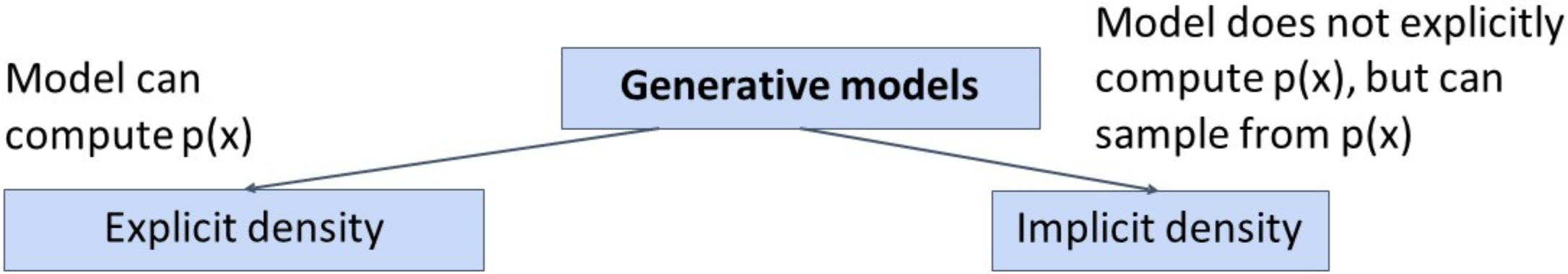
Generative Model learns $p_{model}(x)$, an estimate of actual probability distribution of data (unknown) $p_{data}(x)$ through samples x.

Taxonomy of Generative Models

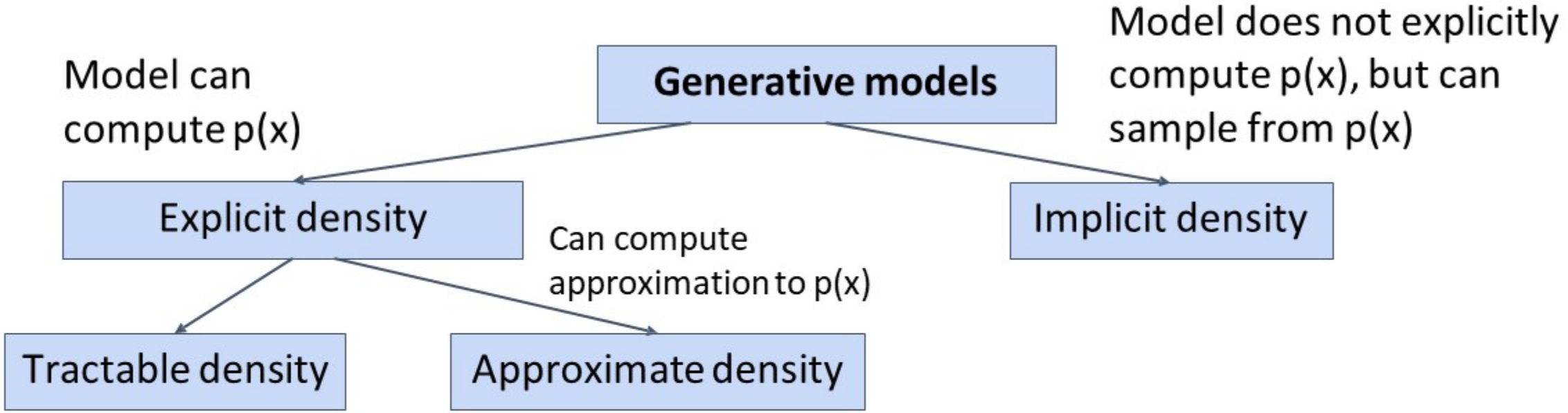
Generative models

Figure adapted from Ian Goodfellow, Tutorial on Generative Adversarial Networks, 2017.

Taxonomy of Generative Models



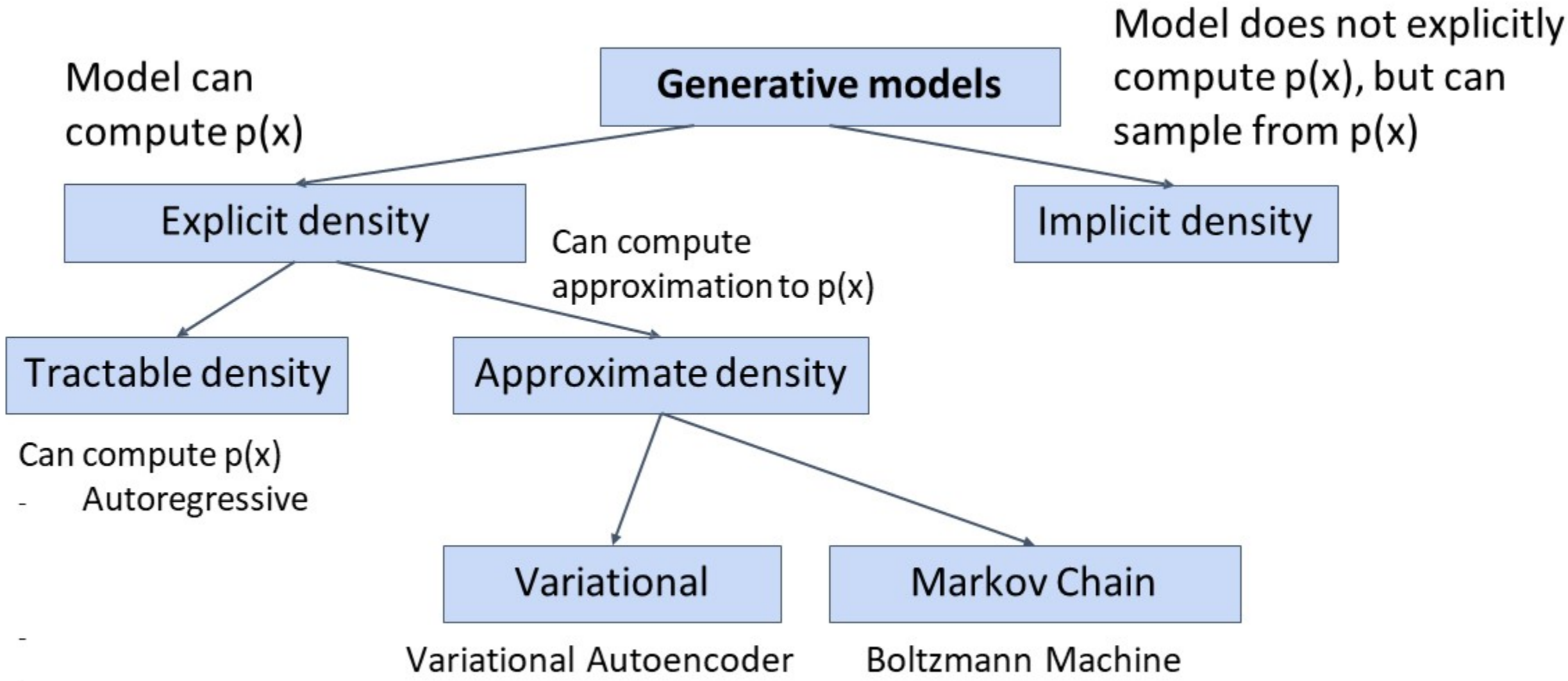
Taxonomy of Generative Models



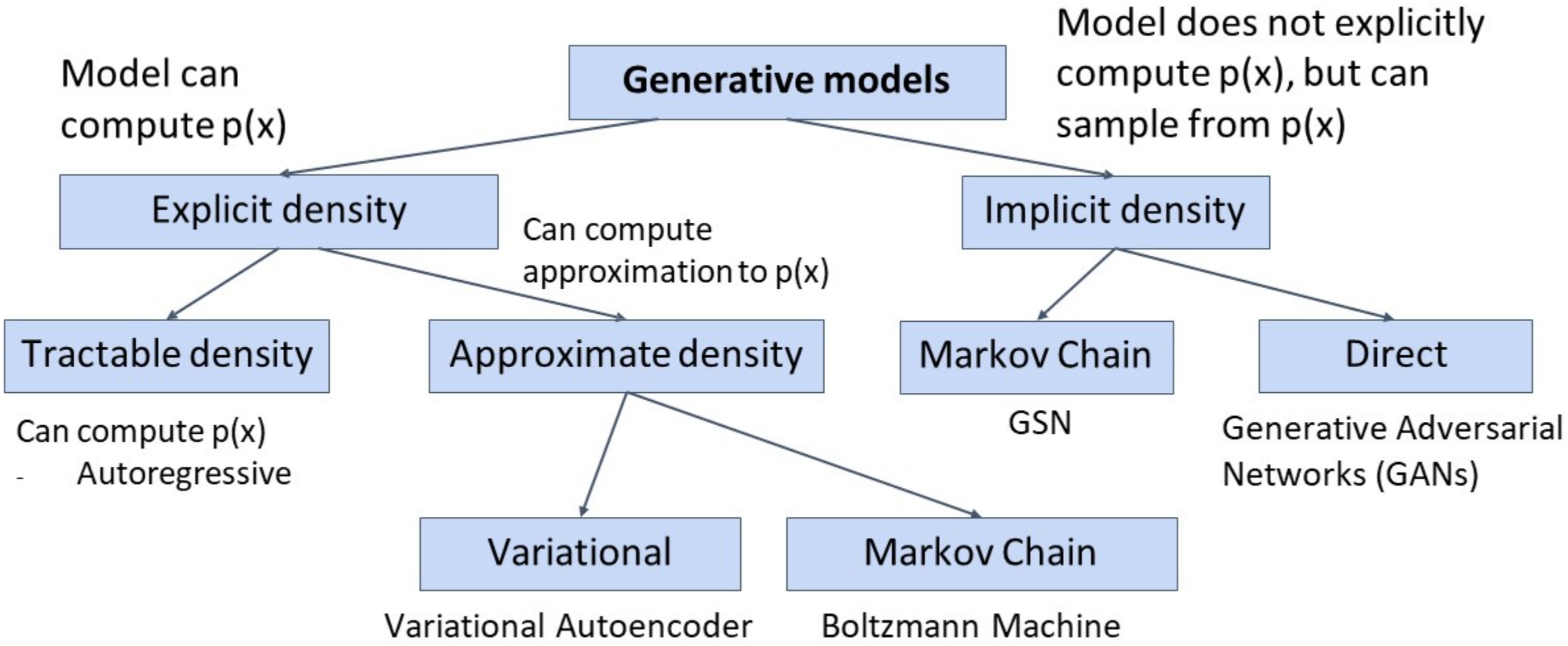
Can compute p(x)

- Autoregressive

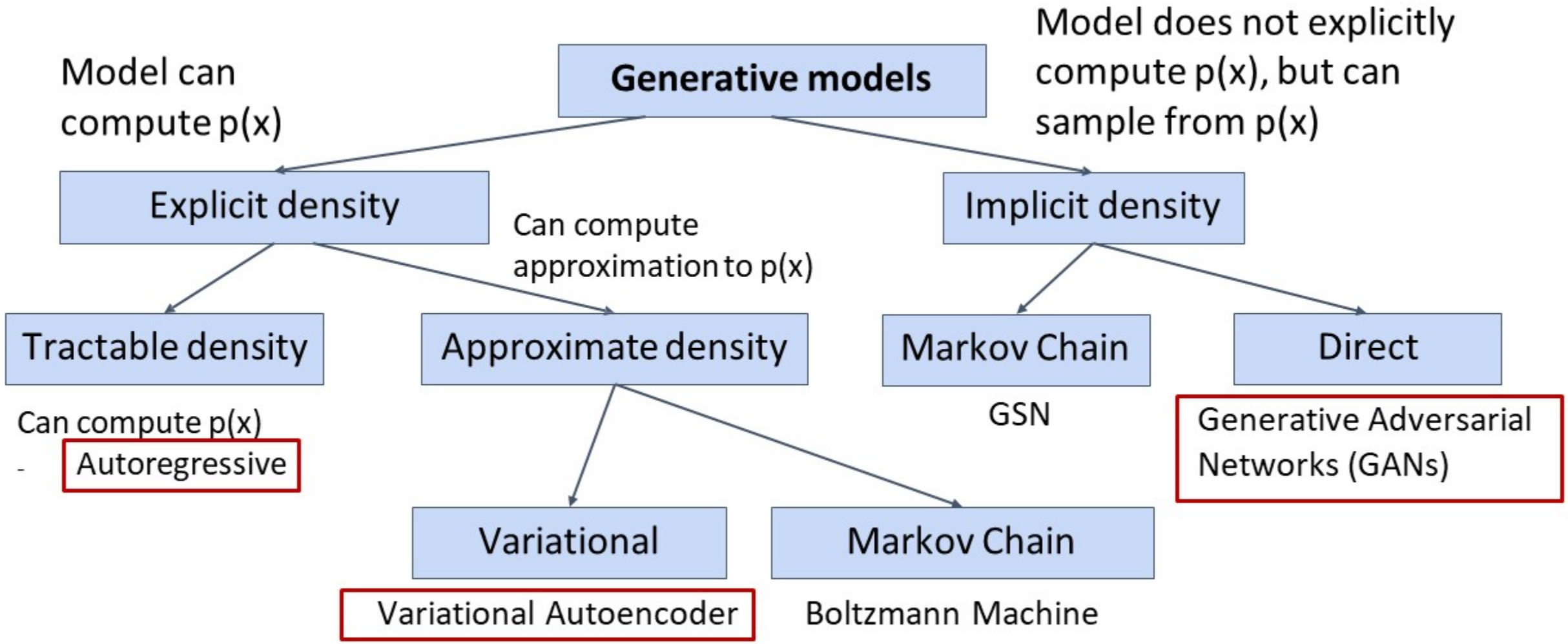
Taxonomy of Generative Models



Taxonomy of Generative Models



Taxonomy of Generative Models



Autoregressive models

Explicit Density Estimation

Goal: Write down an explicit function for $p(x) = f(x, W)$

Explicit Density Estimation

Goal: Write down an explicit function for $p(x) = f(x, W)$

Given dataset $x^{(1)}, x^{(2)}, \ldots x^{(N)}$, train the model by solving:

$$W^* = \arg\max_W \prod_i p(x^i)$$

Maximize probability of training data
(Maximum likelihood estimation)

Note log is monotonically increasing function thus maxima point (W^*) remains same.
Use log to convert product to sum:

$$W^* = \arg\max_W \sum_i \log(p(x^i))$$

Explicit Density Estimation

Goal: Write down an explicit function for $p(x) = f(x, W)$

Given dataset $x^{(1)}, x^{(2)}, \ldots x^{(N)}$, train the model by solving:

$$W^* = \arg\max_W \prod_i p(x^i)$$

Maximize probability of training data
(Maximum likelihood estimation)

Note log is monotonically increasing function thus maxima point (W^*) remains same.
Use log to convert product to sum:

$$W^* = \arg\max_W \sum_i \log(f(p(x^i), W))$$

This will be our loss function!
Train with gradient descent

Explicit Density: Autoregressive Models

Goal: Write down an explicit function for $p(x) = f(x, W)$

Assume x consists of multiple subparts:

$$x = (x_1, x_2, x_3, \ldots, x_T)$$

Explicit Density: Autoregressive Models

Goal: Write down an explicit function for $p(x) = f(x, W)$

Assume x consists of multiple subparts:

$$x = (x_1, x_2, x_3, \ldots, x_T)$$

Break down probability using the chain rule:

$$p(x) = p(x_1, x_2, x_3, \ldots, x_T)$$

$$p(x) = p(x_1)p(x_2|x_1)p(x_3|x_1, x_2)\ldots$$

$$p(x) = \prod_{t=1}^{T} p(x_t|x_1, x_2, \ldots, x_{t-1})$$

Probability of the next subpart given all the previous subparts

Explicit Density: Autoregressive Models

Goal: Write down an explicit function for $p(x) = f(x, W)$

Assume x consists of multiple subparts:

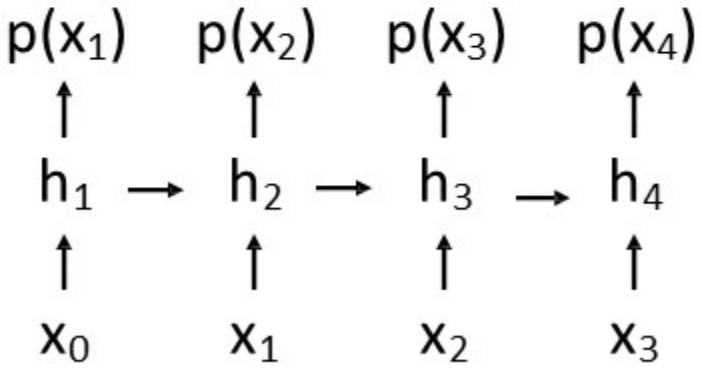
$$x = (x_1, x_2, x_3, \ldots, x_T)$$

Break down probability using the chain rule:

$$p(x) = p(x_1, x_2, x_3, \ldots, x_T)$$

$$p(x) = p(x_1)p(x_2|x_1)p(x_3|x_1, x_2)\ldots$$

$$p(x) = \prod_{t=1}^{T} p(x_t|x_1, x_2, \ldots, x_{t-1})$$



We've already seen this! Language modeling with an RNN!

Probability of the next subpart given all the previous subparts

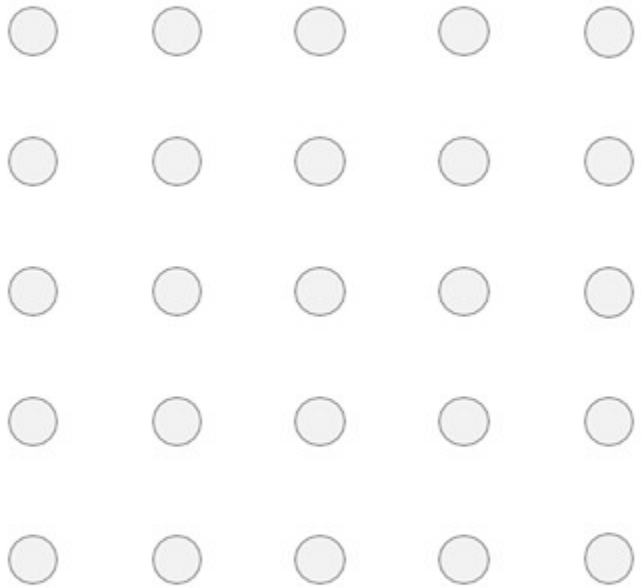
PixelRNN

Generate image pixels one at a time, starting at the upper left corner

Compute a hidden state for each pixel that depends on hidden states and RGB values from the left and from above (LSTM recurrence)

$$h_{x,y} = f(h_{x-1,y}, h_{x,y-1}, W)$$

At each pixel, predict red, then blue, then green: softmax over [0, 1, …, 255]



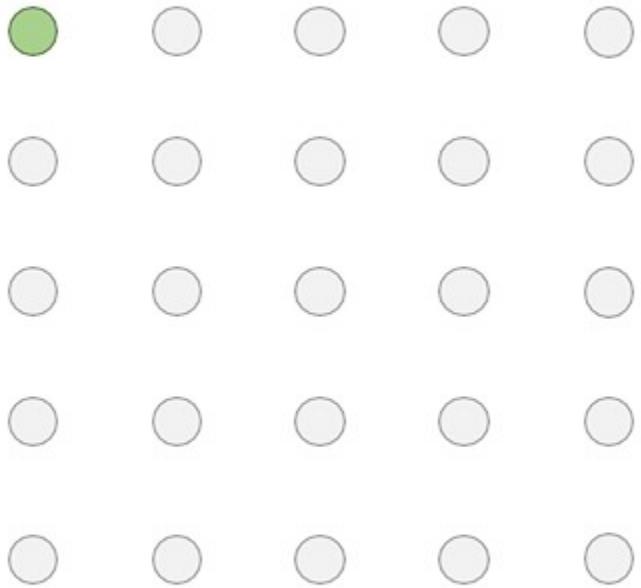
PixelRNN

Generate image pixels one at a time, starting at the upper left corner

Compute a hidden state for each pixel that depends on hidden states and RGB values from the left and from above (LSTM recurrence)

$$h_{x,y} = f(h_{x-1,y}, h_{x,y-1}, W)$$

At each pixel, predict red, then blue, then green: softmax over [0, 1, …, 255]



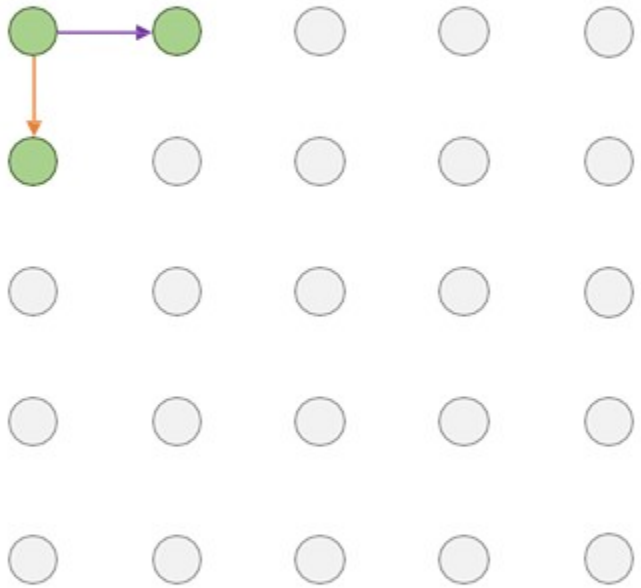
PixelRNN

Generate image pixels one at a time, starting at the upper left corner

Compute a hidden state for each pixel that depends on hidden states and RGB values from the left and from above (LSTM recurrence)

$$h_{x,y} = f(h_{x-1,y}, h_{x,y-1}, W)$$

At each pixel, predict red, then blue, then green: softmax over [0, 1, …, 255]



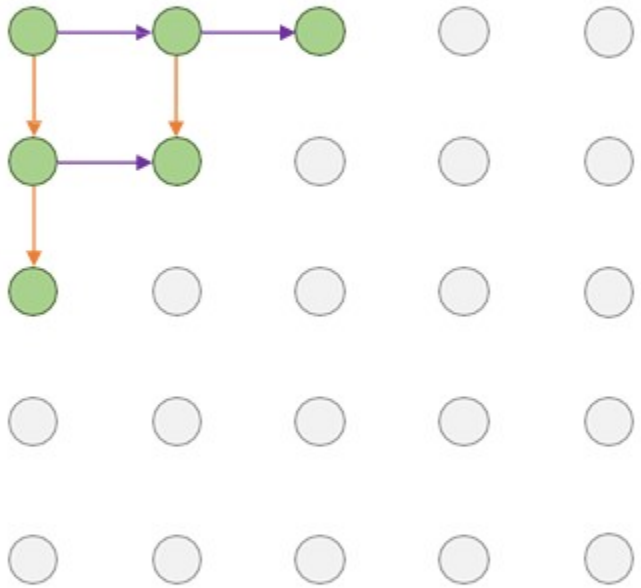
PixelRNN

Generate image pixels one at a time, starting at the upper left corner

Compute a hidden state for each pixel that depends on hidden states and RGB values from the left and from above (LSTM recurrence)

$$h_{x,y} = f(h_{x-1,y}, h_{x,y-1}, W)$$

At each pixel, predict red, then blue, then green: softmax over [0, 1, …, 255]



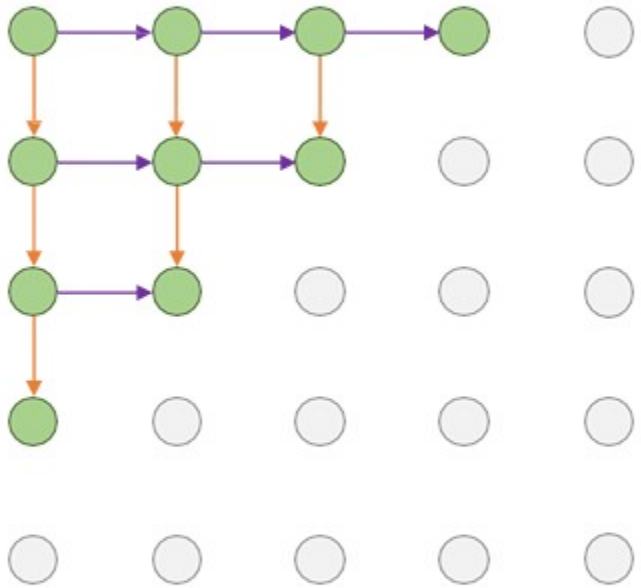
PixelRNN

Generate image pixels one at a time, starting at the upper left corner

Compute a hidden state for each pixel that depends on hidden states and RGB values from the left and from above (LSTM recurrence)

$$h_{x,y} = f(h_{x-1,y}, h_{x,y-1}, W)$$

At each pixel, predict red, then blue, then green: softmax over [0, 1, …, 255]



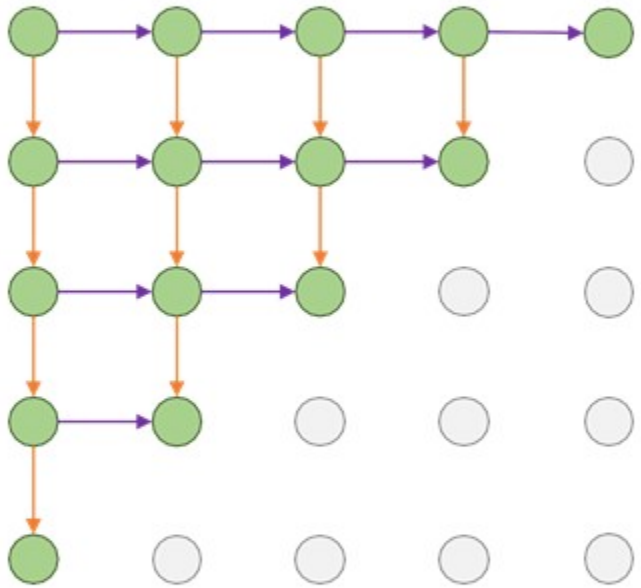
PixelRNN

Generate image pixels one at a time, starting at the upper left corner

Compute a hidden state for each pixel that depends on hidden states and RGB values from the left and from above (LSTM recurrence)

$$h_{x,y} = f(h_{x-1,y}, h_{x,y-1}, W)$$

At each pixel, predict red, then blue, then green: softmax over [0, 1, …, 255]



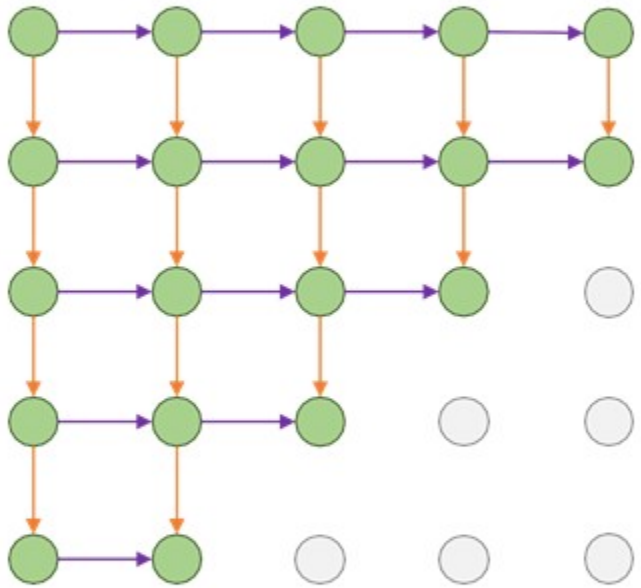
PixelRNN

Generate image pixels one at a time, starting at the upper left corner

Compute a hidden state for each pixel that depends on hidden states and RGB values from the left and from above (LSTM recurrence)

$$h_{x,y} = f(h_{x-1,y}, h_{x,y-1}, W)$$

At each pixel, predict red, then blue, then green: softmax over [0, 1, …, 255]



PixelRNN

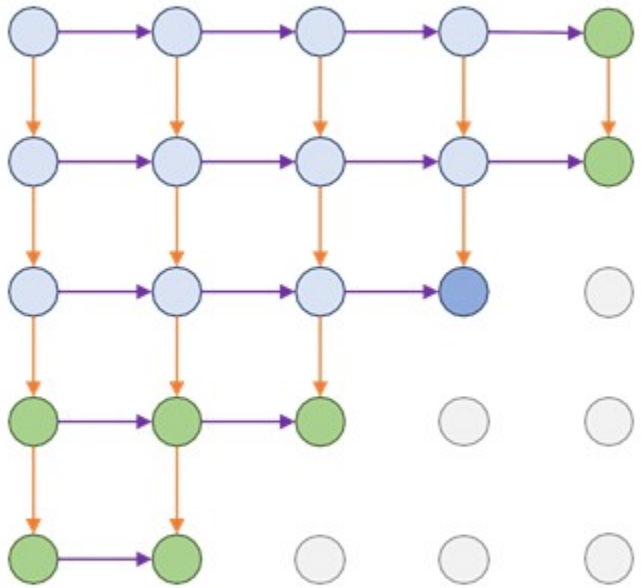
Generate image pixels one at a time, starting at the upper left corner

Compute a hidden state for each pixel that depends on hidden states and RGB values from the left and from above (LSTM recurrence)

$$h_{x,y} = f(h_{x-1,y}, h_{x,y-1}, W)$$

At each pixel, predict red, then blue, then green: softmax over [0, 1, …, 255]

Each pixel depends **implicity** on all pixels above and to the left:



PixelRNN

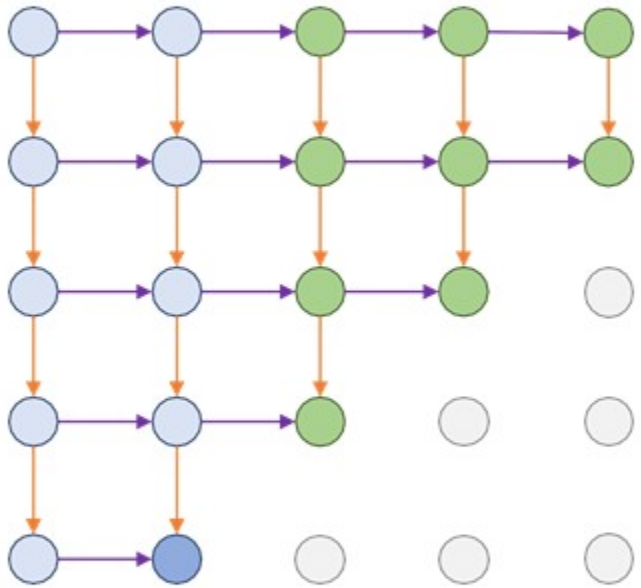
Generate image pixels one at a time, starting at the upper left corner

Compute a hidden state for each pixel that depends on hidden states and RGB values from the left and from above (LSTM recurrence)

$$h_{x,y} = f(h_{x-1,y}, h_{x,y-1}, W)$$

At each pixel, predict red, then blue, then green: softmax over [0, 1, …, 255]

Each pixel depends **implicity** on all pixels above and to the left:



PixelRNN

Generate image pixels one at a time, starting at the upper left corner

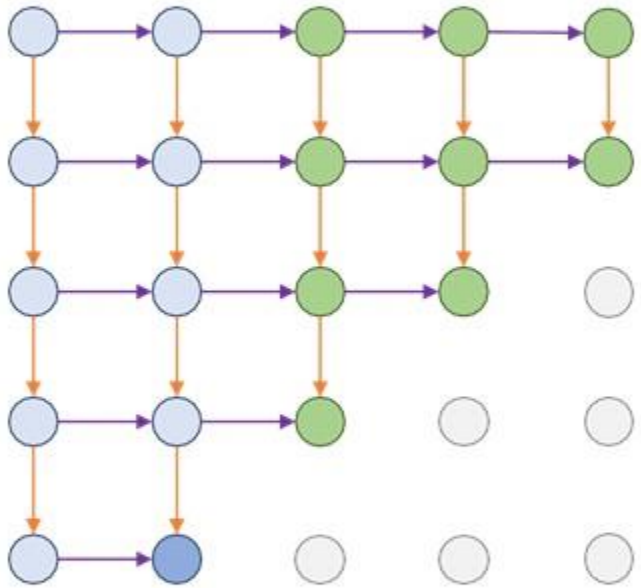
Compute a hidden state for each pixel that depends on hidden states and RGB values from the left and from above (LSTM recurrence)

$$h_{x,y} = f(h_{x-1,y}, h_{x,y-1}, W)$$

At each pixel, predict red, then blue, then green: softmax over [0, 1, …, 255]

Each pixel depends **implicity** on all pixels above and to the left:

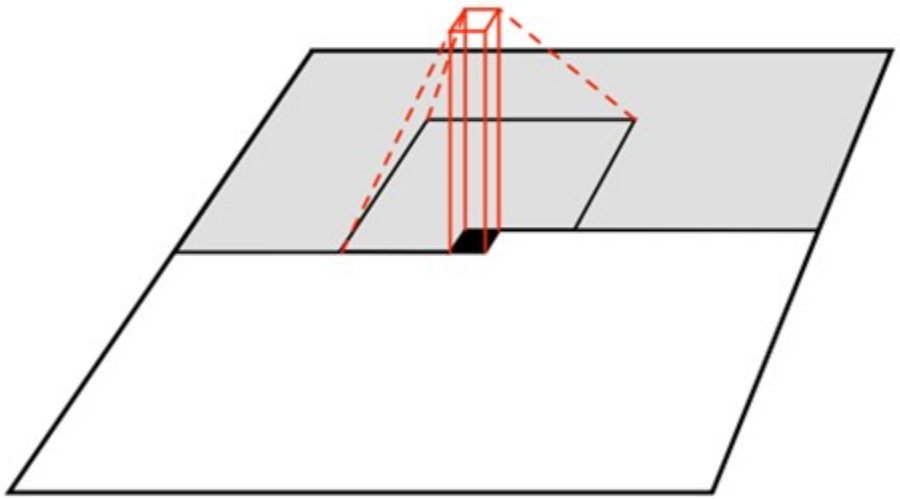
Problem: Very slow during both training and testing; N x N image requires 2N-1 sequential steps



PixelCNN

Still generate image pixels starting from corner

Dependency on previous pixels now modeled using a CNN over context region



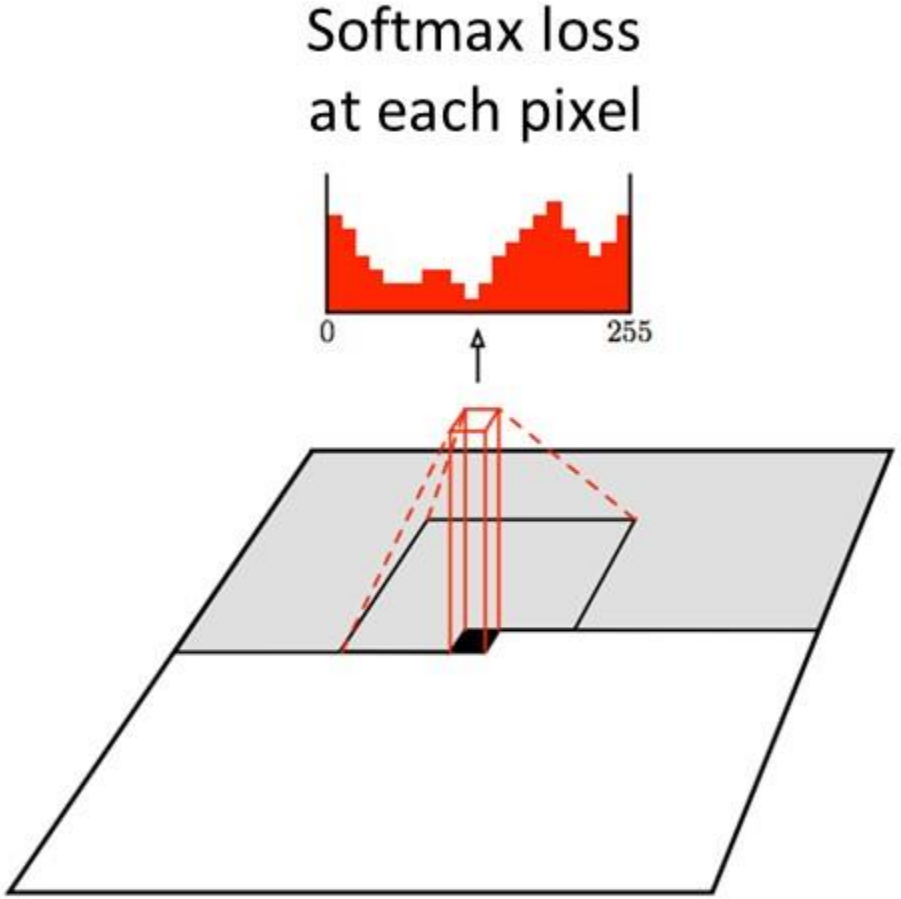
PixelCNN

Still generate image pixels starting from corner

Dependency on previous pixels now modeled using a CNN over context region

Training: maximize likelihood of training images

$$p(x) = \prod_{i=1}^{n} p(x_i | x_1, \ldots, x_{i-1})$$



PixelCNN

Still generate image pixels starting from corner

Dependency on previous pixels now modeled using a CNN over context region

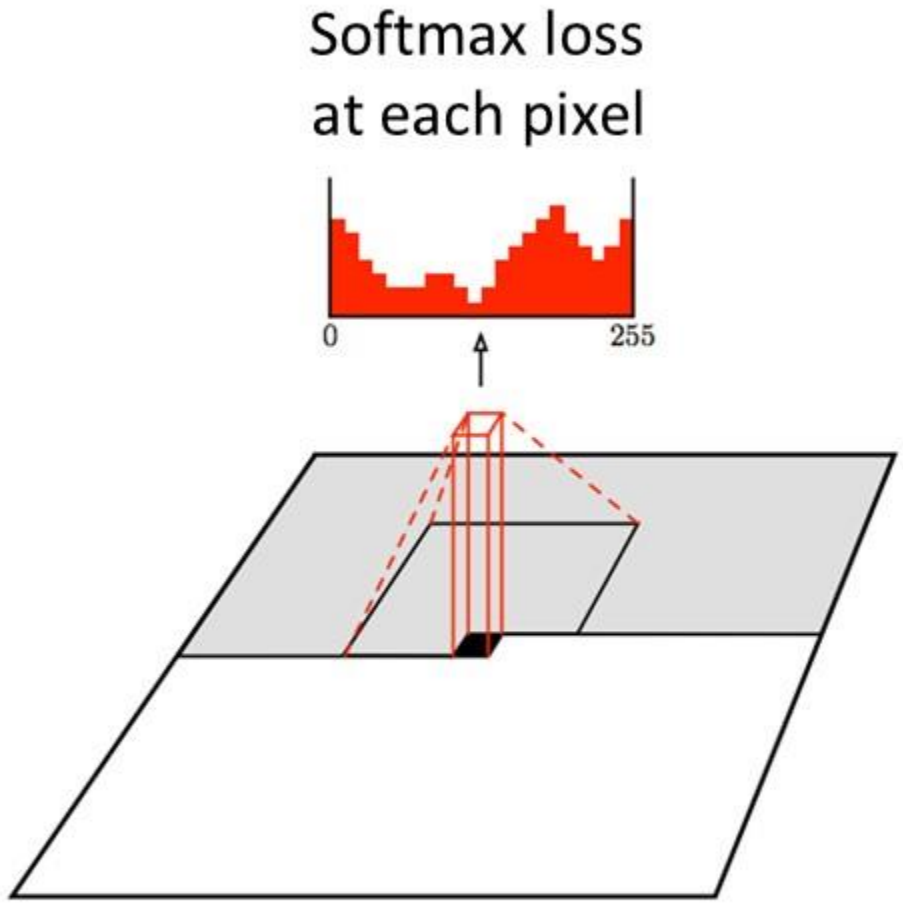
Training: maximize likelihood of training images

Training is faster than PixelRNN

(can parallelize convolutions since context region values known from training images)

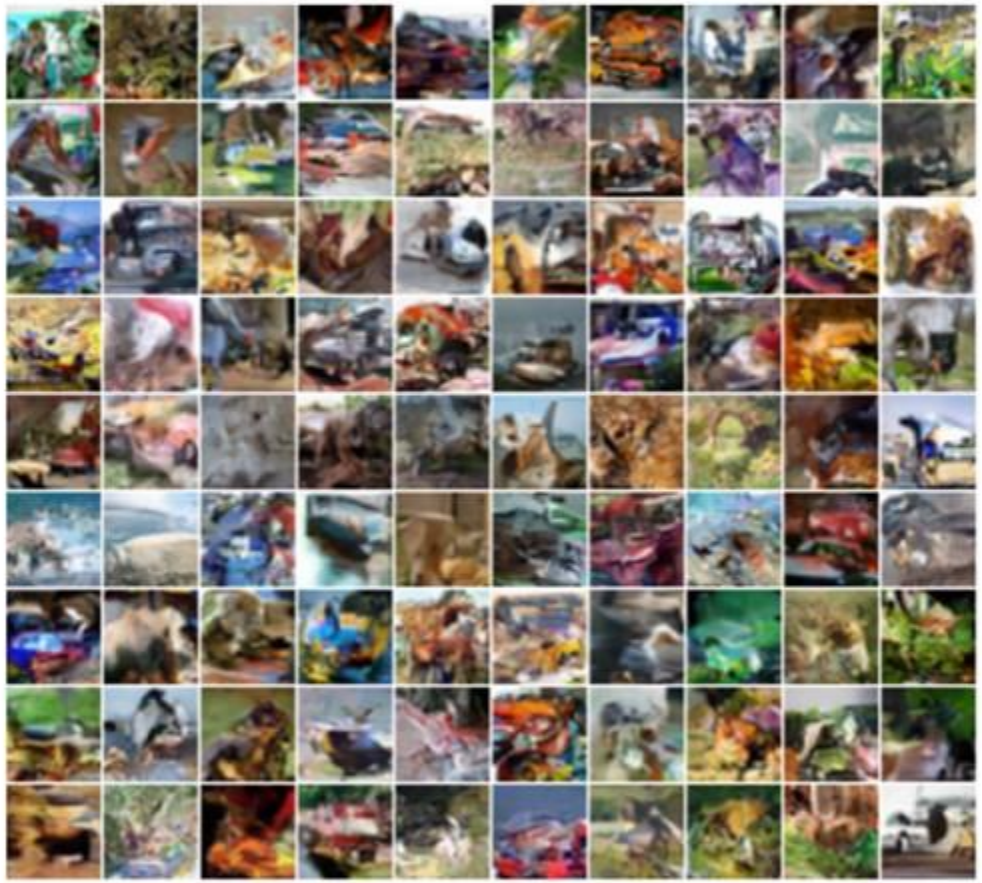
Generation must still proceed sequentially

=> still slow

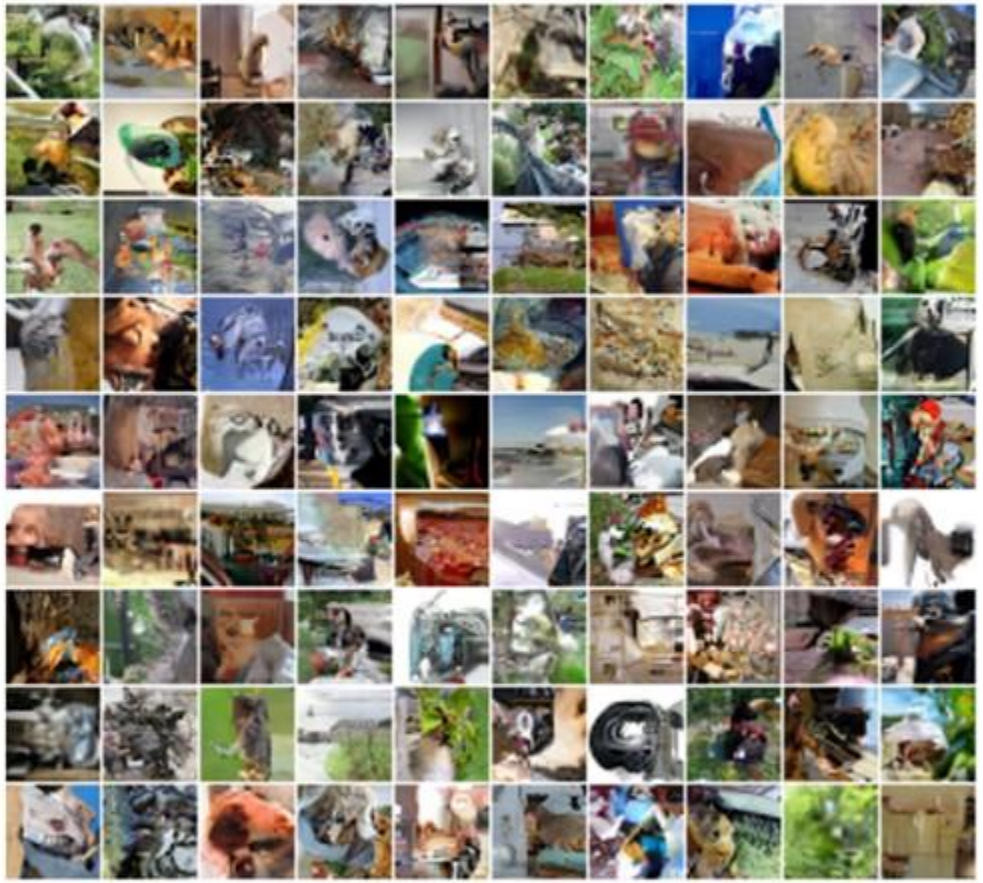


Van den Oord et al, "Conditional Image Generation with PixelCNN Decoders", NeurIPS 2016

PixelRNN: Generated Samples



32x32 CIFAR-10



32x32 ImageNet

Van den Oord et al, “Pixel Recurrent Neural Networks”, ICML 2016

Autoregressive Models: PixelRNN and PixelCNN

Pros:

- Can explicitly compute likelihood p(x)
- Explicit likelihood of training data gives good evaluation metric
- Good samples

Con:

- Sequential generation => slow

Improving PixelCNN performance

- Gated convolutional layers
- Short-cut connections
- Discretized logistic loss
- Multi-scale
- Training tricks
- Etc…

See

- Van der Oord et al. NIPS 2016
- Salimans et al. 2017 (PixelCNN++)

Variational Autoencoders

Variational Autoencoders

PixelRNN / PixelCNN explicitly parameterizes density function with a neural network, so we can train to maximize likelihood of training data:

$$p_W(x) = \prod_{t=1}^{T} p_W(x_t | x_1, x_2, \ldots, x_{t-1})$$

Variational Autoencoders (VAE) define an **intractable density** that we cannot explicitly compute or optimize

But we will be able to directly optimize a **lower bound** on the density

Variational Autoencoders

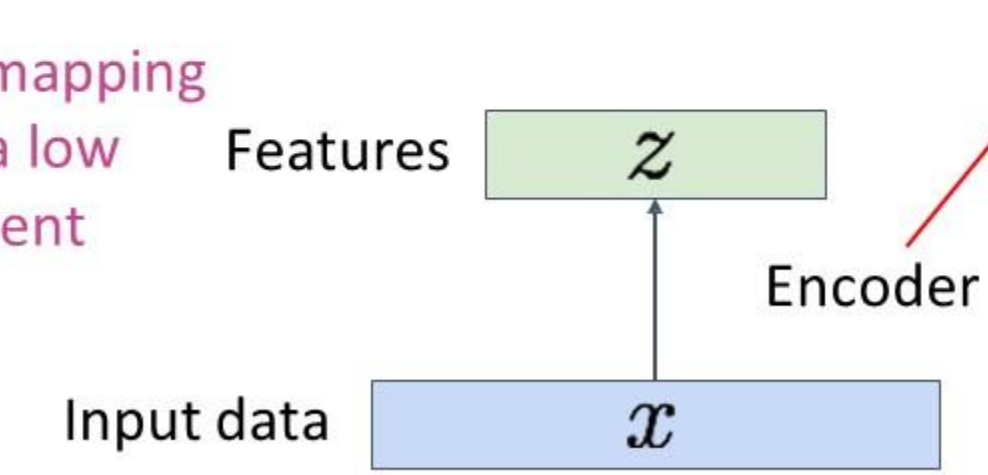
(Regular, non-variational) Autoencoders

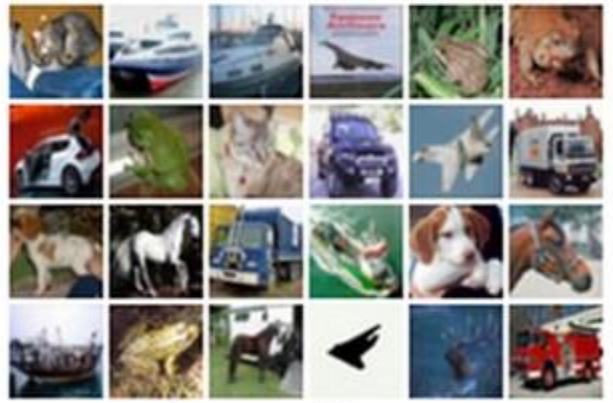
Unsupervised method for learning feature vectors from raw data x, without any labels

Features should extract useful information (maybe object identities, properties, scene type, etc) that we can use for downstream tasks

Originally: Linear + nonlinearity (sigmoid)
Later: Deep, fully-connected
Later: ReLU CNN

Encoder leans mapping from data x to a low dimensional latent space z





Input Data

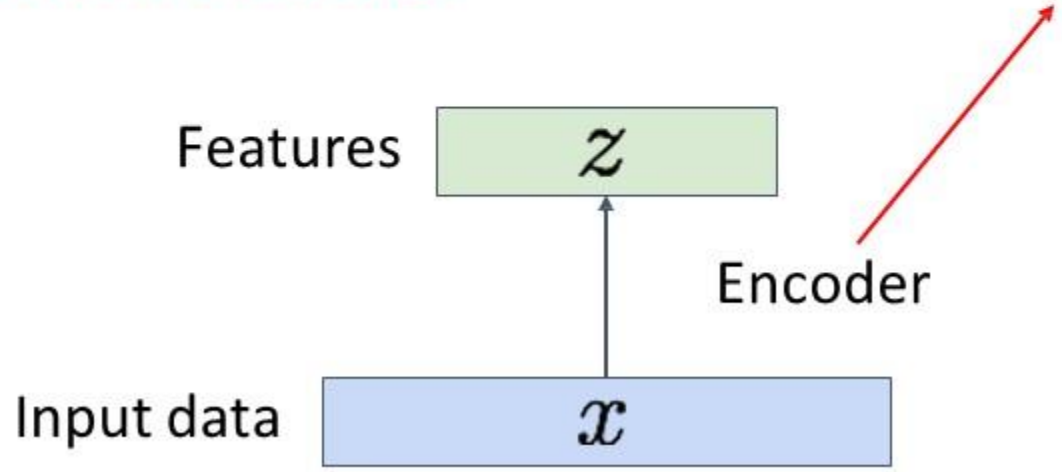
(Regular, non-variational) Autoencoders

Problem: How can we learn this feature transform from raw data?

Features should extract useful information (maybe object identities, properties, scene type, etc) that we can use for downstream tasks

But we can't observe features!

Originally: Linear + nonlinearity (sigmoid)
Later: Deep, fully-connected
Later: ReLU CNN



Input Data

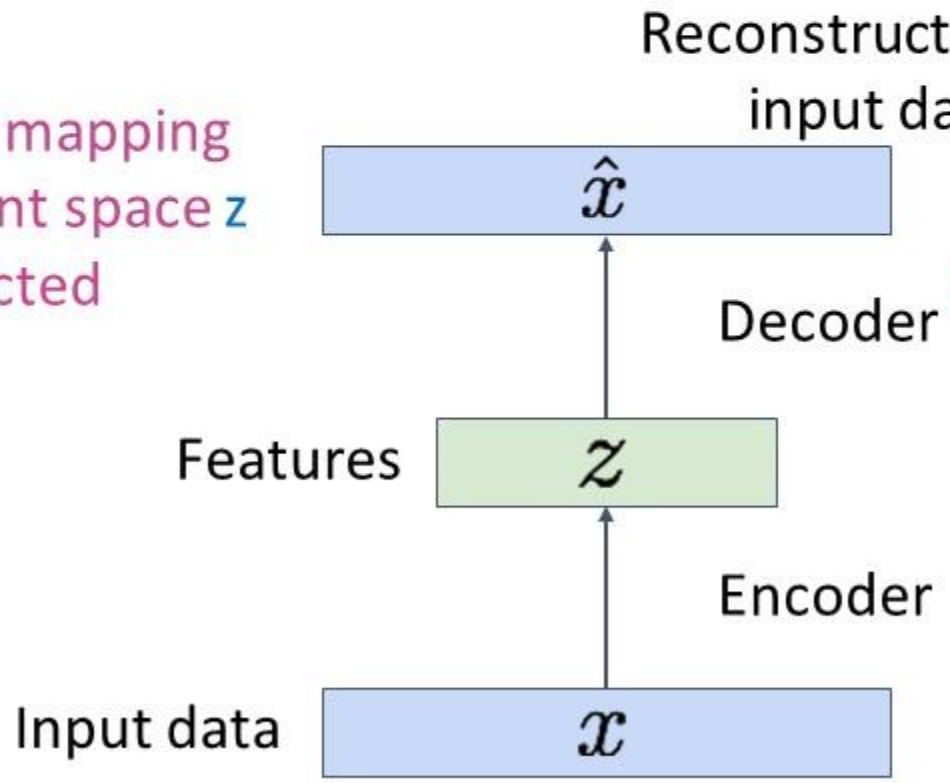
(Regular, non-variational) Autoencoders

Problem: How can we learn this feature transform from raw data?

Idea: Use the features to reconstruct the input data with a **decoder**

"Autoencoding" = encoding itself

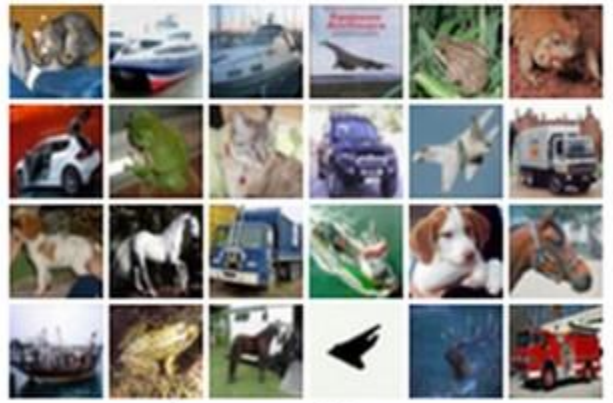
Decoder leans mapping back from latent space z to a reconstructed observation $\hat{x}$



Originally: Linear + nonlinearity (sigmoid)

Later: Deep, fully-connected

Later: ReLU CNN (upconv)

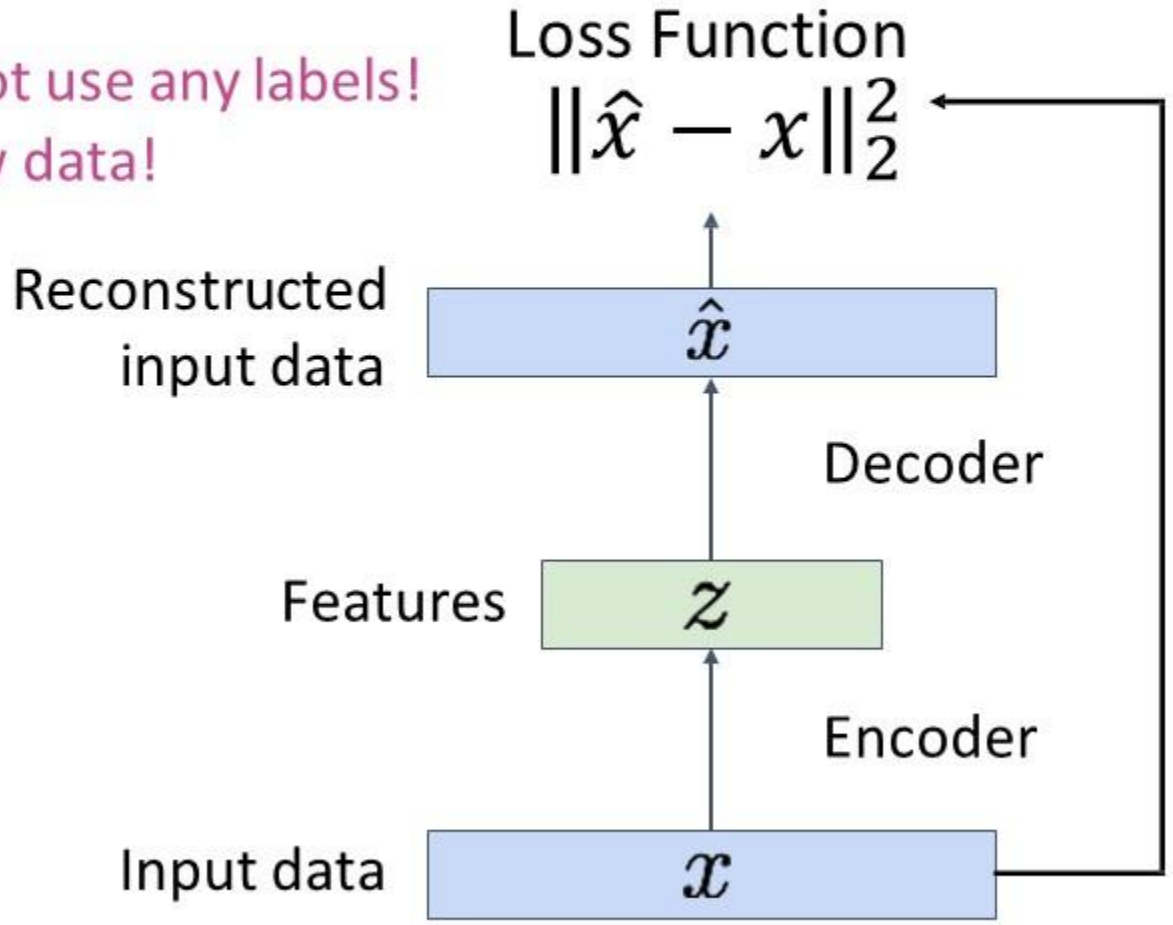


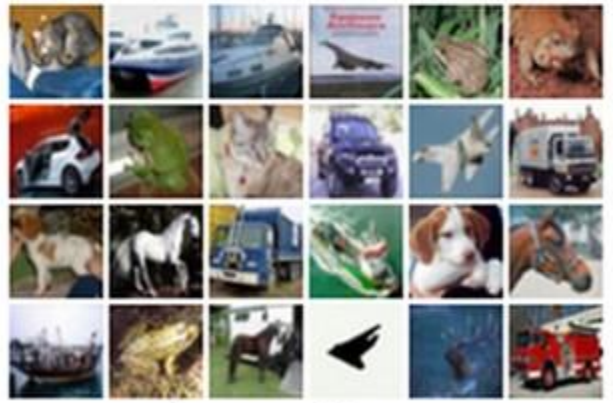
Input Data

(Regular, non-variational) Autoencoders

Loss: L2 distance between input and reconstructed data.

Does not use any labels!
Just raw data!





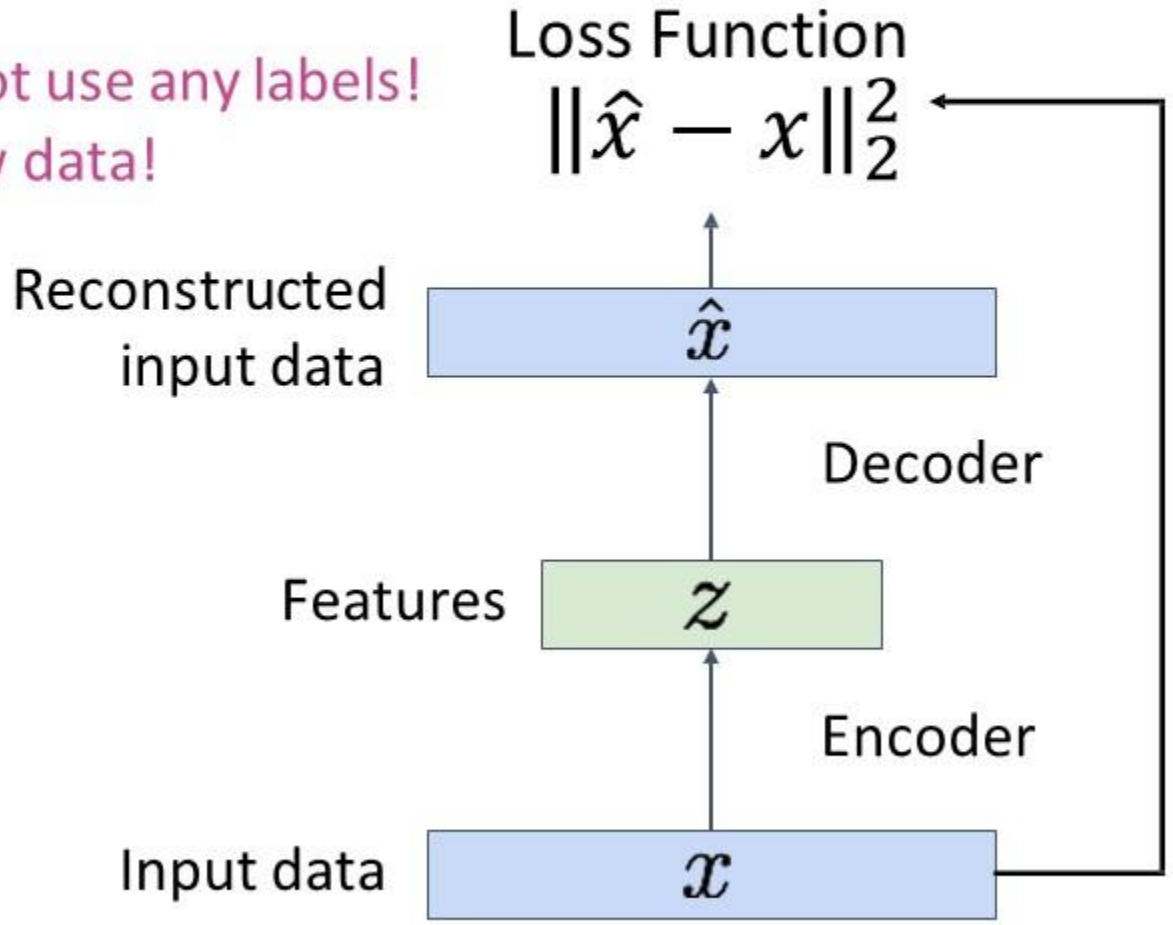
Input Data

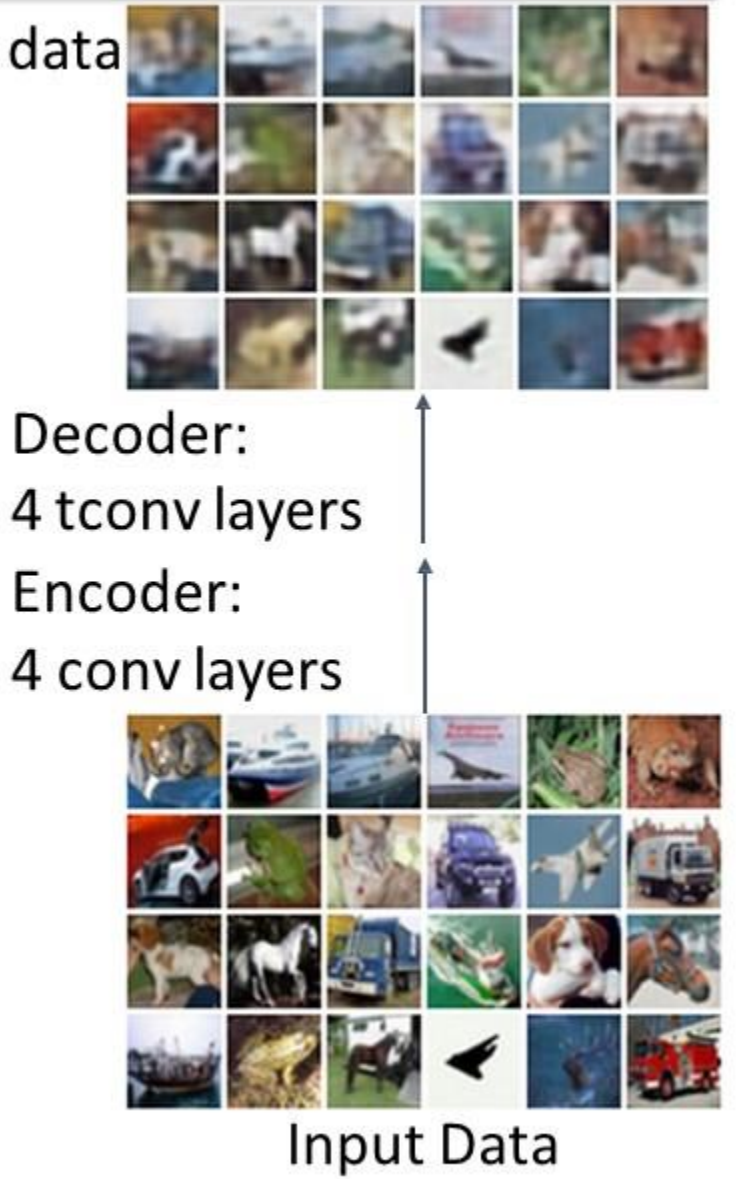
(Regular, non-variational) Autoencoders

Loss: L2 distance between input and reconstructed data.

Does not use any labels!

Just raw data!





(Regular, non-variational) Autoencoders

Loss: L2 distance between input and reconstructed data.

Does not use any labels!
Just raw data!

Loss Function
$\|\hat{x} - x\|_2^2$

Reconstructed input data
$\hat{x}$

Decoder

Features need to be **lower dimensional** than the data

Features
z

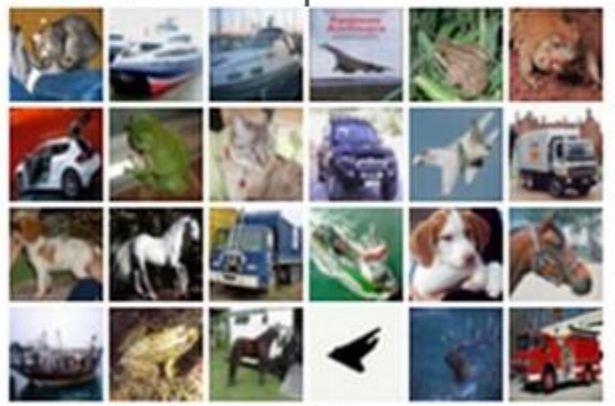
Encoder

Input data
x

Reconstructed data

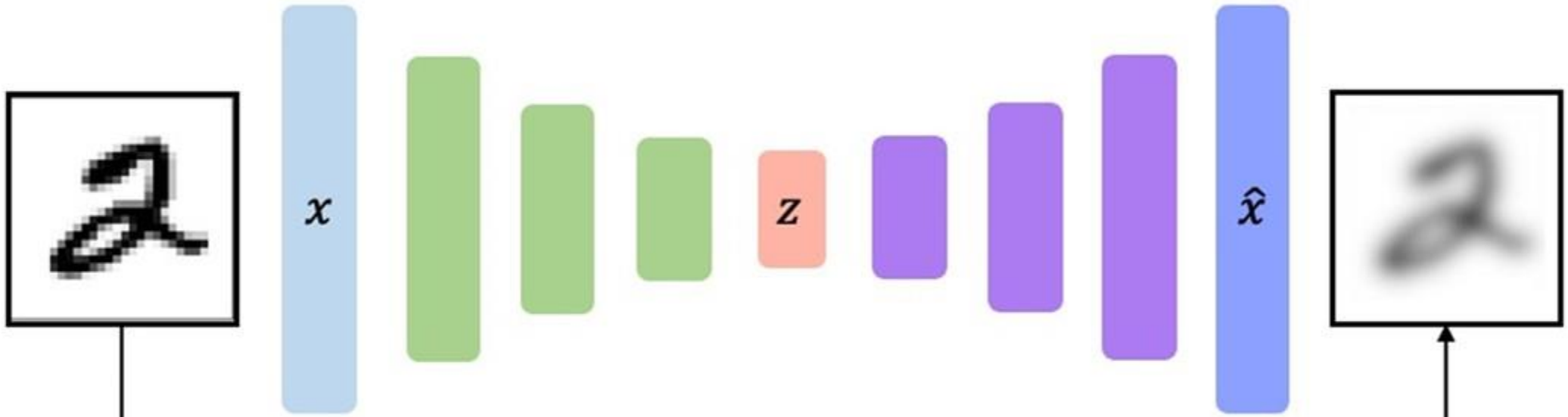
Decoder:
4 tconv layers

Encoder:
4 conv layers



Input Data

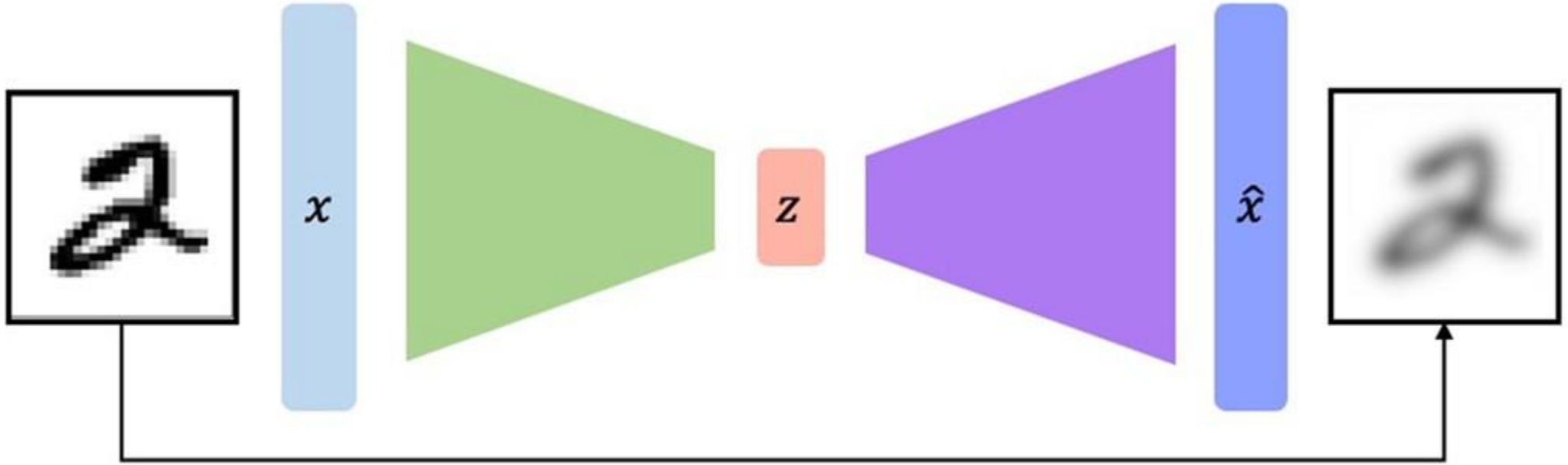
(Regular, non-variational) Autoencoders



$$\mathcal{L}(x, \hat{x}) = \|x - \hat{x}\|^2$$

Loss function doesn't use any labels!!

(Regular, non-variational) Autoencoders



$$\mathcal{L}(x, \hat{x}) = \|x - \hat{x}\|^2$$

Loss function doesn't use any labels!!

Dimensionality of Latent Space

Autoencoding is a form of compression!

Smaller latent space will force a larger training bottleneck

2D latent space

5D latent space

Ground Truth

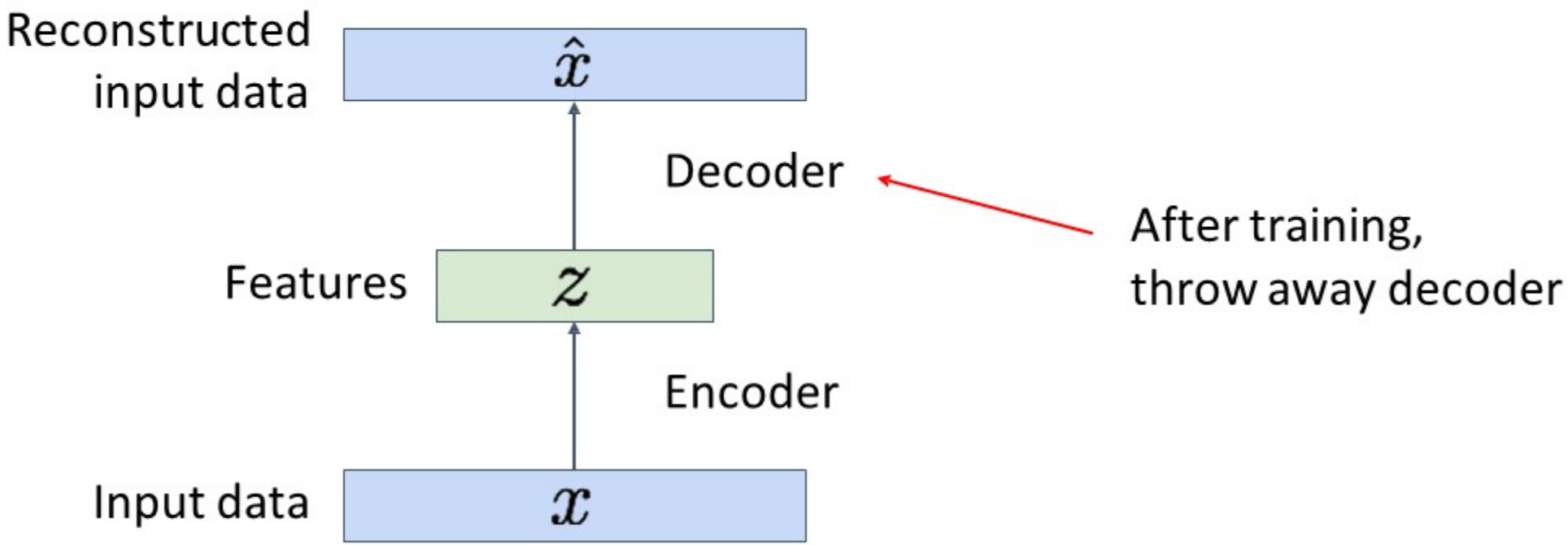
Autoencoders for representational learning

- **Bottleneck hidden layer** forces network to learn compressed latent representation of data x.

- **Reconstruction loss** the latent representation to capture (encode) as much information about the data x as possible.

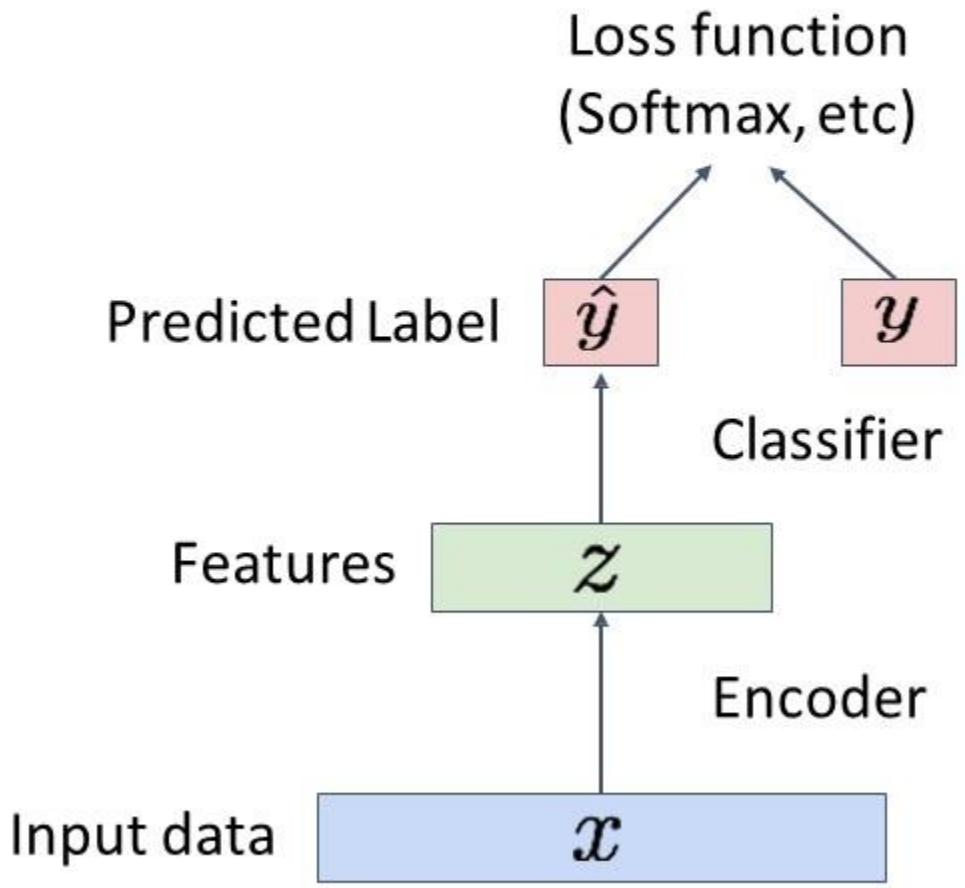
(Regular, non-variational) Autoencoders

After training, **throw away decoder** and use encoder for a downstream task



(Regular, non-variational) Autoencoders

After training, **throw away decoder** and use encoder for a downstream task



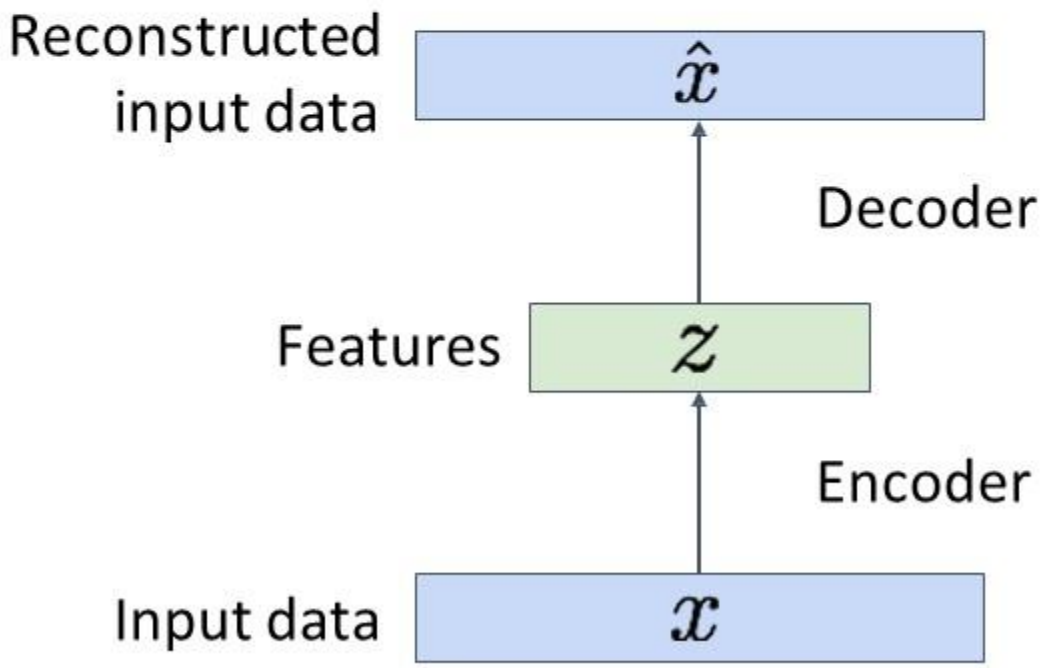
Fine-tune encoder jointly with classifier

Encoder can be used to initialize a **supervised** model

Train for final task (sometimes with small data)

(Regular, non-variational) Autoencoders

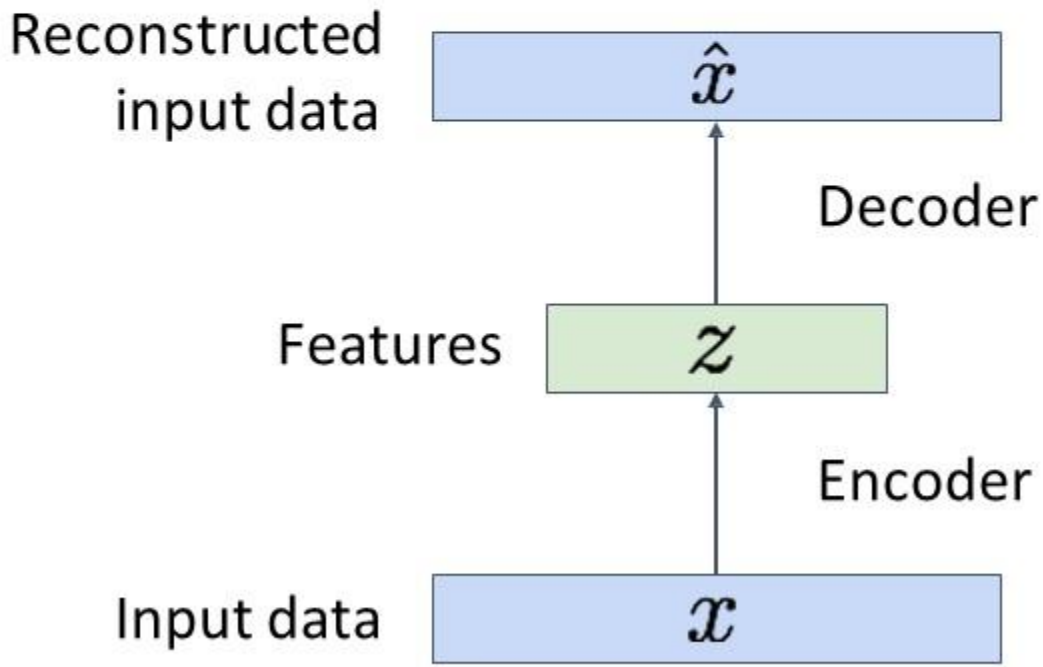
Autoencoders learn **latent features** for data without any labels!
Can use features to initialize a **supervised** model



(Regular, non-variational) Autoencoders

Note **that Autoencoder are deterministic** in nature
i.e. for a given x, a trained autoencoder will always (any time)
generate same output (reconstructed data)

That is , there is **NO way** to sample (generate) new data from a learned model



Variational Autoencoders

Generative Modeling: Variational Autoencoders

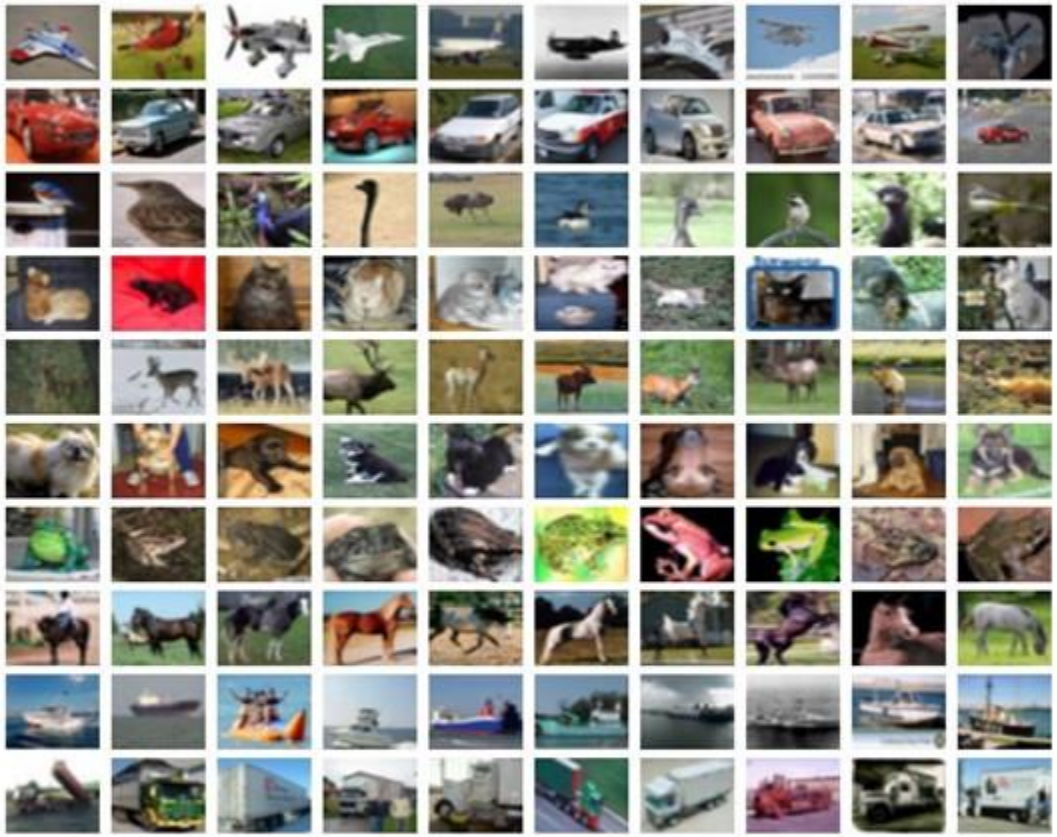
- Generative models aim to learn the probability distribution (density) from which the data samples are drawn.
- In the context of Variational Autoencoders (VAEs), the goal is

  to model this density using neural networks.

- However, the true underlying density of complex data (like images or text) is often very complicated and unknown.

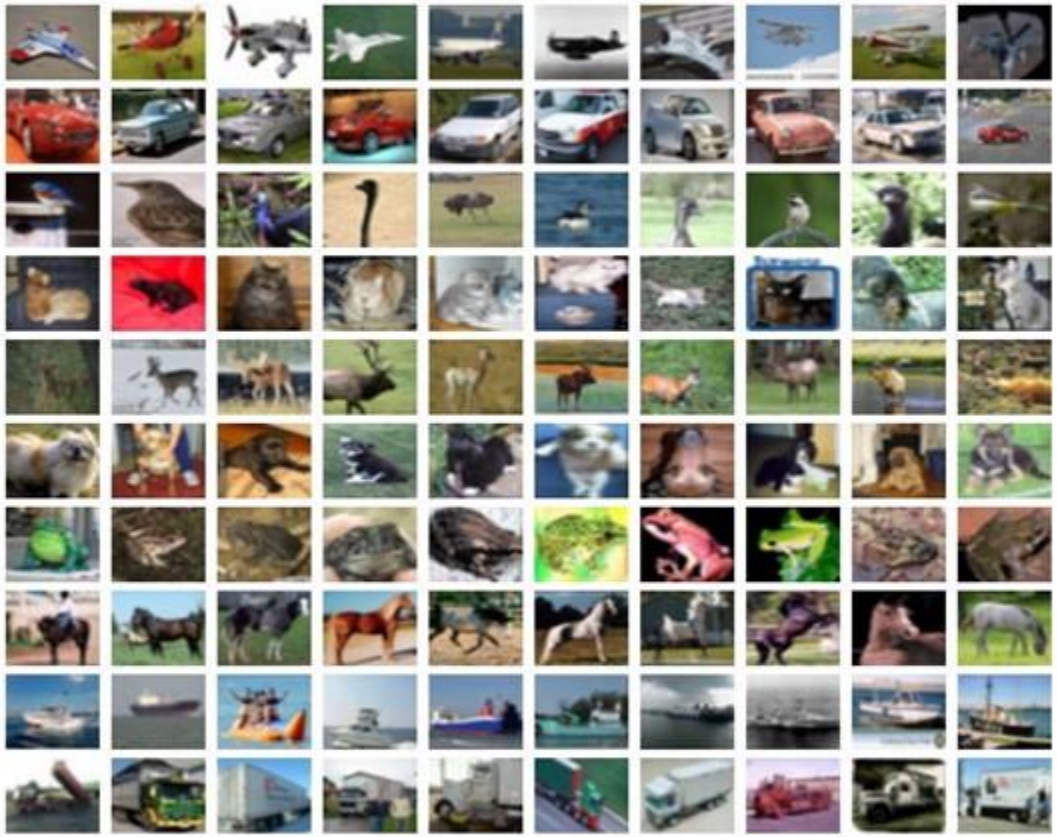
Modeling Image Data

- Imagine we have a dataset of images, for example CIFAR-10 dataset
- Each image can be considered as a high-dimensional data point,
- For example in CIFAR-10 dataset each image is a 3x32×32 pixel image, thus has **3072 dimensions**).



Modeling Image Data

- Our **goal** with a generative model (Variational Autoencoder here) is:
- To model **the distribution** of these images,
- Means, we want to learn how these data points (images) are distributed in this high-dimensional space (in this example 3072 dimensional space)
- be able **to generate new images** that look similar to the observed data.



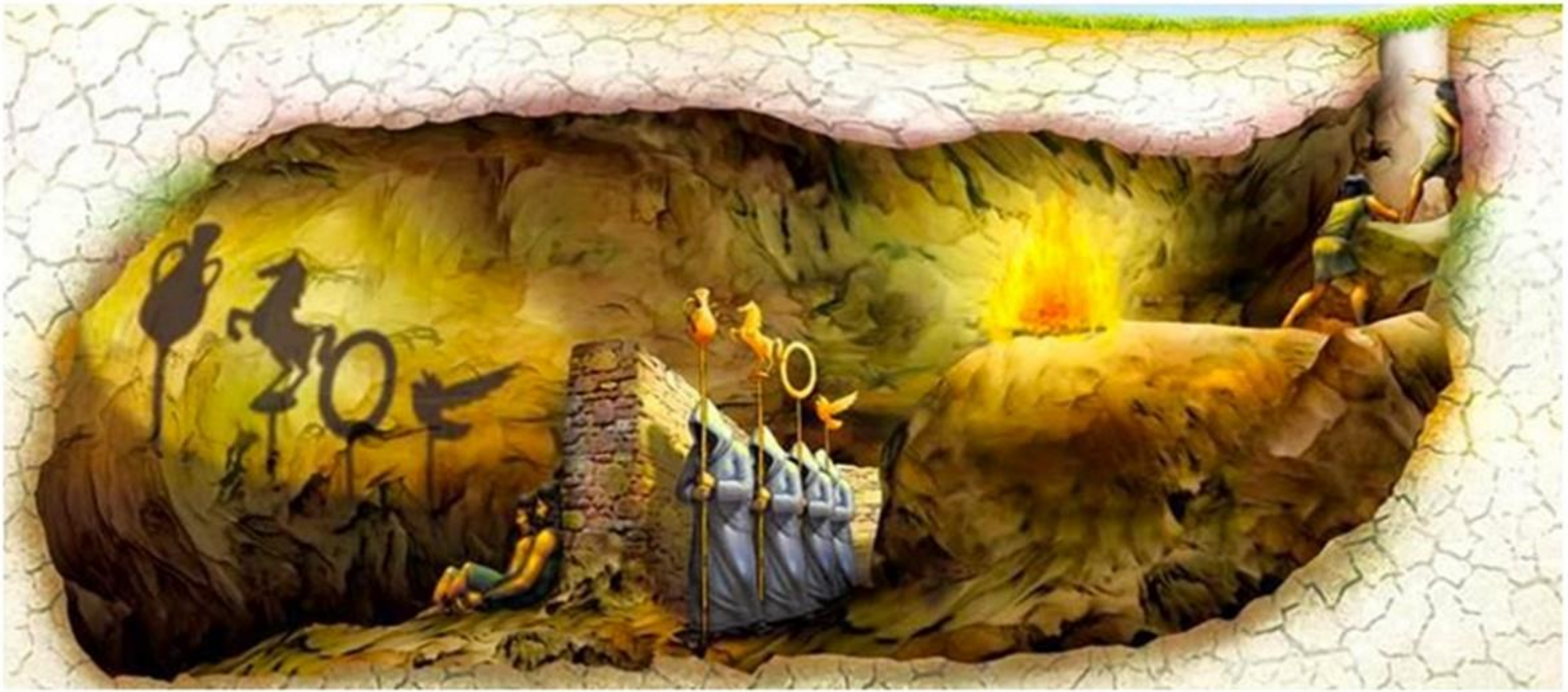
Latent Variable and Latent Space

- Latent variables represent underlying, hidden factors that are not directly observable but are inferred from the observable data.

For example consider a dataset of face images

- **Observable Data**: Images of faces.
- **Latent Variables**: Features that describe underlying attributes of the faces, such as age, gender, expression, etc.
- Latent variables aren't explicitly labeled in the data (hence, latent), but the model learns to infer them to generate or reconstruct faces.

Latent Variable and Latent Space



Myth of the Cave

Latent Variable and Latent Space

- **Latent Space:** The vector space of latent variables where each dimension captures specific, meaningful variations in the data (latent variable).
- For instance, one dimension might control the age appearance in the face, another might adjust the hair color, and so on.
- **Manipulating the Latent Space:** By moving along different axes in the latent space or changing the values of these latent variables, the decoder can alter specific features in the output image.
- For example, adjusting the 'age' variable can make the face look older or younger.

Latent Variable and Latent Space

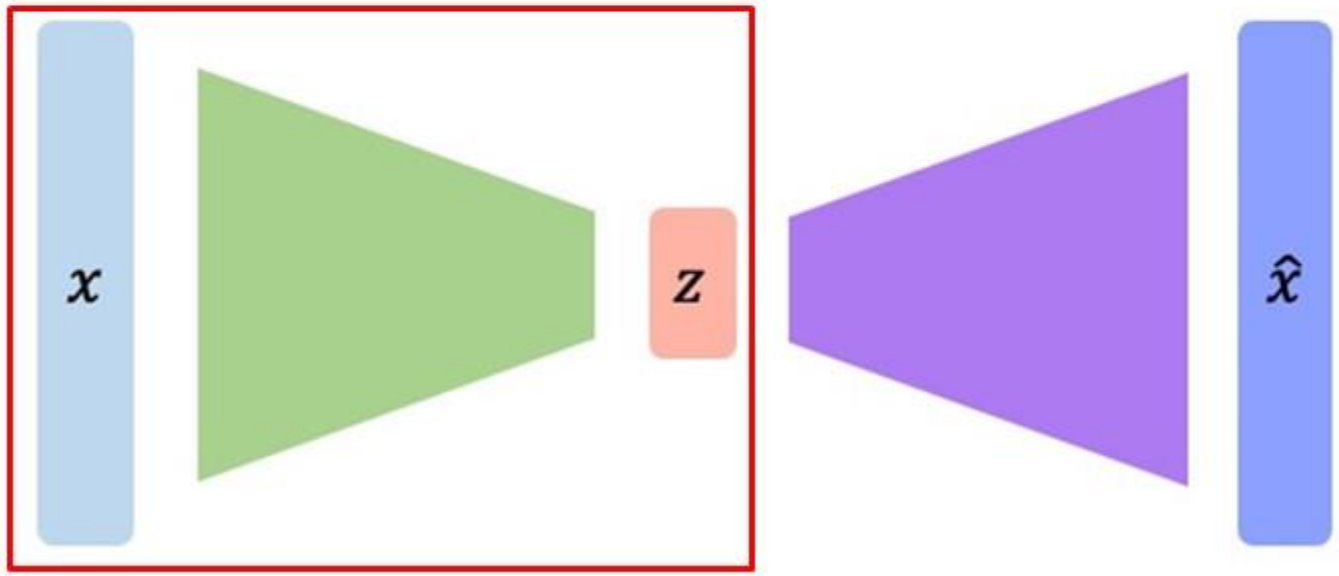
- The VAE learns to map the high-dimensional image data (observable) into a lower-dimensional space represented by latent variables.
- By manipulating these latent variables, the VAE can alter specific features of the generated faces.
- Even though these features were never explicitly provided to the model during training.
- This manipulation demonstrates how the latent space encodes meaningful and interpretable features despite being a compact representation.

Limitation of Autoencoders as a Generative Model

Encoding Process

- In a normal autoencoder, the encoder compresses the input data x into a deterministic, fixed latent representation z.

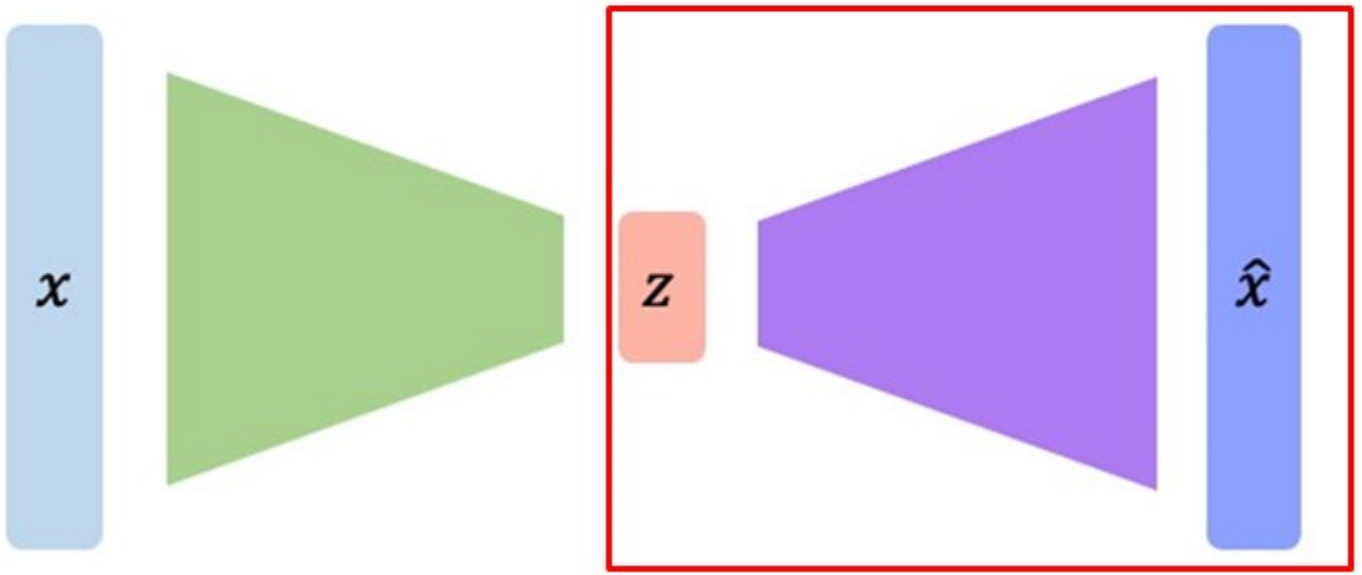
- There's no randomness involved in the encoding process; the same input will always produce the same latent representation.



Limitation of Autoencoders as a Generative Model

Decoding Process

- Decoding in an AE is straightforward; the decoder network maps the **deterministic latent representation** back to the input space, aiming to reconstruct the original input as closely as possible..

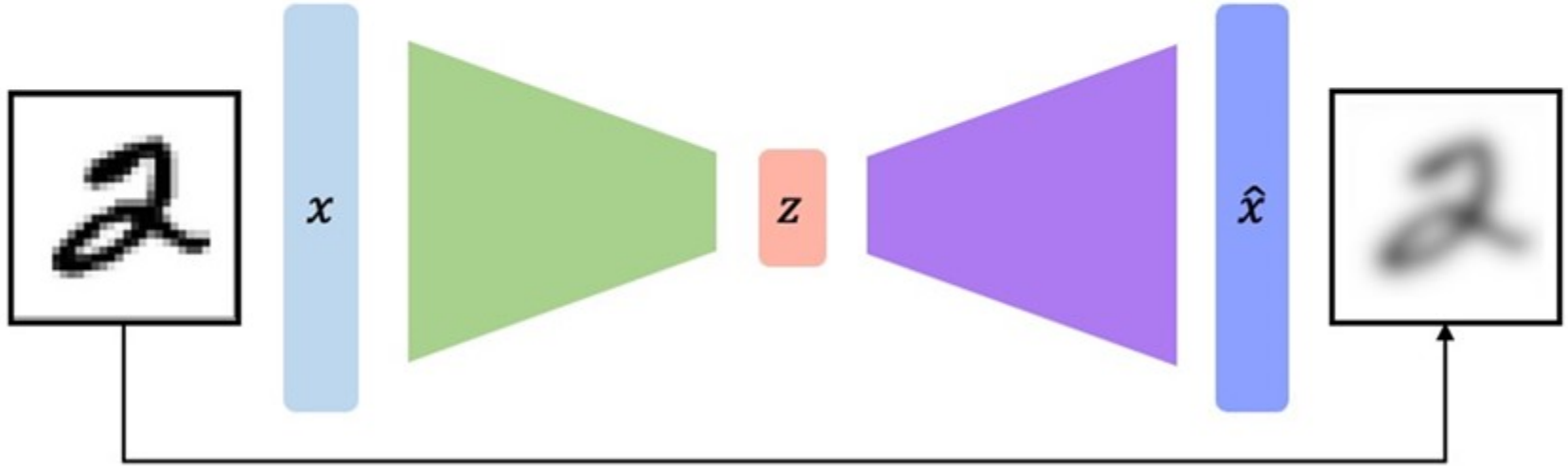


Limitation of Autoencoders as a Generative Model

Objective Function

- The goal is generally to minimize the reconstruction loss, which measures the difference between **the original input** and its **reconstruction from the latent space**.

- Commonly used metrics are mean squared error (MSE) or binary cross-entropy, depending on the type of data.



Limitation of Autoencoders as a Generative Model

Generative Capability

- Generative models are typically defined by their ability to generate new data points from the learned distribution.
- In AE, the encoding process is deterministic, meaning that for a given input x, the encoder produces a fixed latent representation z.
- This latent code is a specific point in the latent space, uniquely determined by the input and the encoder's parameters.

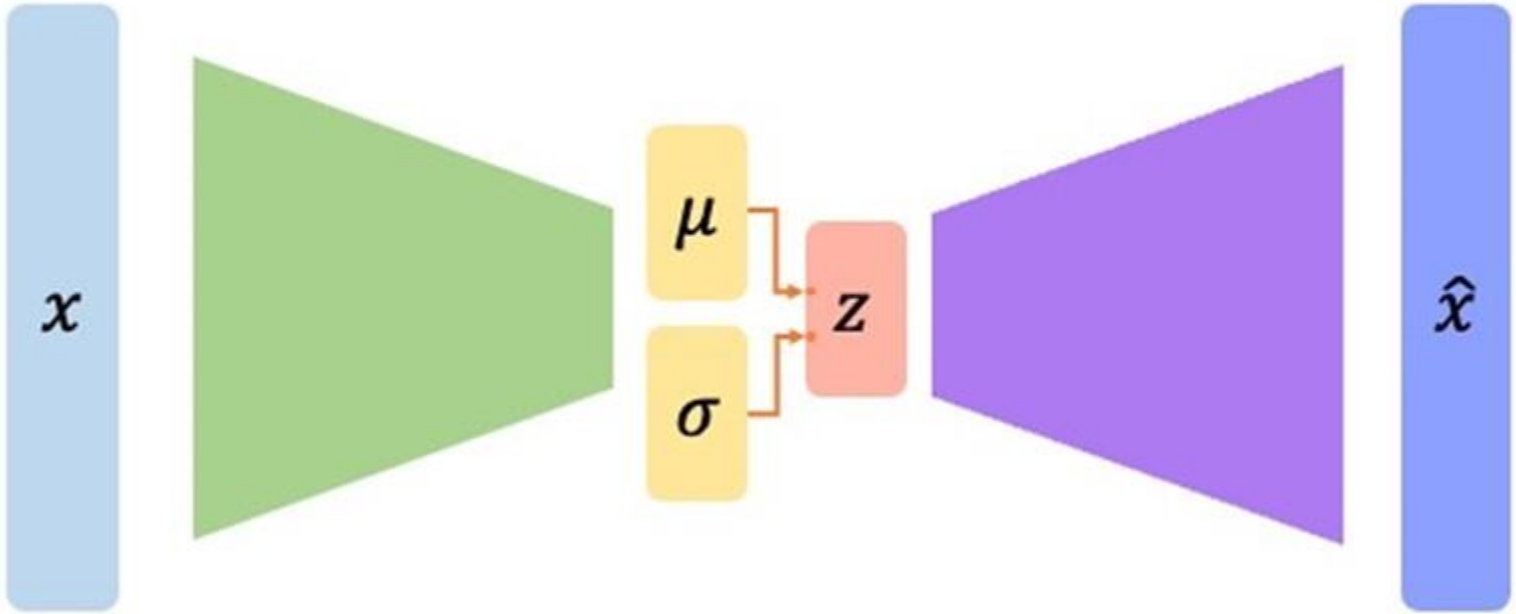
Limitation of Autoencoders as a Generative Model

Generative Capability

- There's no element of randomness in this encoding process, so each time you input x, you will get the same z.
- They can struggle with generating new instances of data that are not close to the training samples because their latent spaces might not interpolate smoothly..

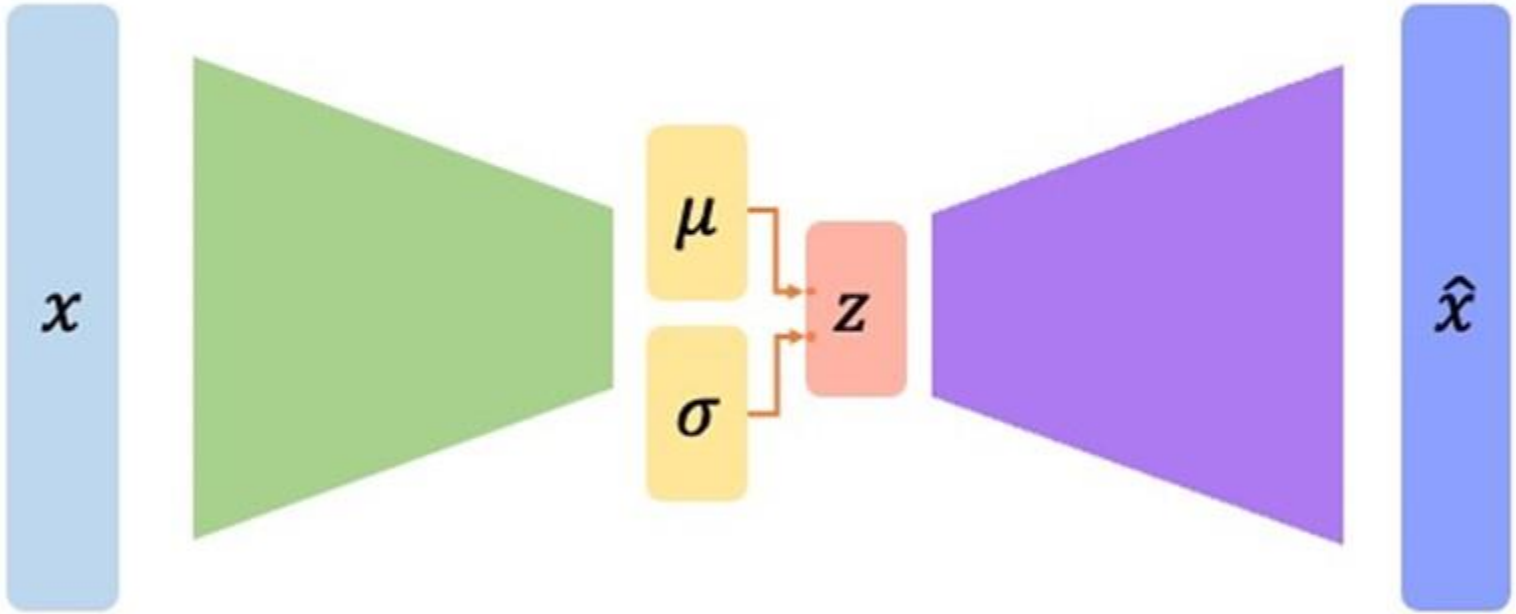
Generative Modeling: Variational Autoencoders

- In contrast to AE, a VAE encodes the input data x into a distribution over the latent space, typically defined by mean (μ) and variance (σ^2) parameters.



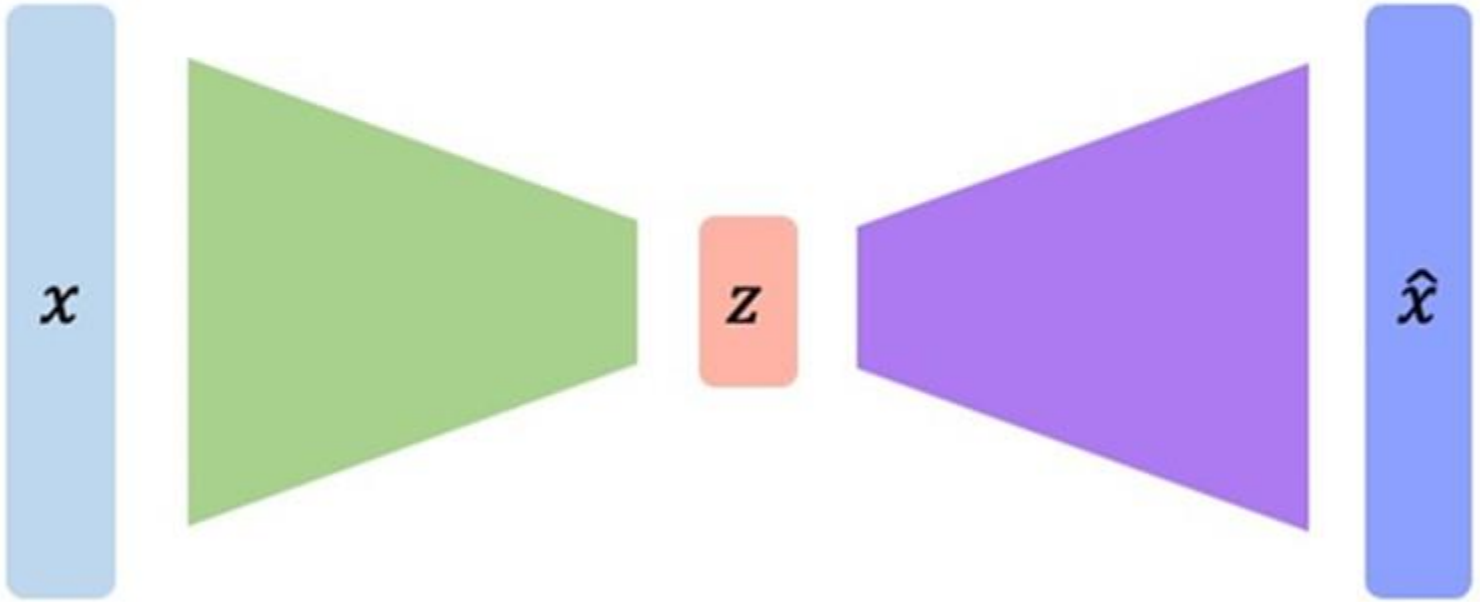
Generative Modeling: Variational Autoencoders

- VAEs introduce an element of randomness, as the latent representation z for a given input is sampled from this distribution, making it stochastic.



Generative Modeling: Variational Autoencoders

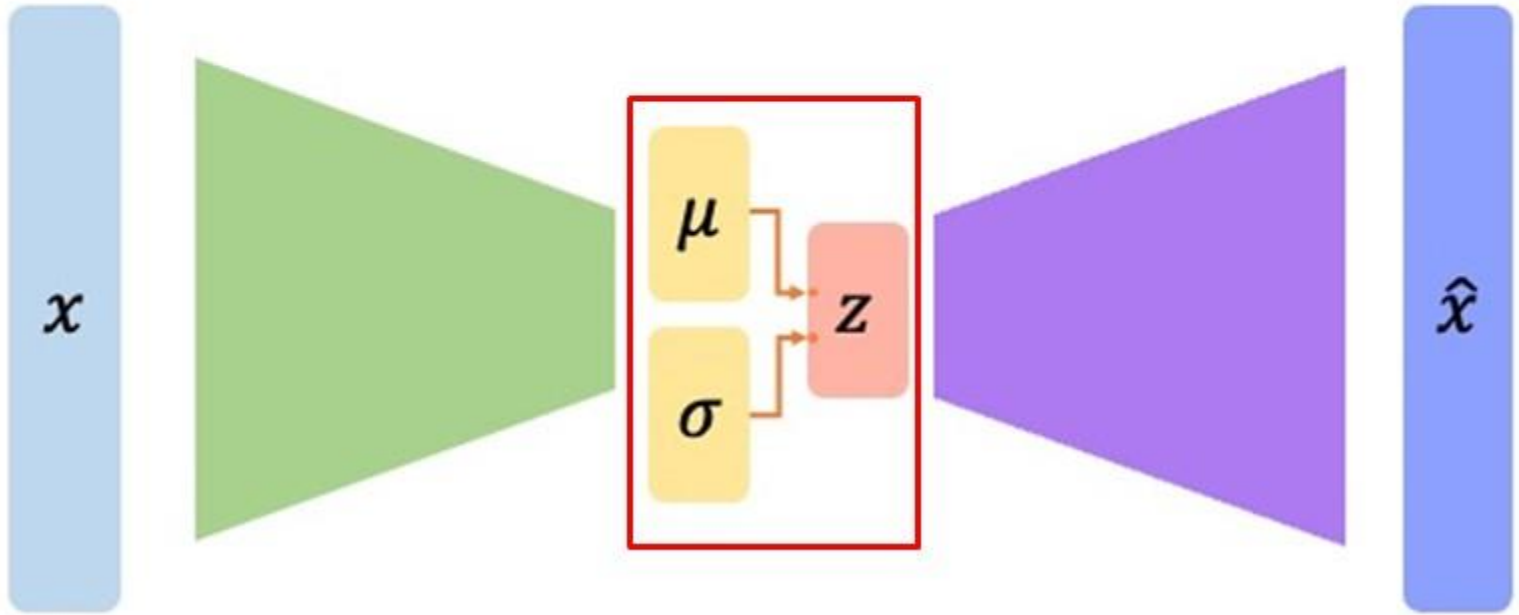
- This is in contrast to AE, the encoder compresses the input data x into a deterministic, fixed latent representation z.



Generative Modeling: Variational Autoencoders

- In a VAE, the decoder takes a **sampled latent variable** and reconstructs the input $\hat{x}$ from this sample.

- This **stochastic nature allows VAEs** not only to reconstruct the input but also to generate new, novel inputs that resemble the training data by sampling different points in the latent space.



Variational Autoencoders: Latent Space

- In VAEs and similar models, the latent space is a conceptual representation where **each point** corresponds to **a potential state** or configuration of the input data.

- Instead of encoding an input x directly to a single point in latent space (as traditional autoencoders do), VAEs encode inputs to a probability distribution $q_\phi(z|x)$ over possible points (z) in latent space.

- i.e. $q_\phi(z|x)$ describes the distribution of the latent variables z given image x. Here ϕ are parameters of Encoder network.

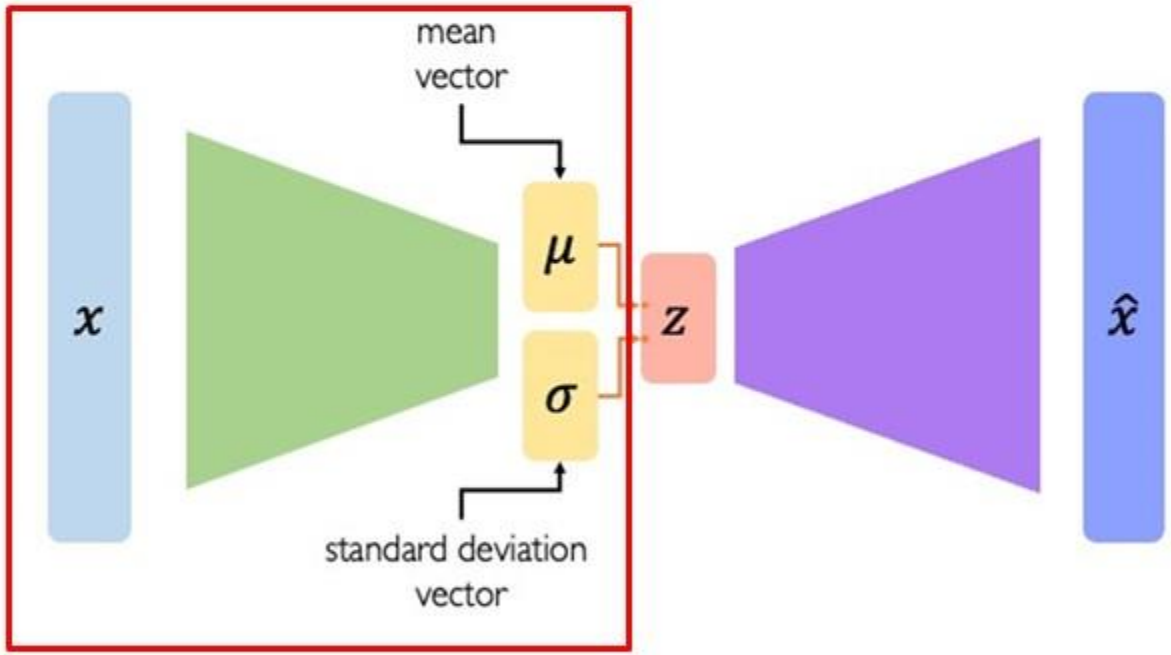
Variational Autoencoders: Latent Space

- Using a distribution rather than a fixed point allows the model to capture the uncertainty and variability inherent in the data.
- It acknowledges that there could be multiple plausible latent representations for a given input, reflecting different aspects or interpretations of the data.

Variational Autoencoders

Encoding Process

- VAE encodes the input x into a distribution $q_\phi(z|x)$ over the latent space, typically defined by mean (μ) and variance (σ^2) parameters. .

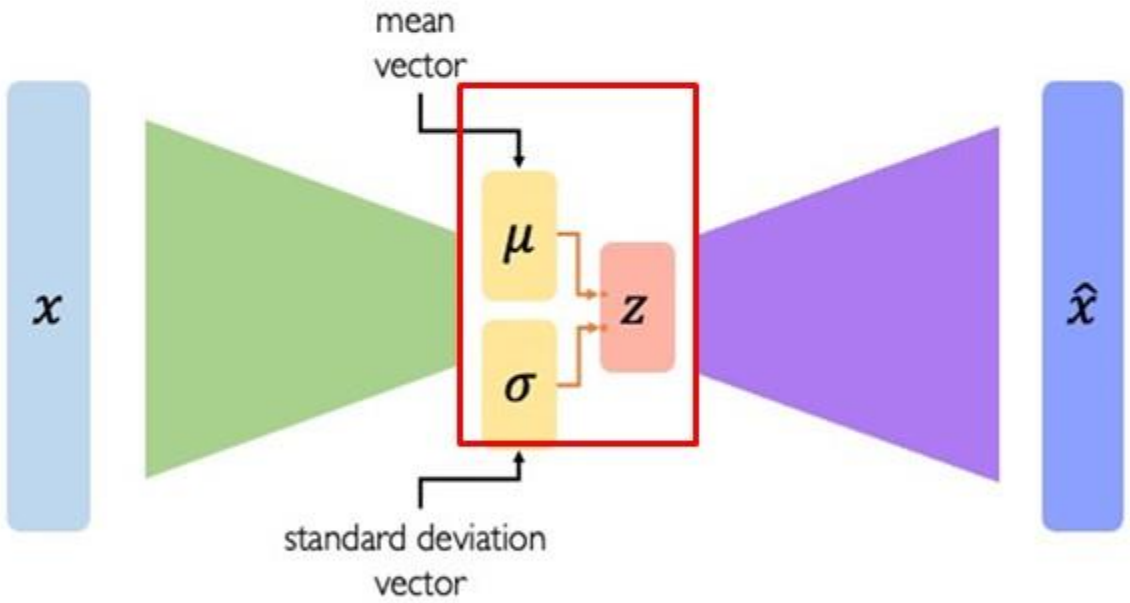


- These parameters (μ and σ) are outputs of the encoder network when given an input x.

Variational Autoencoders

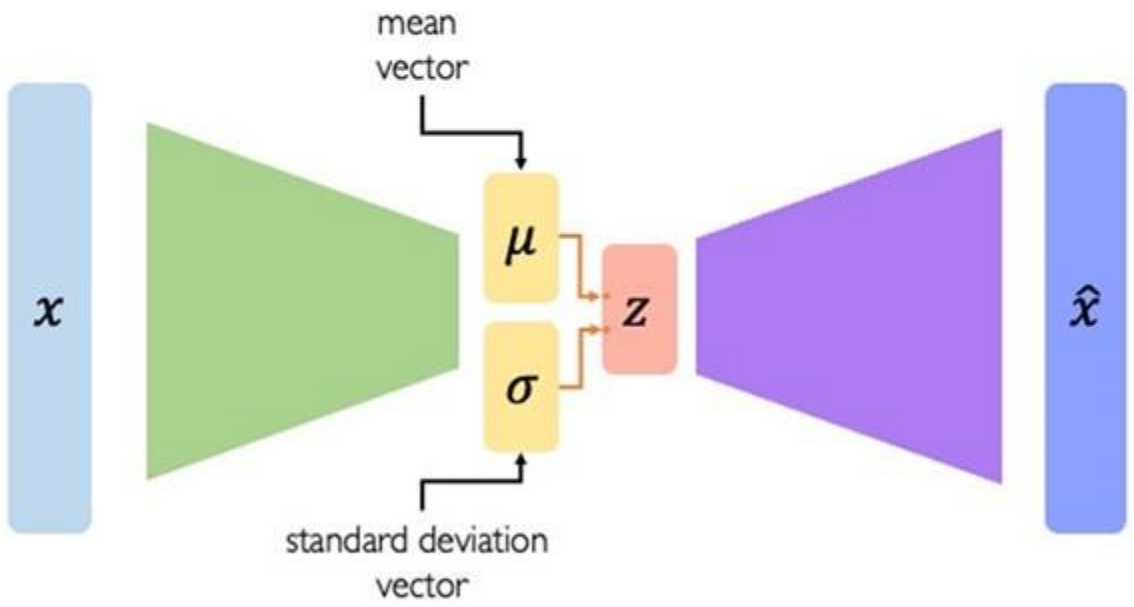
Sampling Process

- Latent variables z are then sampled from the distribution $q_\phi(z|x)$, which is typically a Gaussian distribution with parameters (μ and σ) .



Variational Autoencoders

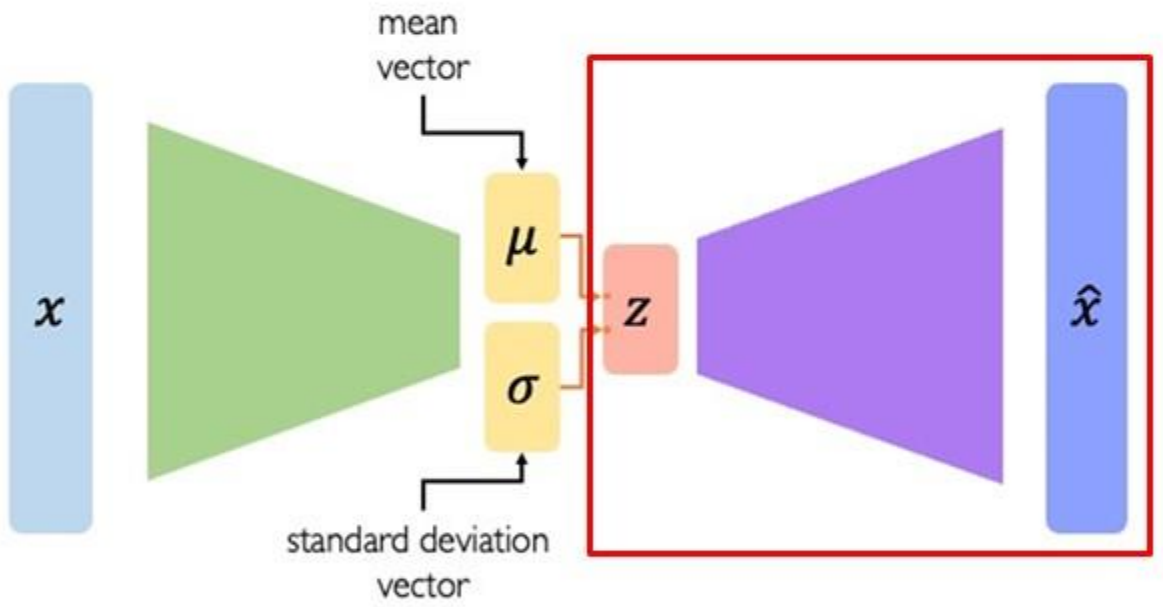
- This sampling introduces randomness, which means that
- Even for the **same input** x, different instances of z can be generated on different occasions of the encoding process.



Variational Autoencoders

Decoding Process

- The decoder network, parameterized by θ uses z (sampled form$q_\phi(z|x)$) to generate a reconstruction of the original image or variations thereof.



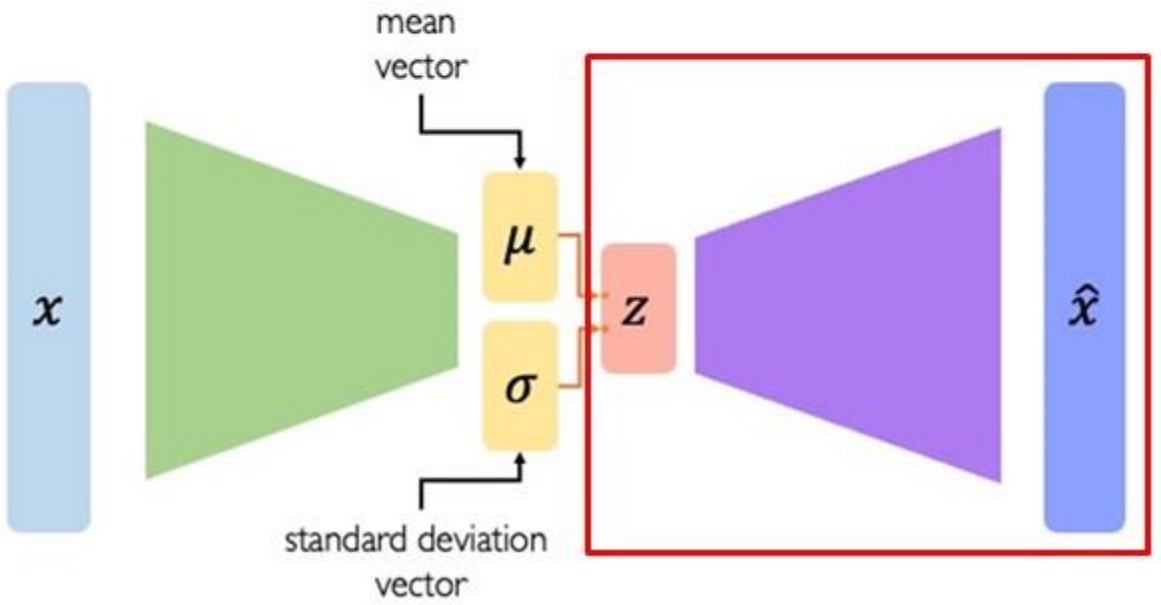
- The decoder's output is represented by the distribution $p_\theta(z|x)$

Variational Autoencoders

Decoding Process

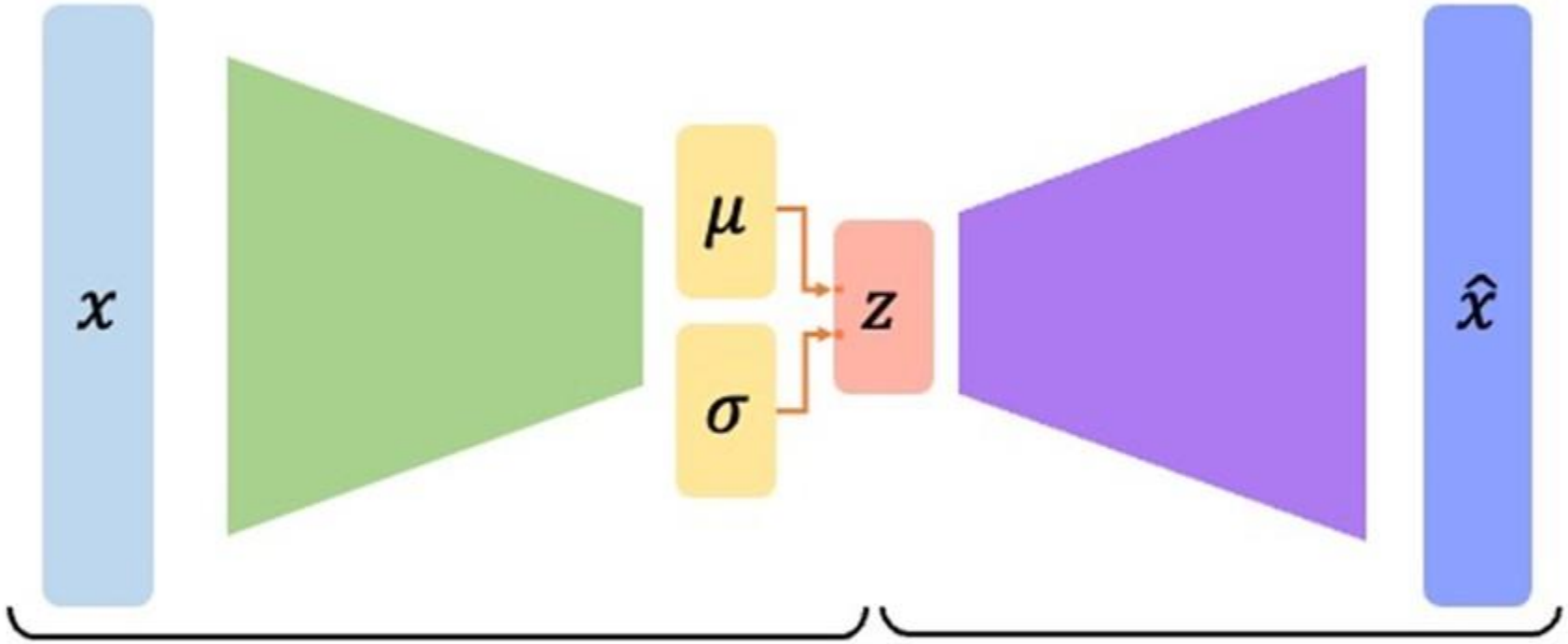
- This stochastic nature of sampled latent variable allows VAEs not only to reconstruct the input but

- Also to generate new, novel inputs that resemble the training data by sampling different points in the latent space.



Variational Autoencoders

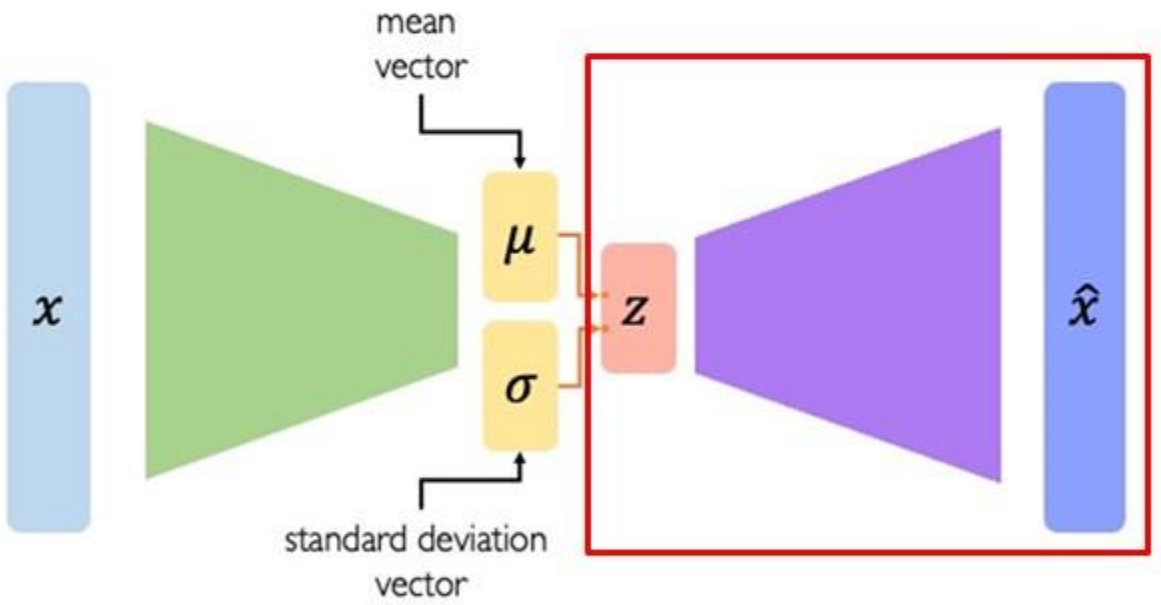
Encoder-Decoder



Variational Autoencoders

Prior Distribution

- Prior distribution $p(z)$ is the assumed distribution of the latent variables z before observing any data x.

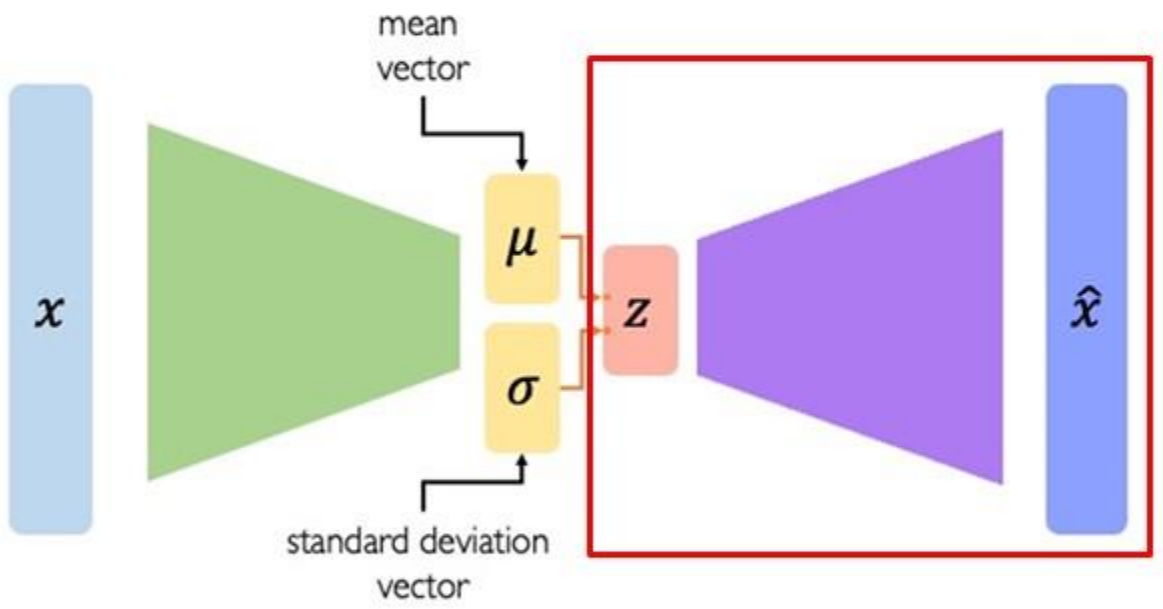


- In most VAEs, this prior is set to be a standard Gaussian distribution ($\mathcal{N}(0, I)$) i.e. assumes z are normally distributed with a mean of zero and a unit variance.

Variational Autoencoders

Prior Distribution

- Assuming standard Gaussian distribution ($\mathcal{N}(0, I)$) as a prior, forces statically indecency of different dimensions (latent variables) in the latent space.



- It helps in independently control different variations in the data, making the model easier to interpret and manipulate.

Variational Autoencoders

Objective Function

- VAEs try to achieve two goals:

1. To ensure that the latent variables z contain sufficient and relevant information about the input data x (encoding) to allow for accurate reconstruction from z. (Reconstruction loss)

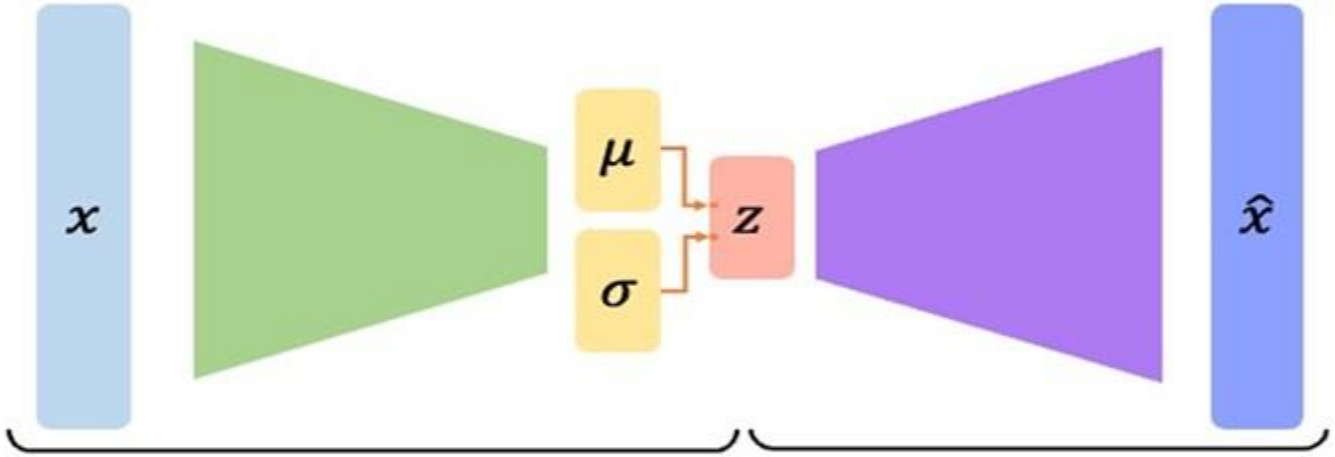
2. To structure the **learned latent distribution** $q_\phi(z|x)$ ensuring that it adheres to a predefined probabilistic model $p(z)$(usually a standard Gaussian distribution). (Regularization term)

$$\mathcal{L}(\emptyset, \theta, x) = (Reconstruction\ loss\ + Regularization\ term)$$

Variational Autoencoders

Objective Function

- The first term encourages the latent variables to be well-formed and disentangled, and prevents overfitting by ensuring the distribution of latent variables doesn't stray too far from the prior.

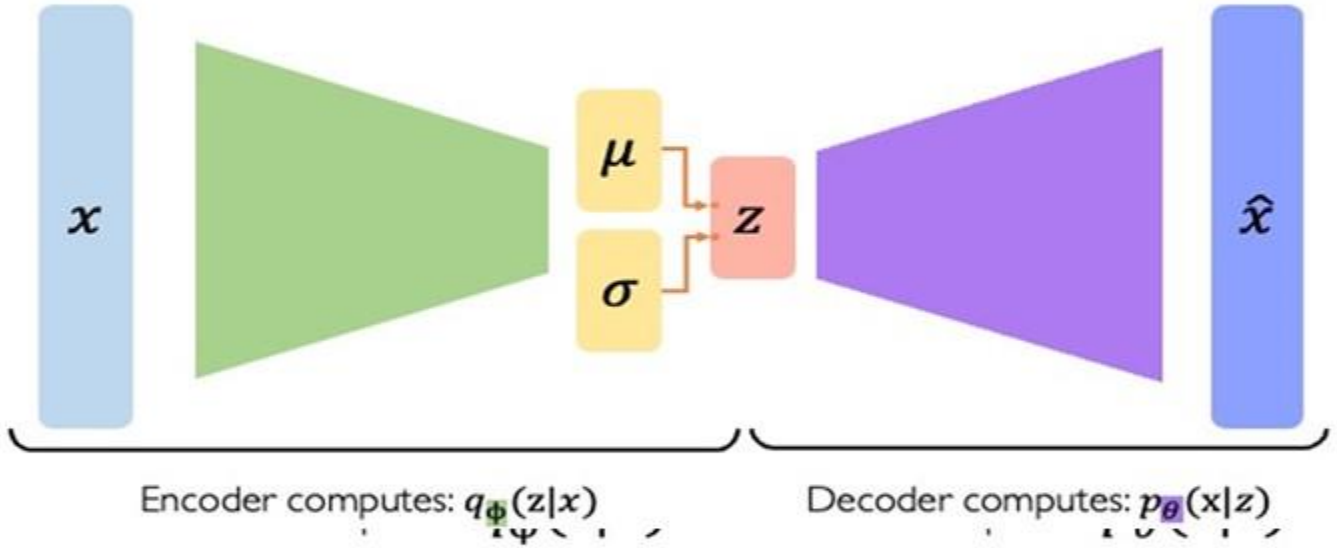


$$\mathcal{L}(\emptyset, \theta, x) = (Reconstruction\ loss + Regularization\ term)$$

Variational Autoencoders

Objective Function

- The regularization helps prevent overfitting to the training data by discouraging the encoder from mapping inputs to arbitrary points in the latent space.



$$\mathcal{L}(\emptyset, \theta, x) = (Reconstruction\ loss\ + Regularization\ term)$$

- Instead, it encourages a smooth and continuous latent space where similar points result in similar outputs when decoded.

Variational Autoencoders

Objective Function

$$\mathcal{L}(\emptyset, \theta, x) = (Reconstruction\ loss\ + Regularization\ term)$$

- The reconstruction objective aims to ensure that the encoder must capture as much detail about the input data as possible, compressing it into z without losing significant information.

- On the other hand, the regularization objective seeks to ensure that the distribution of the latent variables z, as derived from the input data x, closely follows a predetermined distribution (typically a standard Gaussian).

Variational Autoencoders

Objective Function

$$\mathcal{L}(\emptyset, \theta, x) = (Reconstruction\ loss\ + Regularization\ term)$$

- If the **regularization is too strong**, the model might not capture enough details of the data, leading to poor reconstructions (underfitting).
- If the **reconstruction objective is overly emphasized**, the model might memorize the training data too specifically, resulting in a latent space that does not generalize well (overfitting).
- Together, these two objectives enable VAEs to not only learn compact and efficient representations of data but also to generalize beyond the specific instances seen during training.

Variational Autoencoders

Objective Function

- The total loss function of a VAE is typically formulated as:

$$Loss = Reconstruction\ Loss + \beta\ KL\ divergenec$$

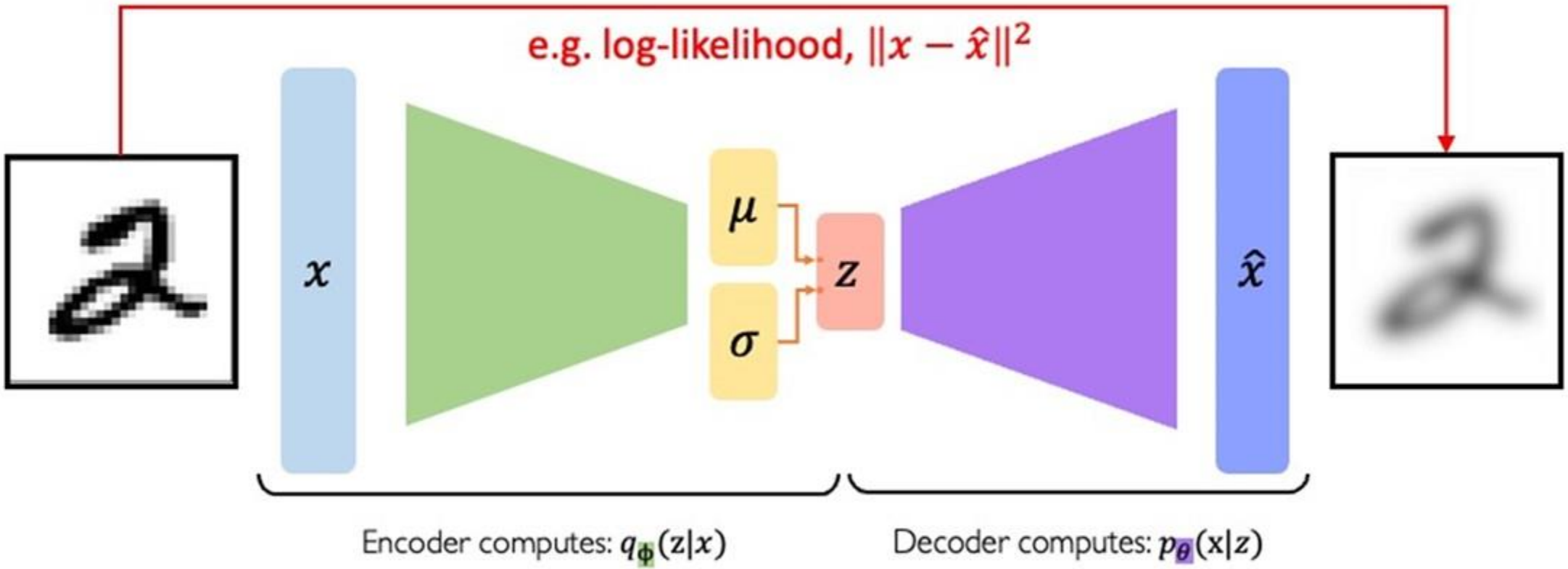
- Where β is a hyperparameter that controls the balance between the reconstruction loss and the KL divergence.

- A higher weight on the reconstruction loss might lead to better input reconstruction at the cost of less meaningful latent representations,

- While a higher weight on the KL divergence encourages more "normal" latent distributions, potentially at the expense of reconstructing details.

Variational Autoencoders

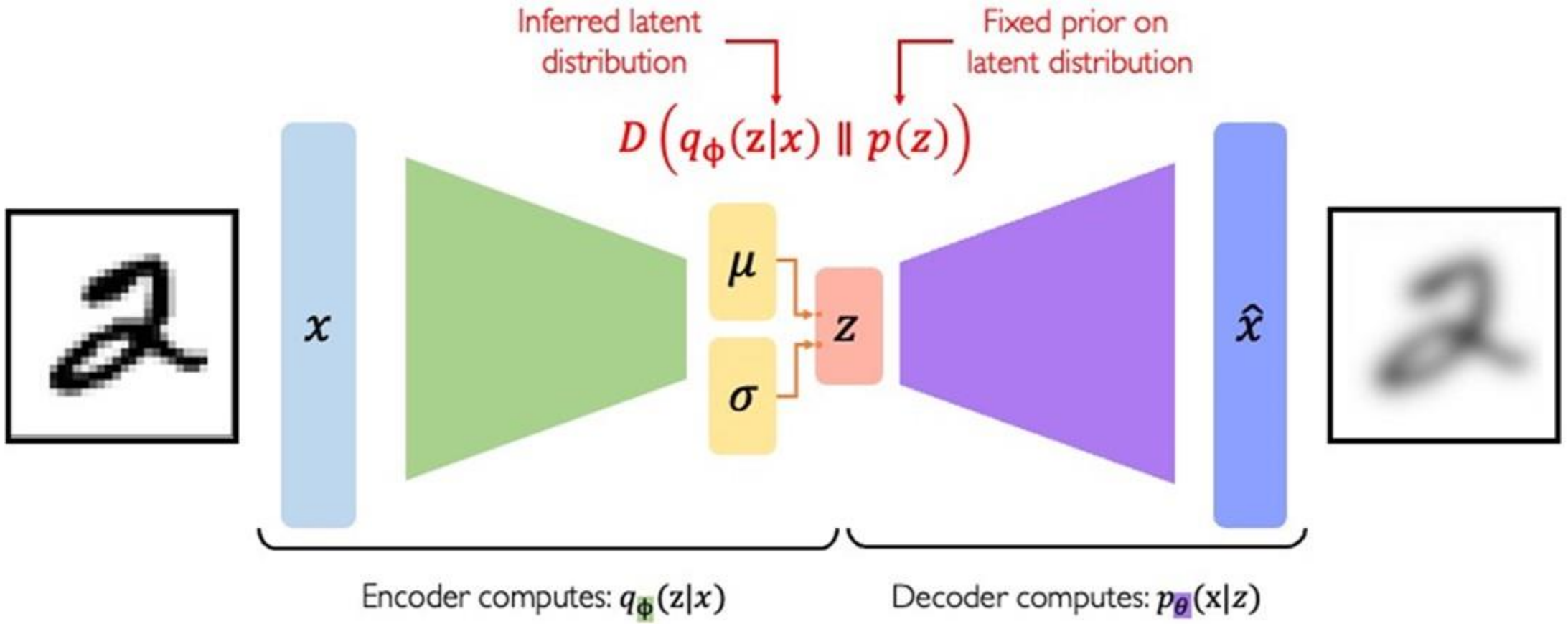
Objective Function



$$\mathcal{L}(\phi, \theta, x) = \boxed{\text{(reconstruction loss)}} + \text{(regularization term)}$$

Variational Autoencoders

Objective Function



$$\mathcal{L}(\phi, \theta, x) = \text{(reconstruction loss)} + \boxed{\text{(regularization term)}}$$

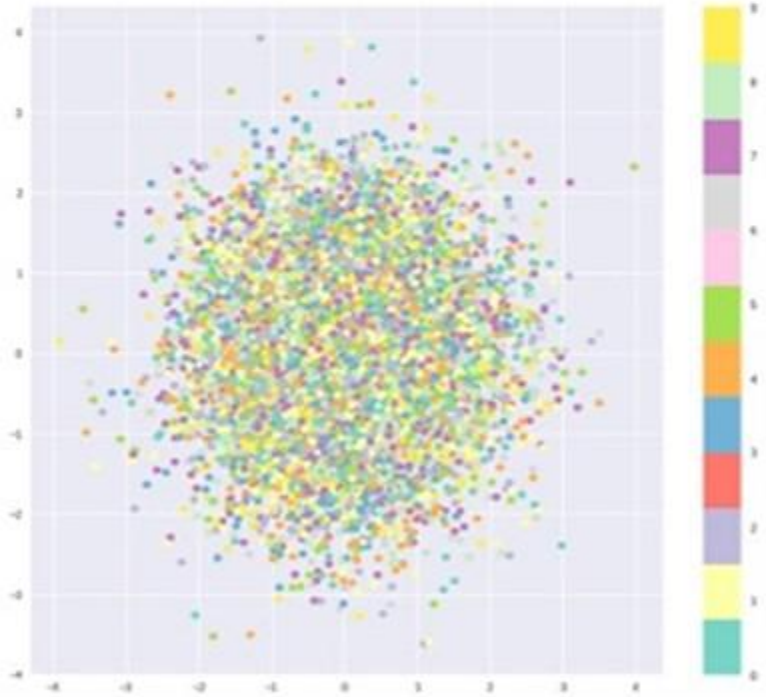
Variational Autoencoders

Prior on Latent distribution

$$D\left(q_\phi(\mathrm{z}|x) \parallel p(z)\right)$$

Inferred latent distribution

Fixed prior on latent distribution



Common choice of prior – Normal Gaussian:

$$p(z) = \mathcal{N}(\mu = 0, \sigma^2 = 1)$$

- Encourages encodings to distribute encodings evenly around the center of the latent space
- Penalize the network when it tries to "cheat" by clustering points in specific regions (i.e., by memorizing the data)

Variational Autoencoders

Intuition behind Regularization and Normal Prior

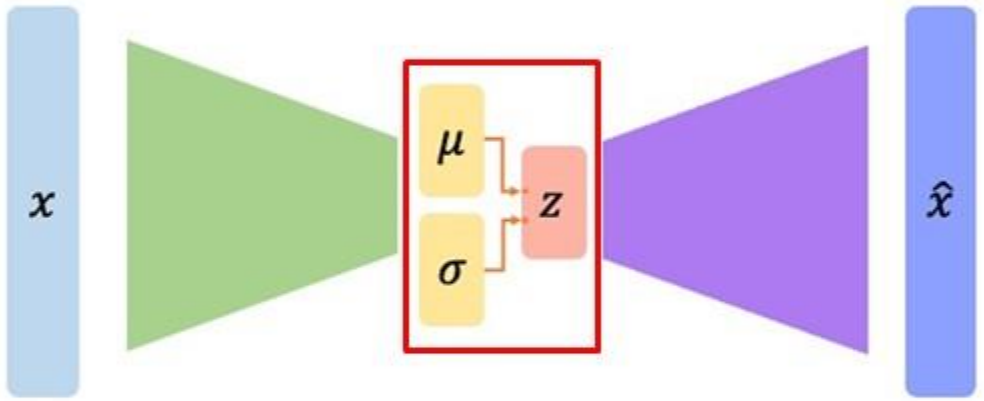
- Desired Properties:

- **Continuity:** Points that are close in latent space should result in similar content after decoding

- **Completeness:** If we sample any *z* form latent space, it should result in meaningful content (similar to training data) after decoding

Variational Autoencoders

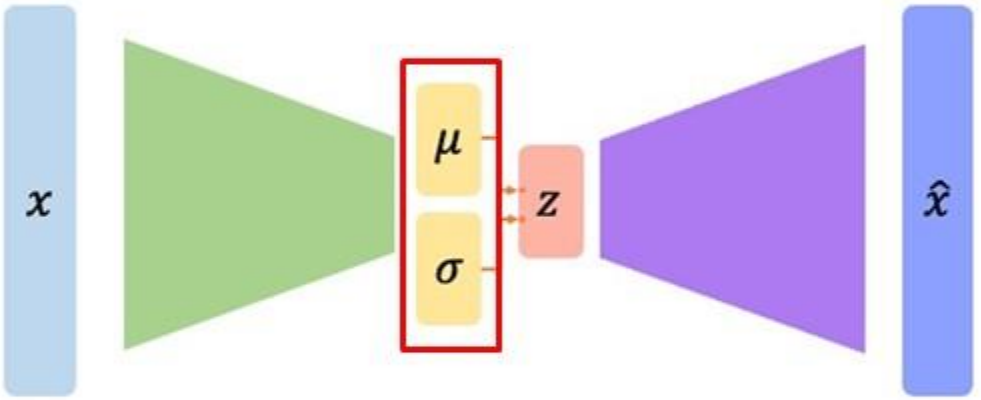
Reparametrization Trick



- **Problem:** Directly sampling from this distribution to generate latent variables introduces randomness that prevents straightforward gradient backpropagation, because gradients cannot flow through a random node.

Variational Autoencoders

Reparametrization Trick



This noise ϵ provides the **stochastic element** needed for the model,
but it is **independent of the parameters** of the model.

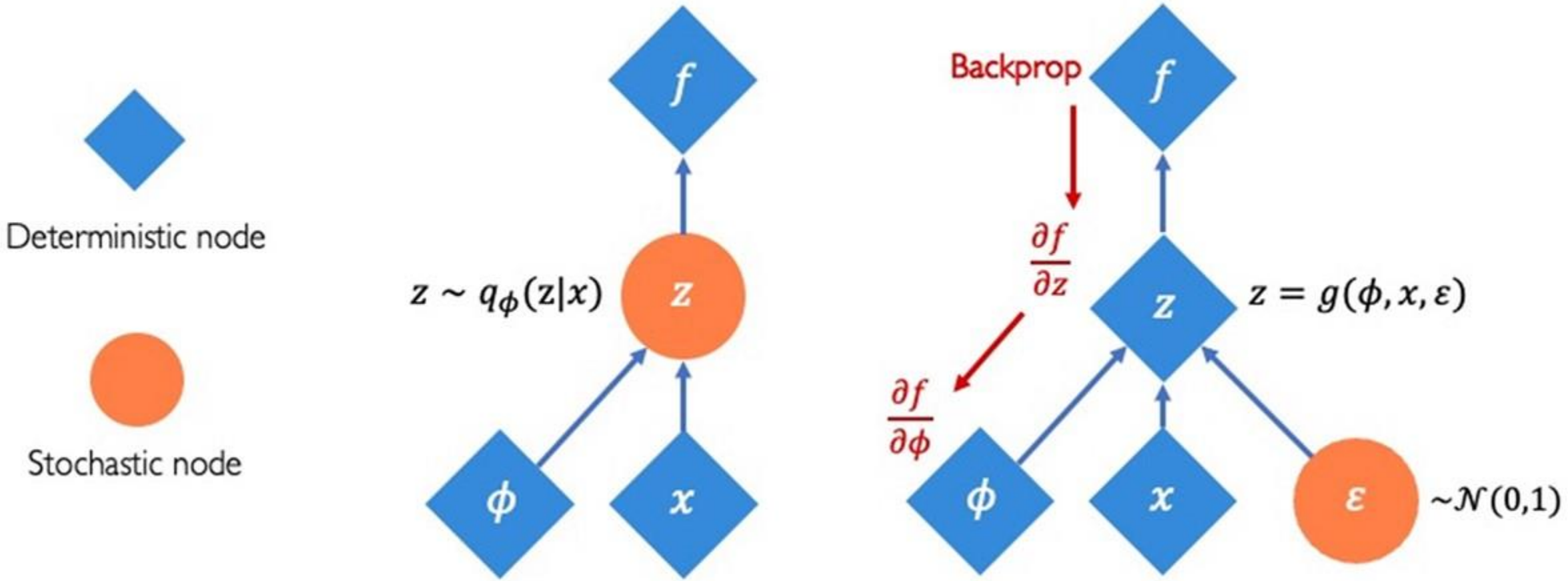
- **Solution:** Consider the sampled latent vector $\boldsymbol{z}$ as sum of

- A deterministic vector $\boldsymbol{\mu}$ and
- A deterministic vector $\boldsymbol{\sigma}$, sacled by a random value ϵ drawn from prior, normal distribution $\mathcal{N}(0, I)$:

- $z = \mu + \sigma \odot \epsilon$

Variational Autoencoders

Reparametrization Trick



Variational Autoencoders

Probabilistic twist on autoencoders:

1. Learn latent features z from raw data

2. Sample from the model to generate new data

Variational Autoencoders

Generative Assumption:

- The observable data x (such as an image, text, or any other type of data) are considered to be generated by some latent (hidden or unobservable) factors or variables z.
- These latent variables z abstracts the underlying factors or characteristics that explain or contribute to the observable properties of the data.
- For instance, in the case of images, z could represent aspects like the object type, position, lighting conditions, and so on.

Variational Autoencoders

Generative Assumption:

- These factors are not typically direct features of the observable data (e.g. images) but rather are higher-level attributes that are inferred by z.
- The encoder learns to map data x to the latent space
- The decoder learns to reconstruct data from this latent space,
- Thereby bridging the gap between the unobservable latent factors and the observable outcomes.

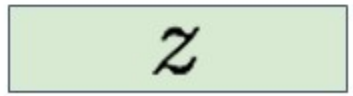
Variational Autoencoders

Assume training data $\{x^{(i)}\}_{i=1}^{N}$ is generated from an unobservable (latent) representation z.

Assume simple prior $p(z)$, e.g. Gaussian

After training, sample new data like this:

Sample z from prior $p_{\theta^*}(z)$



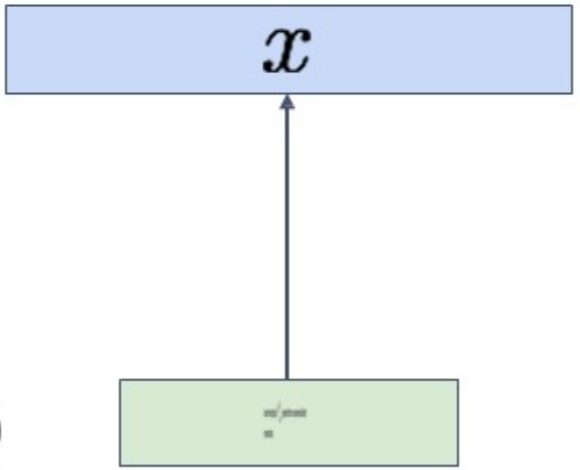
Variational Autoencoders

Assume training data $\{x^{(i)}\}_{i=1}^{N}$ is generated from an unobservable (latent) representation z.

Assume simple prior $p(z)$, e.g. Gaussian

After training, sample new data like this:

Sample from conditional $p_{\theta^*}(x \mid z^{(i)})$



Sample z from prior $p_{\theta^*}(z)$

Variational Autoencoders

Assume training data $\{x^{(i)}\}_{i=1}^{N}$ is generated from an unobservable (latent) representation z.

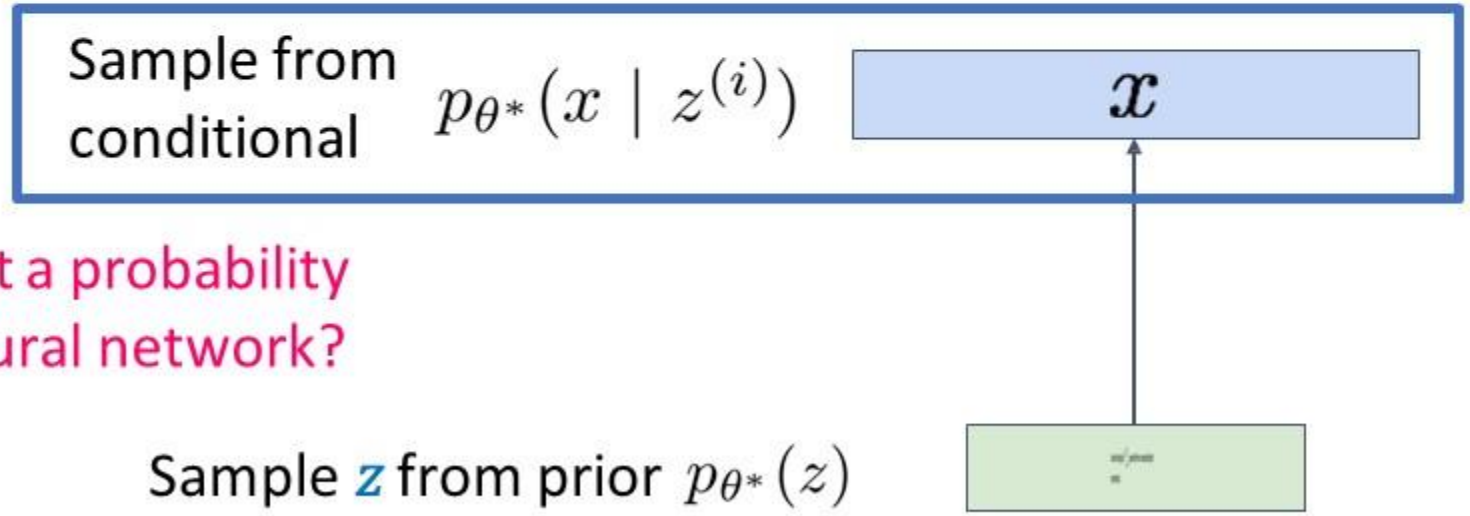
Assume simple prior $p(z)$, e.g. Gaussian

Represent $p(x|z)$ with a neural network
(Similar to **decoder** from autencoder)

After training, sample new data like this:

Sample from conditional $p_{\theta^*}(x \mid z^{(i)})$

How can we represent a probability distribution with a neural network?



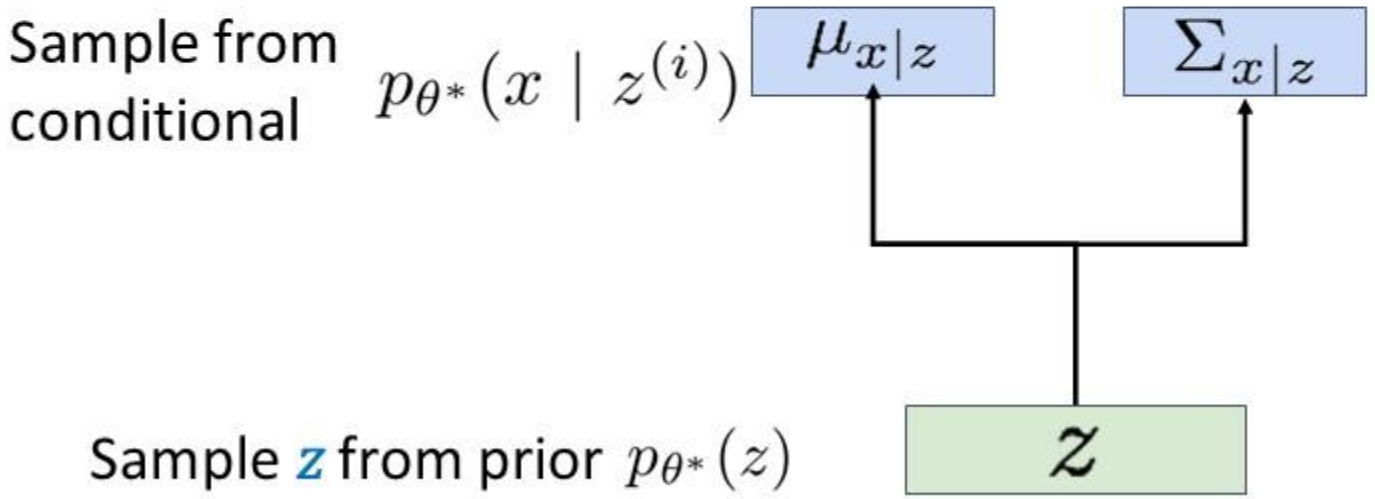
Sample z from prior $p_{\theta^*}(z)$

Variational Autoencoders

- The distribution $p(x|z)$ is assumed to be a Gaussian with parameters mean $\mu_{x|z}$ and variance $\Sigma_{x|z}$.

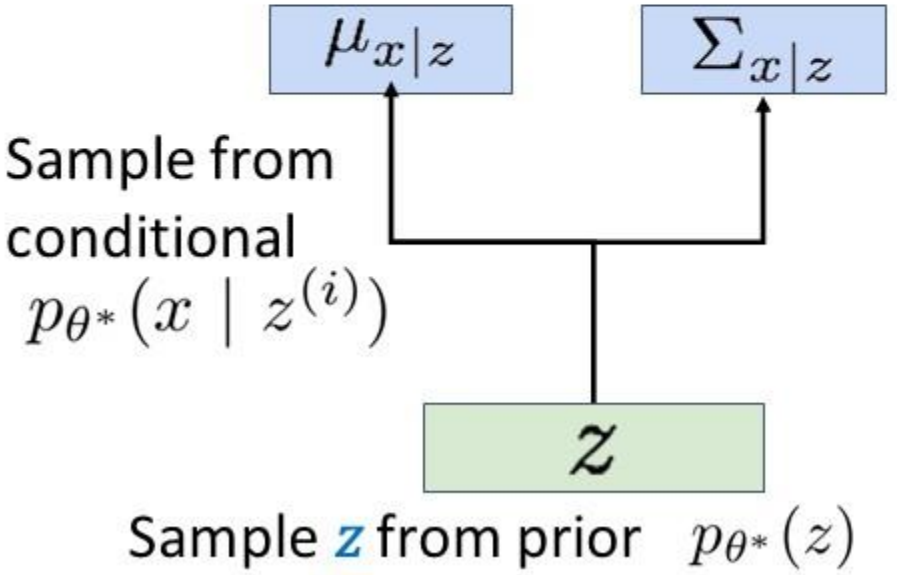
- In case of image generation, dimension of $p(x|z)$ is same as # pixels in the image.

- Dimensions of distribution are assumed to be conditionally independent, $\Sigma_{x|z}$ is diagonal matrix. Thus, the dimension of $\mu_{x|z}$ and $\Sigma_{x|z}$ is equal to # pixels in the image.



Variational Autoencoders

- Decoder must be **probabilistic**: Decoder inputs z, outputs mean $\mu_{x|z}$ and (diagonal) covariance $\sum_{x|z}$
- Sample x from Gaussian with mean $\mu_{x|z}$ and (diagonal) covariance $\sum_{x|z}$

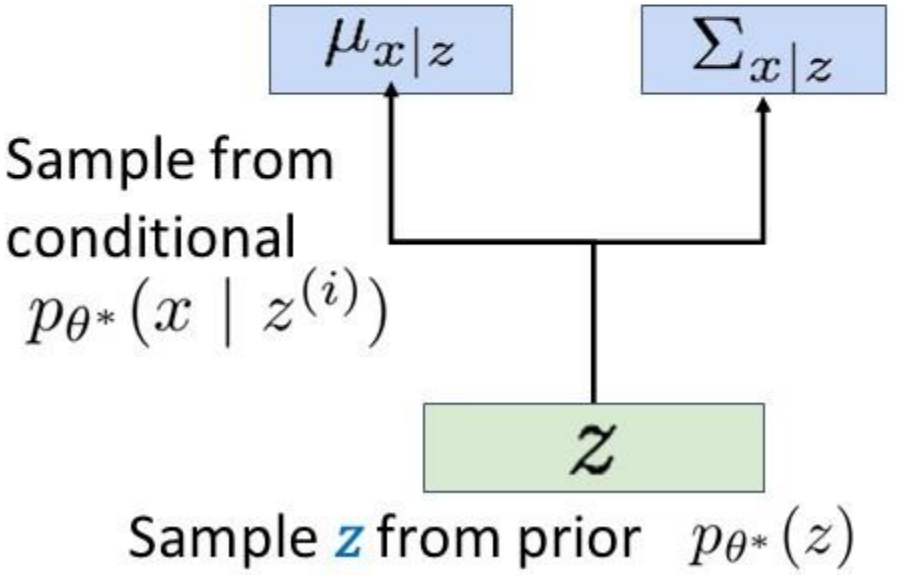


Variational Autoencoders

How to train this model?

Basic idea: **maximize likelihood of data**

If we could observe the z for each x, then could train a *conditional generative model* p(x|z)



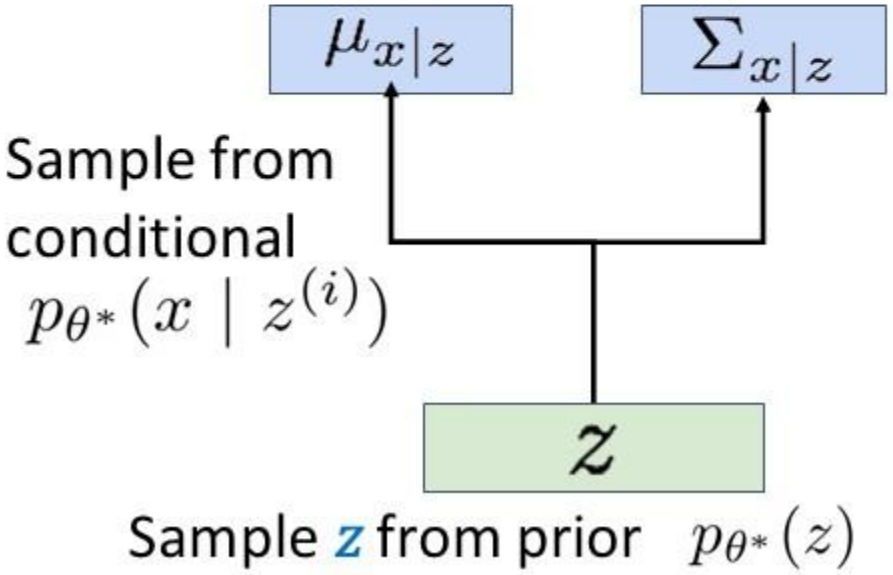
Variational Autoencoders

How to train this model?

Basic idea: **maximize likelihood of data**

But, we don't observe z (latent), so we need to marginalize it.

$$p_\theta(x) = \int p_\theta(x, z)\, dz = \int p_\theta(x|z) p_\theta(z)\, dz$$



Variational Autoencoders

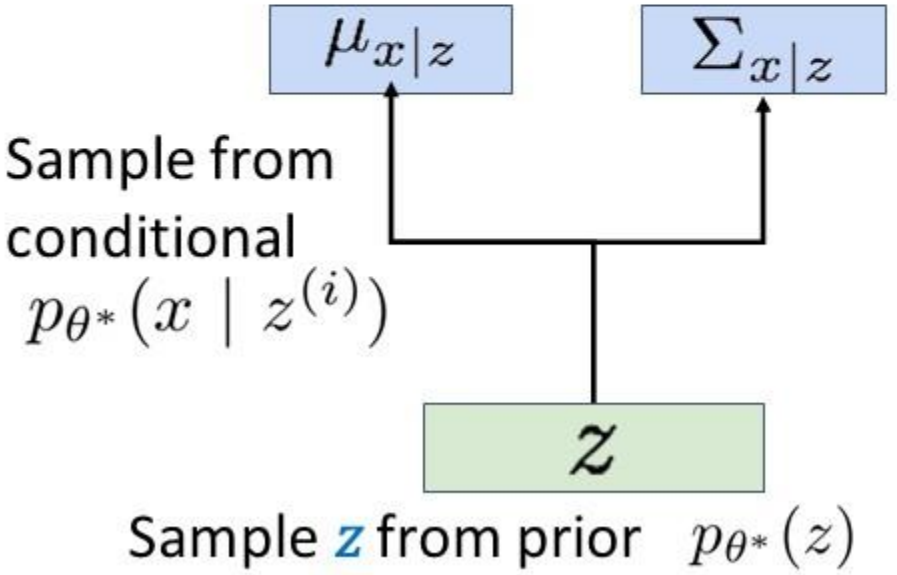
How to train this model?

Basic idea: **maximize likelihood of data**

But, we don't observe z (latent), so we need to marginalize it.

$$p_\theta(x) = \int p_\theta(x|z) p_\theta(z)\, dz$$

Ok, can compute this with decoder network



Variational Autoencoders

How to train this model?

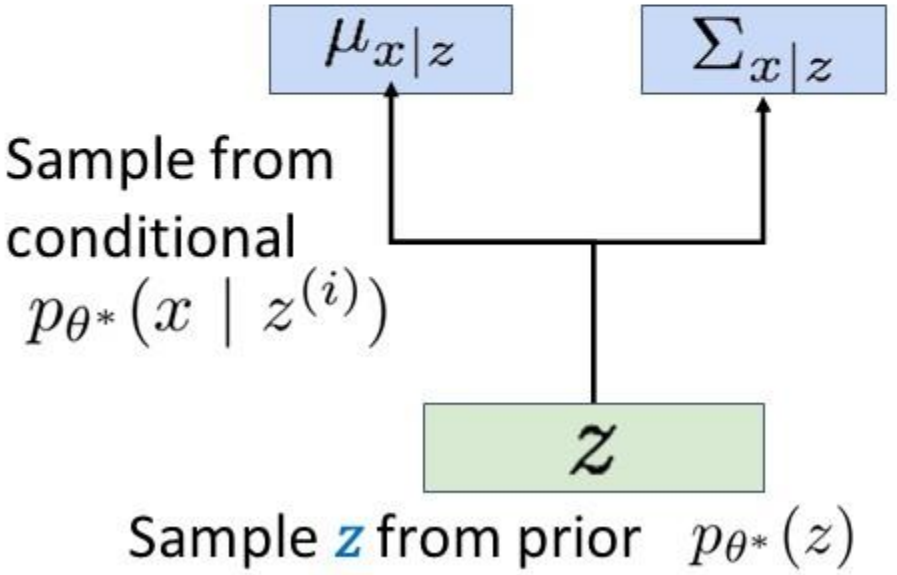
Basic idea: **maximize likelihood of data**

But, we don't observe z (latent), so we need to marginalize it.

$$p_\theta(x) = \int p_\theta(x|z)p_\theta(z)\,dz$$

Ok, we assumed
Gaussian prior for z

$$p(z) = \mathcal{N}(z, 0, I)$$



Variational Autoencoders

How to train this model?

Basic idea: **maximize likelihood of data**

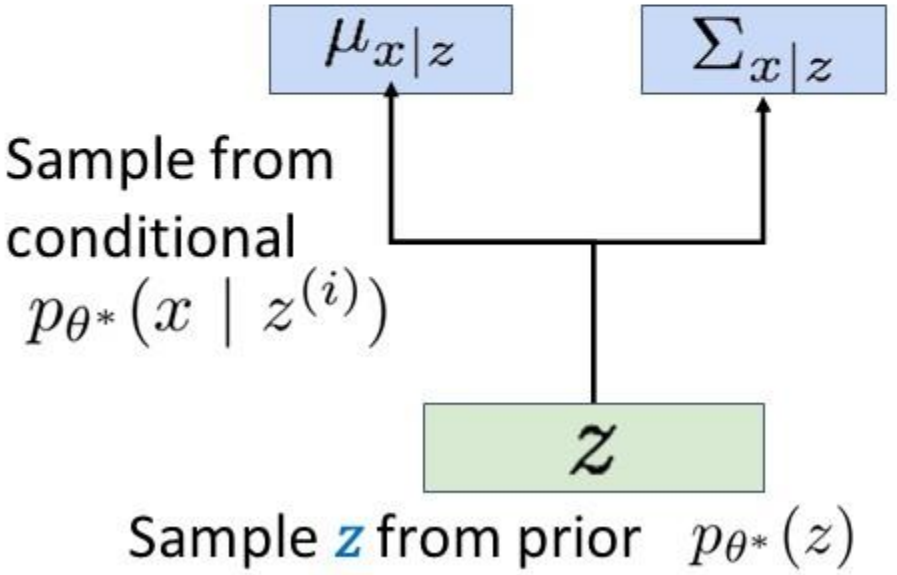
But, we don't observe z (latent), so we need to marginalize it.

Problem: Impossible to integrate over all z!

$$p_\theta(x) = \int p_\theta(x|z)p_\theta(z)\,dz$$

Integration over all possible configurations of z, each weighted by likelihood $p(x|z)$ and own prior probability $p(z)$.

For high-dimensional z, this computation becomes mathematically and computationally prohibitive.



Variational Autoencoders

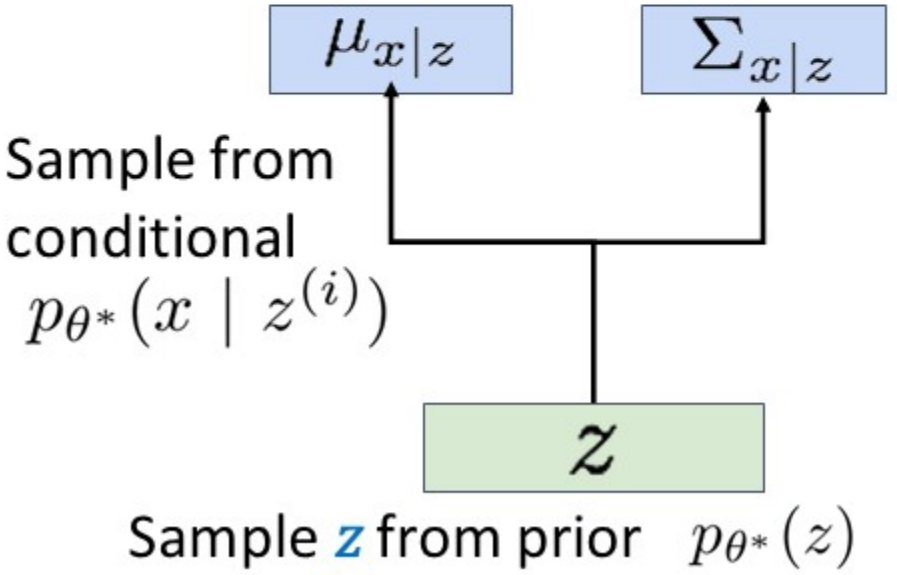
How to train this model?

Basic idea: **maximize likelihood of data**

Another idea: Try Bayes' Rule:

$$p_\theta(x) = \frac{p_\theta(x|z)p(z)}{p_\theta(z|x)}$$

Ok, can compute this with decoder network



Variational Autoencoders

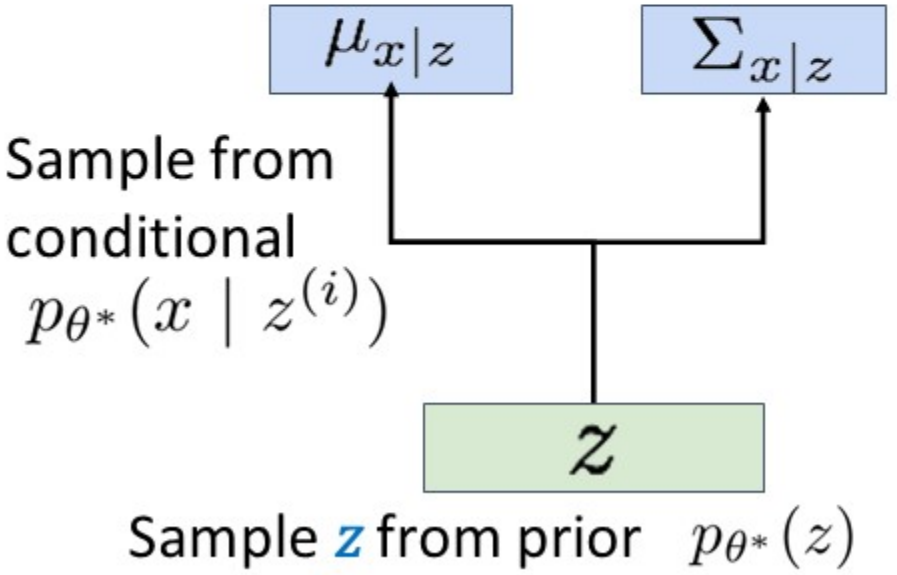
How to train this model?

Basic idea: **maximize likelihood of data**

Another idea: Try Bayes' Rule:

$$p_\theta(x) = \frac{p_\theta(x|z)p_\theta(z)}{p_\theta(z|x)}$$

Ok, we assumed
Gaussian prior for z



Variational Autoencoders

How to train this model?

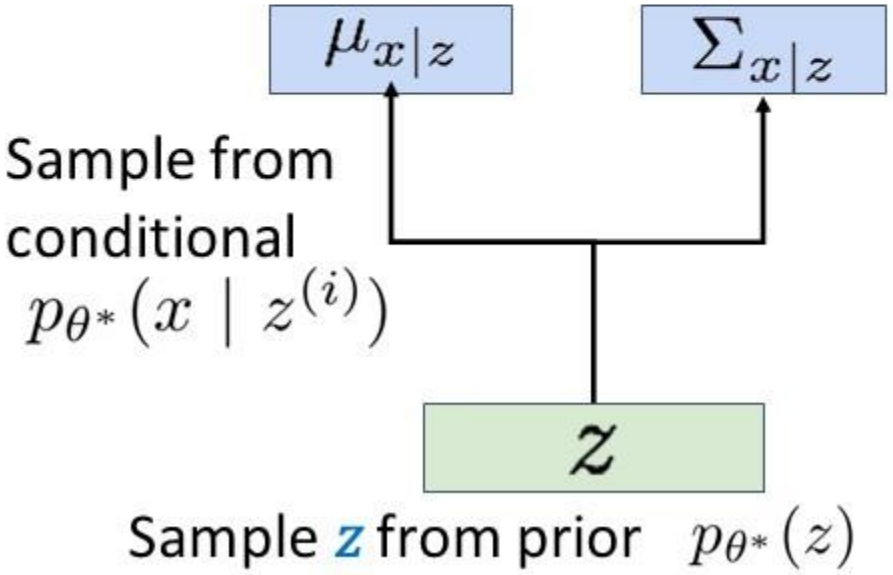
Basic idea: **maximize likelihood of data**

Another idea: Try Bayes' Rule:

$$p_\theta(x) = \frac{p_\theta(x|z)p_\theta(z)}{p_\theta(z|x)}$$

Problem: No way to compute this!

With this network



Variational Autoencoders

How to train this model?

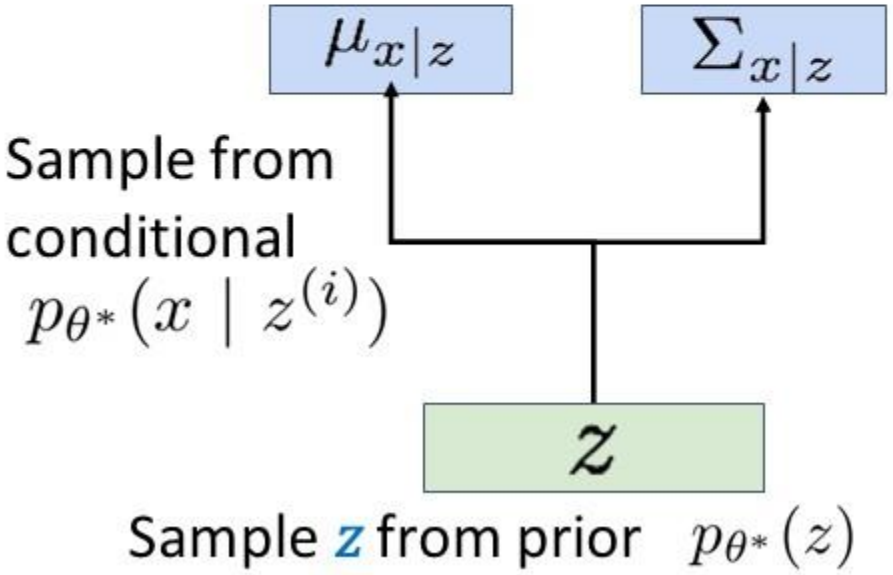
Basic idea: **maximize likelihood of data**

Another idea: Try Bayes' Rule:

$$p_\theta(x) = \frac{p_\theta(x|z)p_\theta(z)}{p_\theta(z|x)}$$

Solution: Train another network **(encoder)** that learns

$$q_\phi(z|x) \approx p_\theta(z|x)$$



Variational Autoencoders

How to train this model?

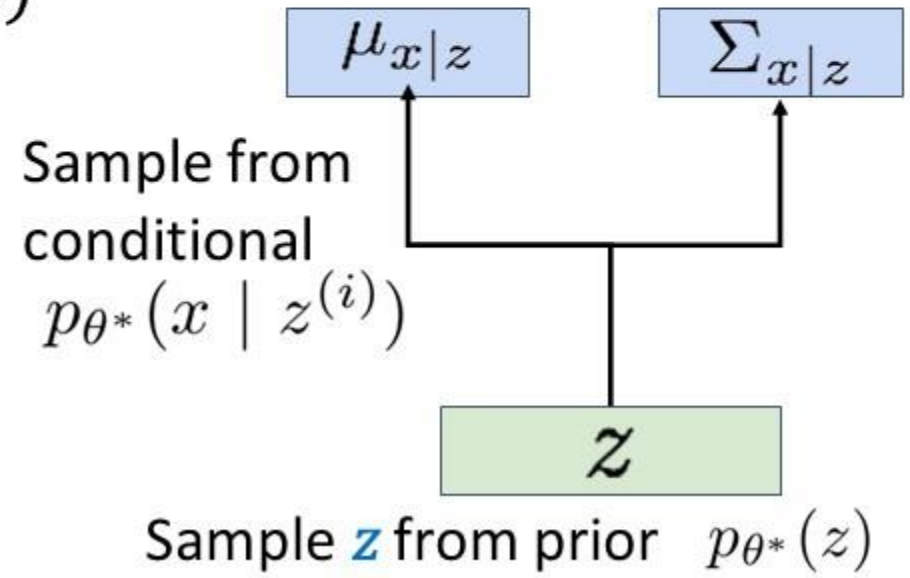
Basic idea: **maximize likelihood of data**

Another idea: Try Bayes' Rule:

$$p_\theta(x) = \frac{p_\theta(x|z)p_\theta(z)}{p_\theta(z|x)} \approx \frac{p_\theta(x|z)p_\theta(z)}{q_\phi(z|x)}$$

Use **encoder** to compute $q_\phi(z|x) \approx p_\theta(z|x)$

$$q_\phi(z|x) \approx p_\theta(z|x)$$



Variational Autoencoders

How to train this model?

Basic idea: **maximize likelihood of data**

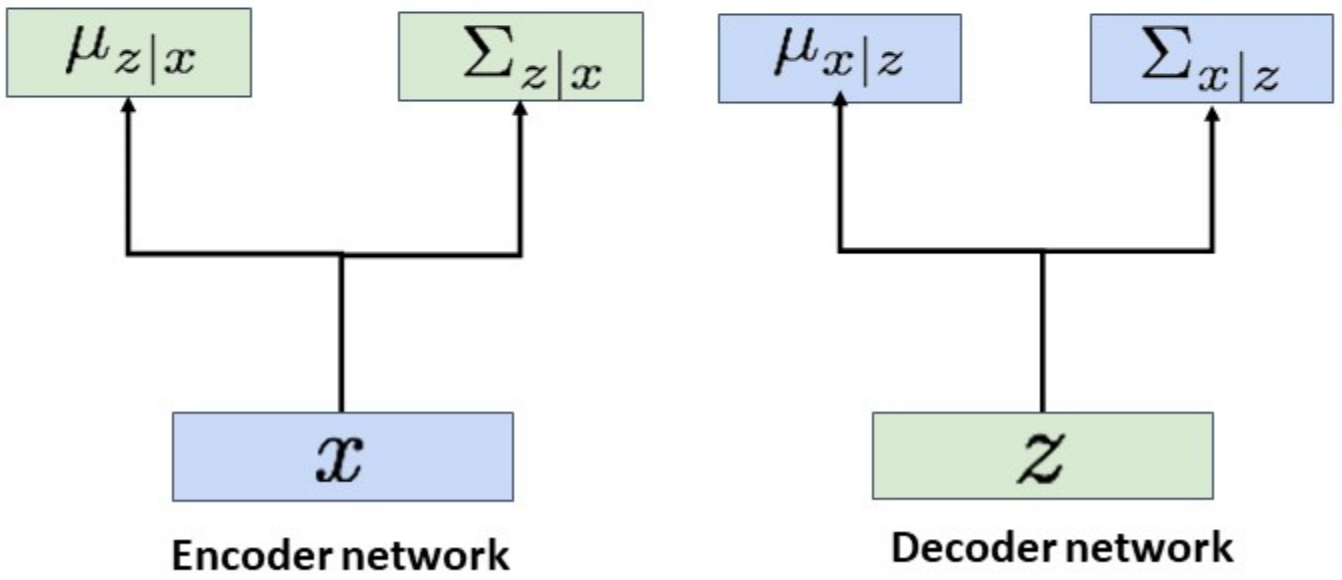
Another idea: Try Bayes' Rule:

If we can ensure that $q_\phi(z|x) \approx p_\theta(z|x)$

Then we can approximate

$$p_\theta(x) \approx \frac{p_\theta(x|z)p_\theta(z)}{q_\phi(z|x)}$$

Idea: Jointly train both encoder and decoder



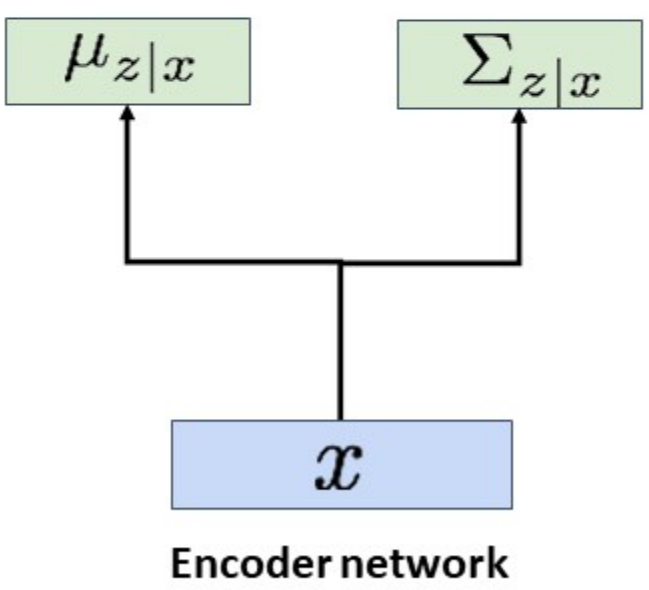
Variational Autoencoders

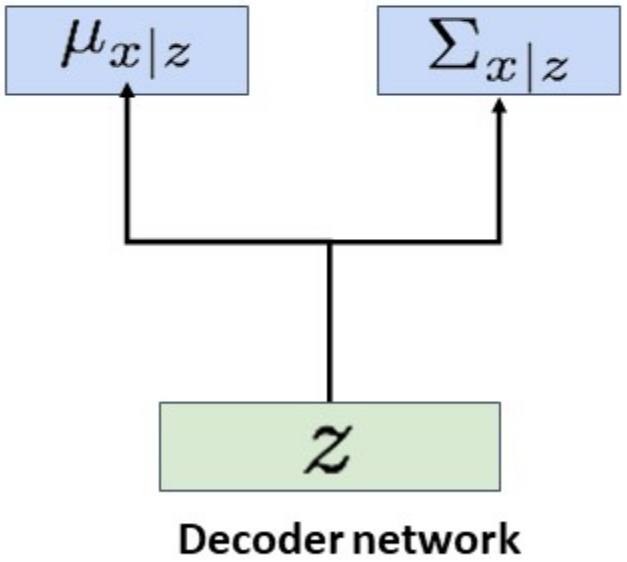
Encoder network inputs data x, gives distribution over latent codes z

Decoder network inputs latent code z, gives distribution over data x

$$q_\phi(z|x) = \mathcal{N}(\mu_{z|x}, \Sigma_{z|x})$$

$$p_\theta(x|z) = \mathcal{N}(\mu_{x|z}, \Sigma_{x|z})$$





Variational Autoencoders

How to train this model?

Basic idea: **maximize likelihood of data**

Instead of Maximizing likelihood $p_\theta(x)$, we can maximize loglikelihood: $\log p_\theta(x)$, as log is monotonically increasing function

$$\log p_\theta(x) = \log \frac{p_\theta(x|z)p(z)}{p_\theta(z|x)}$$

Variational Autoencoders

How to train this model?

Basic idea: **maximize likelihood of data**

Instead of Maximizing likelihood $p_\theta(x)$, we can maximize loglikelihood: $\log p_\theta(x)$, as log is monotonically increasing function

$$\log p_\theta(x) = \log \frac{p_\theta(x|z)p(z)}{p_\theta(z|x)} = \log \frac{p_\theta(x|z)p(z)q_\phi(z|x)}{p_\theta(z|x)q_\phi(z|x)}$$

Multiply top and bottom by $q_\phi(z|x)$

Variational Autoencoders

How to train this model?

Basic idea: **maximize likelihood of data**

Instead of Maximizing likelihood $p_\theta(x)$, we can maximize loglikelihood: $\log p_\theta(x)$, as log is monotonically increasing function

$$\log p_\theta(x) = \log \frac{p_\theta(x|z)p(z)}{p_\theta(z|x)} = \log \frac{p_\theta(x|z)p(z)q_\phi(z|x)}{p_\theta(z|x)q_\phi(z|x)}$$

$$\log p_\theta(x) = \log p_\theta(x|z) - \log \frac{q_\phi(z|x)}{p(z)} + \log \frac{q_\phi(z|x)}{p_\theta(z|x)}$$

Split up using rules for logarithms

Variational Autoencoders

How to train this model?

Basic idea: **maximize likelihood of data**

Instead of Maximizing likelihood $p_\theta(x)$, we can maximize loglikelihood: $\log p_\theta(x)$, as log is monotonically increasing function

$$\log p_\theta(x) = \log \frac{p_\theta(x|z)p(z)}{p_\theta(z|x)} = \log \frac{p_\theta(x|z)p(z)q_\phi(z|x)}{p_\theta(z|x)q_\phi(z|x)}$$

$$\log p_\theta(x) = \log p_\theta(x|z) - \log \frac{q_\phi(z|x)}{p(z)} + \log \frac{q_\phi(z|x)}{p_\theta(z|x)}$$

Split up using rules for logarithms

Variational Autoencoders

By definition, Expected value of a function $p(x)$ with respect to a probability distribution $q(z|x)$

$$\mathbb{E}_{z\sim q(z/x)}[p(x)] = \int p(x)q(z|x)\,dz$$

Where $z \sim q(z/x)$ means latent variable z is being sampled or drawn from the distribution $q(z|x)$

Since, $p(x)$ is function x only and not of z. It is independent with respect to z or any probability distribution of z like $q(z|x)$, thus

$$\mathbb{E}_{z\sim q(z/x)}[p(x)] = \int p(x)q(z|x)\,dz = p(x)$$

Variational Autoencoders

The marginal probability $p(x)$ represents the overall likelihood of observing the data x regardless of the latent variables z.

$p(x)$ is a constant with respect to z (it does not change no matter what z values are), its expectation under any distribution related to z remains the same.

In general, for any random variable Z with distribution $q(z|x)$, the expectation of a constant c is just c:

$$\mathbb{E}_{z\sim q(z/x)}[c] = c$$

Similarly, $$\log p(x) = \mathbb{E}_{z\sim q(z/x)}[\log p(x)]$$

Variational Autoencoders

How to train this model?

Basic idea: **maximize likelihood of data**

Instead of Maximizing likelihood $p_\theta(x)$, we can maximize loglikelihood: $\log p_\theta(x)$, as log is monotonically increasing function

$$\log p_\theta(x) = \log p_\theta(x|z) - \log \frac{q_\phi(z|x)}{p(z)} + \log \frac{q_\phi(z|x)}{p_\theta(z|x)}$$

$$\log p_\theta(x) = \mathbb{E}_{z\sim q(z/x)}[\log p_\theta(x)]$$

$$\log p_\theta(x) = \boxed{\mathbb{E}_z[\log p_\theta(x|z)]} - \mathbb{E}_z\left[\log \frac{q_\phi(z|x)}{p(z)}\right] + \mathbb{E}_z\left[\log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$$

Data reconstruction

VAE: Reconstruction Loss

$$\mathbb{E}_{z\sim q(z/x)}[\log p_\theta(x|z)]$$

- This expectation is taken over $q_\phi(z/x)$, the distribution of the latent variables z, conditioned on the input x modeled by the encoder.
- The function $p_\theta(x|z)$ is the likelihood of the data x given the latent variables z, as modeled by the decoder. This term measures the probability of reconstructing the input x accurately from the latent representation z.

VAE: Reconstruction Loss

$$\mathbb{E}_{z\sim q(z/x)}[\log p_\theta(x|z)]$$

- In practical VAE implementations (especially those dealing with continuous data),
- Log-likelihood $\log p_\theta(x|z)$ is modeled by a Gaussian distribution with a mean of $\hat{x}$ and a fixed variance.

$$p(x|z) = \mathcal{N}\left(x; \mu_{x|z}, \Sigma_{x|z}\right)$$

$$p(x|z) = \mathcal{N}(x; \hat{x}, \sigma^2 I)$$

- Here, the mean of distribution μ = reconstructed data $\hat{x}$, , and
- $\Sigma_{x|z}$ is a diagonal covariance matrix $\sigma^2 I$, where σ is often treated as a constant for simplification.

VAE: Reconstruction Loss

- **Gaussian distribution:**

$$p(x|z) = \frac{1}{\sqrt{(2\pi)^d \det(\Sigma_{x|z})}} \exp\left(-\frac{1}{2}(x-\mu_{x|z})^T \Sigma_{x|z} (x-\mu_{x|z})\right)$$

Note: $\mu_{x|z} = \hat{x}$, and $\Sigma_{x|z} = \sigma^2 I$,

Thus, $\det(\Sigma_{x|z}) = \sigma^{2d}$, Inverse of $\Sigma_{x|z} = \frac{1}{\sigma^2} I$,

where d is the dimension of vector x.

$$p(x|z) = \frac{1}{\sqrt{(2\pi\sigma^2)^d}} \exp\left(-\frac{1}{2}(x-\hat{x})^T (x-\hat{x})\right)$$

VAE: Reconstruction Loss

- **Gaussian distribution:**

$$p(x|z) = \frac{1}{\sqrt{(2\pi\sigma^2)^d}} \exp\left(-\frac{1}{2\sigma^2}(x-\hat{x})^T(x-\hat{x})\right)$$

- Since $(x-\hat{x})$ is a vector, $(x-\hat{x})^T(x-\hat{x}) = \sum_d (x_i - \hat{x}_i)^2$.

- **Log-Likelihood**

$$\log p(x|z) = \log\left[\frac{1}{\sqrt{(2\pi\sigma^2)^d}} \exp\left(-\frac{1}{2\sigma^2}\sum_d (x_i - \hat{x}_i)^2\right)\right]$$

VAE: Reconstruction Loss

- **Log-Likelihood**

$$\log p(x|z) = \log\left[\frac{1}{\sqrt{(2\pi\sigma^2)^d}}\exp\left(-\frac{1}{2\sigma^2}\sum_d (x_i - \hat{x}_i)^2\right)\right]$$

$$\log p(x|z) = \log\left[\frac{1}{\sqrt{(2\pi\sigma^2)^d}}\right] + \log\left[\exp\left(-\frac{1}{2\sigma^2}\sum_d (x_i - \hat{x}_i)^2\right)\right]$$

$$\log p(x|z) = -\frac{1}{2\sigma^2}\sum_d (x_i - \hat{x}_i)^2 + \log\left[\frac{1}{\sqrt{(2\pi\sigma^2)^d}}\right]$$

- Note: Since, σ^2 is assumed to be fixed, term $\log\left[\frac{1}{\sqrt{(2\pi\sigma^2)^d}}\right]$ is considered as a constant with respect to the parameters being optimized.

VAE: Reconstruction Loss

- Thus, for many practical VAE implementations (especially those dealing with continuous data), Log-likelihood $\log p_\theta(x|z)$ is

$$\log p(x|z) = -\frac{1}{2\sigma^2}\|\boldsymbol{x} - \hat{\boldsymbol{x}}\|_2^2 + constant$$

- Therefore, MSE is used as a proxy for the negative log-likelihood in many practical VAE applications when is assumed constant.

- i.e. minimize $\|\boldsymbol{x} - \hat{\boldsymbol{x}}\|_2^2$ (reconstruction loss) to maximized log-likelihood i.e. $\mathbb{E}_{z\sim q(z/x)}[\log p_\theta(x|z)]$

VAE: Reconstruction Loss

- Similarly, in case of VAEs dealing with binary data:
- The conditional distribution of each data point x_i given z is modeled as a Bernoulli distribution, which is suitable for binary data:

$$p(x_i|z) = \hat{x}_i^{x_i}(1 - \hat{x}_i)^{(1-x_i)}$$

- Here $\hat{x}_i$ is the probability of x_i being 1 as predicted by the decoder (parameterized by θ), given the latent variable z.

VAE: Reconstruction Loss

- **Log-likelihood of Bernoulli distribution:** The log-likelihood of observing the entire data vector $\boldsymbol{x}$ (let n elements in data vector)given z, under the Bernoulli model, is the sum of the log-likelihoods of individual observations:

$$\log p(\boldsymbol{x}|\boldsymbol{z}) = \sum_{i=1}^{n} \log p(x_i|\boldsymbol{z}) = \sum_{i=1}^{n} [x_i \log \hat{x}_i + (1 - x_i) \log(1 - \hat{x}_i)]$$

- For binary classification, the binary cross-entropy between the true labels x_i and predicted probabilities $\hat{x}_i$ is given by:

$$Binary\ Cross - Entropy = -\sum_{i=1}^{n} [x_i \log \hat{x}_i + (1 - x_i) \log(1 - \hat{x}_i)]$$

VAE: Reconstruction Loss

- Thus,

$$\text{Log-likelihood} = -\,Binary\ Cross\ Entropy$$

- Therefore, maximizing the log-likelihood $\log p(\boldsymbol{x}|\boldsymbol{z})$ of the Bernoulli distribution (which is what we want in the context of maximum likelihood estimation) is equivalent to minimizing the binary cross-entropy loss.

Variational Autoencoders

How to train this model?

Basic idea: **maximize likelihood of data**

Instead of Maximizing likelihood $p_\theta(x)$, we can maximize loglikelihood: $\log p_\theta(x)$, as log is monotonically increasing function

$$\log p_\theta(x) = \mathbb{E}_z[\log p_\theta(x|z)] - \mathbb{E}_z\left[\log \frac{q_\phi(z|x)}{p(z)}\right] + \mathbb{E}_z\left[\log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$$

$$\log p_\theta(x) = \boxed{\mathbb{E}_z[\log p_\theta(x|z)]} - \boxed{D_{KL}\big(q_\phi(z|x)\|p(z)\big)} + D_{KL}\big(q_\phi(z|x)\|p_\theta(z|x)\big)$$

Data reconstruction

KL divergence between prior, and samples from the encoder network

Kullback-Leibler (KL)divergence

- KL divergence used to measure the difference between two probability distributions.
- **Definition:** KL divergence from a probability distribution p to another distribution q over the same variable x is defined as:

$$D_{KL}(p\|q) = \sum_{x} p(x) \log \frac{p(x)}{q(x)}$$

$$D_{KL}(p\|q) = \mathbb{E}_{x\sim p(x)} \left[\log \frac{p(x)}{q(x)} \right]$$

- Where p and q are probability distributions.
- For continuous distributions, the sum is replaced by an integral.

Kullback-Leibler (KL)divergence

$$D_{KL}(p\|q) = \mathbb{E}_{x\sim p(x)}\left[\log\frac{p(x)}{q(x)}\right]$$

- **Interpretation:** KL divergence quantifies how much information is lost when q is used to approximate p.

✓ It is a measure of inefficiency and discrepancy, indicating how one distribution diverges from another.

- **Asymmetry:** It is important to note that KL divergence is not symmetric.

✓ This means that in general $D_{KL}(p\|q)$ not same as $D_{KL}(q\|p)$

- **Non-negativity:** The value of KL divergence is always non-negative.

$$D_{KL}(p\|q) \geq 0$$

✓ It equals zero if and only if p and q are exactly the same distribution over the x.

Variational Autoencoders

How to train this model?

Basic idea: **maximize likelihood of data**

Instead of Maximizing likelihood $p_\theta(x)$, we can maximize loglikelihood: $\log p_\theta(x)$, as log is monotonically increasing function

$$\log p_\theta(x) = \mathbb{E}_z[\log p_\theta(x|z)] - \mathbb{E}_z\left[\log \frac{q_\phi(z|x)}{p(z)}\right] + \mathbb{E}_z\left[\log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$$

$$\log p_\theta(x) = \boxed{\mathbb{E}_z[\log p_\theta(x|z)]} - \boxed{D_{KL}\big(q_\phi(z|x)\|p(z)\big)} + D_{KL}\big(q_\phi(z|x)\|p_\theta(z|x)\big)$$

Data reconstruction

KL divergence between prior, and samples from the encoder network

Variational Autoencoders

How to train this model?

Basic idea: **maximize likelihood of data**

Instead of Maximizing likelihood $p_\theta(x)$, we can maximize loglikelihood: $\log p_\theta(x)$, as log is monotonically increasing function

$$\log p_\theta(x) = \mathbb{E}_z[\log p_\theta(x|z)] - \mathbb{E}_z\left[\log \frac{q_\phi(z|x)}{p(z)}\right] + \mathbb{E}_z\left[\log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$$

$$\log p_\theta(x) = \mathbb{E}_z[\log p_\theta(x|z)] - D_{KL}\big(q_\phi(z|x)\|p(z)\big) + D_{KL}\big(q_\phi(z|x)\|p_\theta(z|x)\big)$$

KL divergence between encoder and posterior of decoder

Problem: No way to compute this! as it involves $p_\theta(z|x)$

Variational Autoencoders

How to train this model?

Basic idea: **maximize likelihood of data**

Instead of Maximizing likelihood $p_\theta(x)$, we can maximize loglikelihood: $\log p_\theta(x)$, as log is monotonically increasing function

$$\log p_\theta(x) = \mathbb{E}_z[\log p_\theta(x|z)] - \mathbb{E}_z\left[\log \frac{q_\phi(z|x)}{p(z)}\right] + \mathbb{E}_z\left[\log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$$

$$\log p_\theta(x) = \mathbb{E}_z[\log p_\theta(x|z)] - D_{KL}\big(q_\phi(z|x)\|p(z)\big) + D_{KL}\big(q_\phi(z|x)\|p_\theta(z|x)\big)$$

However, KL is >= 0, so dropping this term gives a **lower bound** on the data likelihood:

Variational Autoencoders

How to train this model?

Basic idea: **maximize likelihood of data**

Instead of Maximizing likelihood $p_\theta(x)$, we can maximize loglikelihood: $\log p_\theta(x)$, as log is monotonically increasing function

$$\log p_\theta(x) = \mathbb{E}_z[\log p_\theta(x|z)] - \mathbb{E}_z\left[\log \frac{q_\phi(z|x)}{p(z)}\right] + \mathbb{E}_z\left[\log \frac{q_\phi(z|x)}{p_\theta(z|x)}\right]$$

$$\log p_\theta(x) = \mathbb{E}_z[\log p_\theta(x|z)] - D_{KL}\big(q_\phi(z|x)\|p(z)\big) + D_{KL}\big(q_\phi(z|x)\|p_\theta(z|x)\big)$$

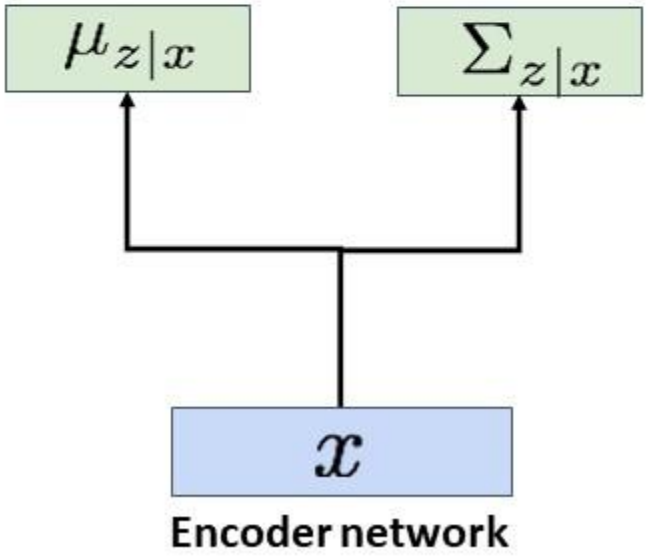
$$\log p_\theta(x) \geq \mathbb{E}_{z\sim q_\phi(z/x)}[\log p_\theta(x|z)] - D_{KL}\big(q_\phi(z|x)\|p(z)\big)$$

Variational Autoencoders

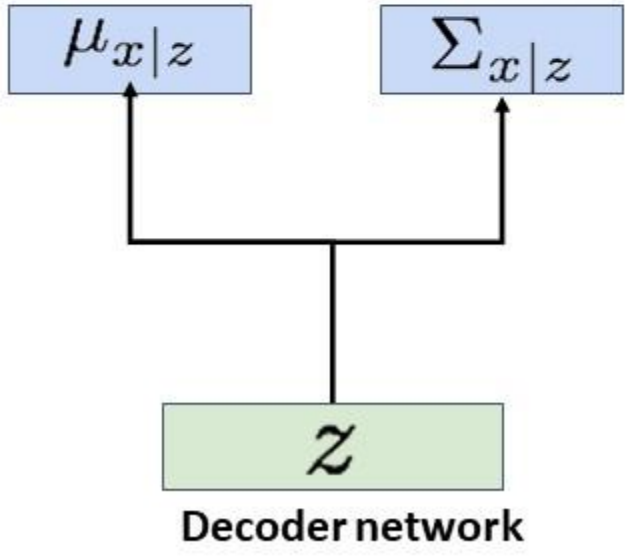
Jointly train **encoder** q and **decoder** p to maximize the **variational lower bound** on the data likelihood Also called optimization of **Evidence Lower Bound** (**ELBO**)

$$\log p_\theta(x) \geq \mathbb{E}_{z\sim q_\phi(z/x)}[\log p_\theta(x|z)] - D_{KL}\big(q_\phi(z|x)\|p(z)\big)$$

$q_\phi(z|x) = \mathcal{N}(\mu_{z|x}, \Sigma_{z|x})$



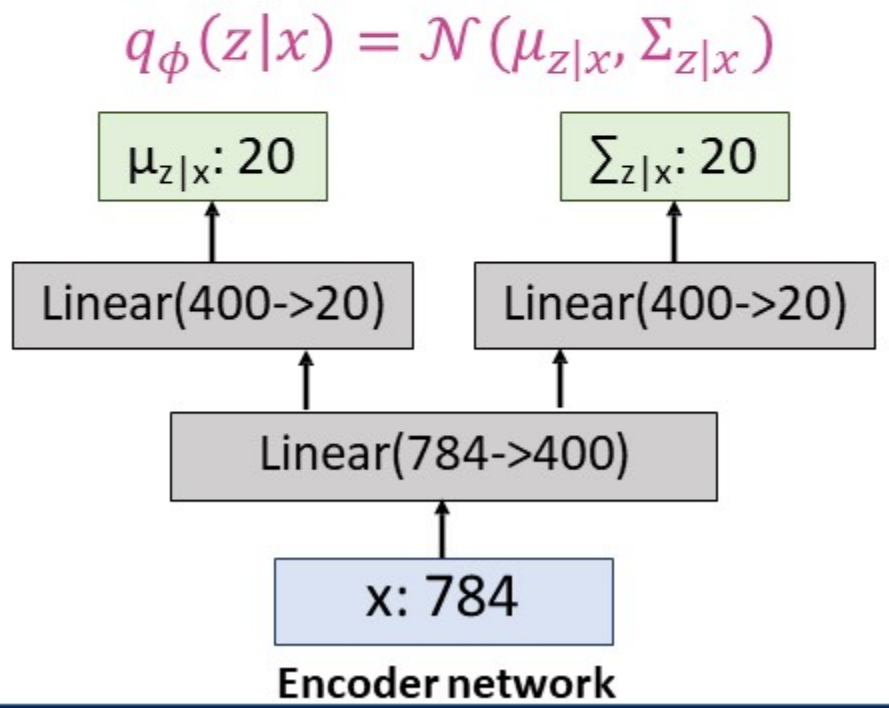
$p_\theta(x|z) = \mathcal{N}(\mu_{x|z}, \Sigma_{x|z})$

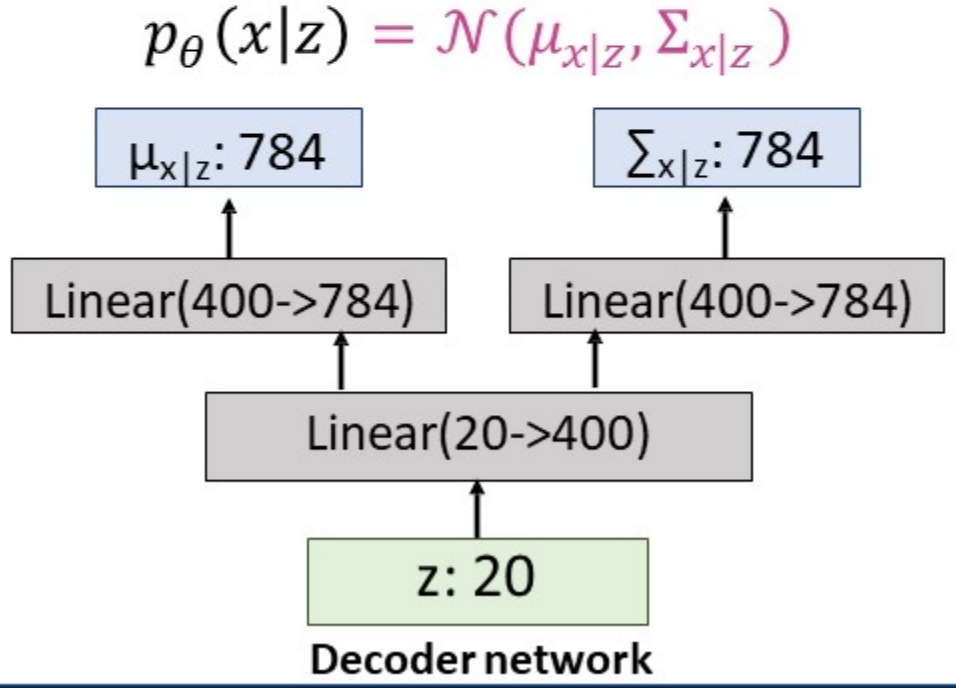


Example: Fully-Connected VAE

x: 28x28 image, flattened to 784-dim vector

z: 20-dim vector





Variational Autoencoders

Train by maximizing the
variational lower bound

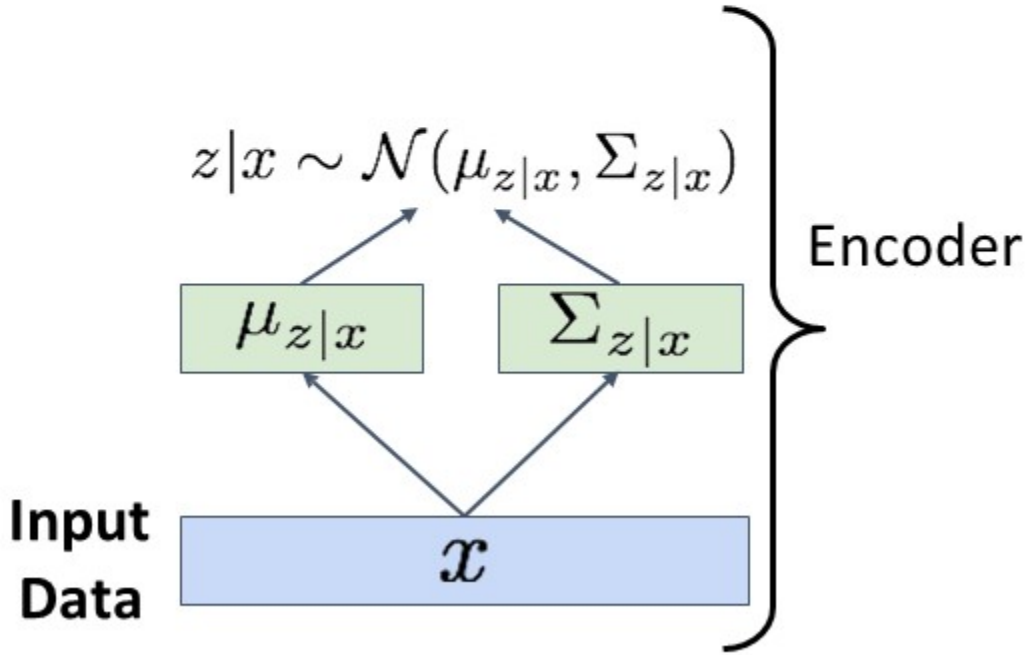
$$\mathbb{E}_{z\sim q_\phi(z/x)}[\log p_\theta(x|z)] - D_{KL}\big(q_\phi(z|x)\|p(z)\big)$$

Variational Autoencoders

Train by maximizing the
variational lower bound

$$\mathbb{E}_{z\sim q_\phi(z/x)}[\log p_\theta(x|z)] - D_{KL}\big(q_\phi(z|x)\|p(z)\big)$$

1. Run input data through **encoder** to get a distribution over latent codes

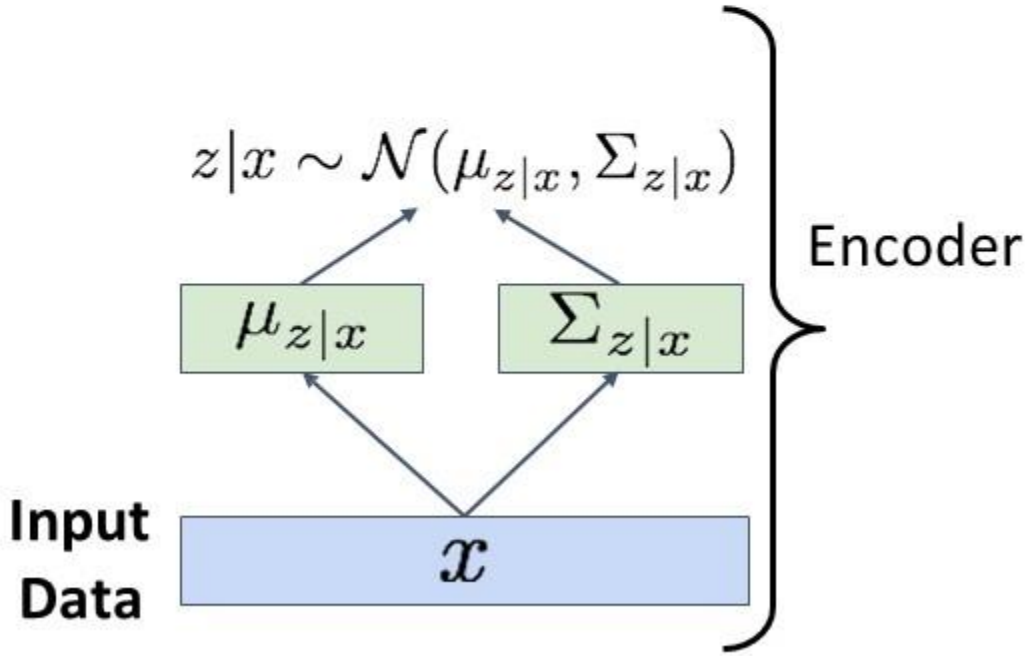


Variational Autoencoders

Train by maximizing the
variational lower bound

$$\mathbb{E}_{z\sim q_\phi(z/x)}[\log p_\theta(x|z)] - D_{KL}\big(q_\phi(z|x)\|p(z)\big)$$

1. Run input data through **encoder** to get a distribution over latent codes
2. **Encoder output should match the prior p(z)!**



Variational Autoencoders

Train by maximizing the **variational lower bound**

$$\mathbb{E}_{z\sim q_\phi(z/x)}[\log p_\theta(x|z)] - D_{KL}\big(q_\phi(z|x)\|p(z)\big)$$

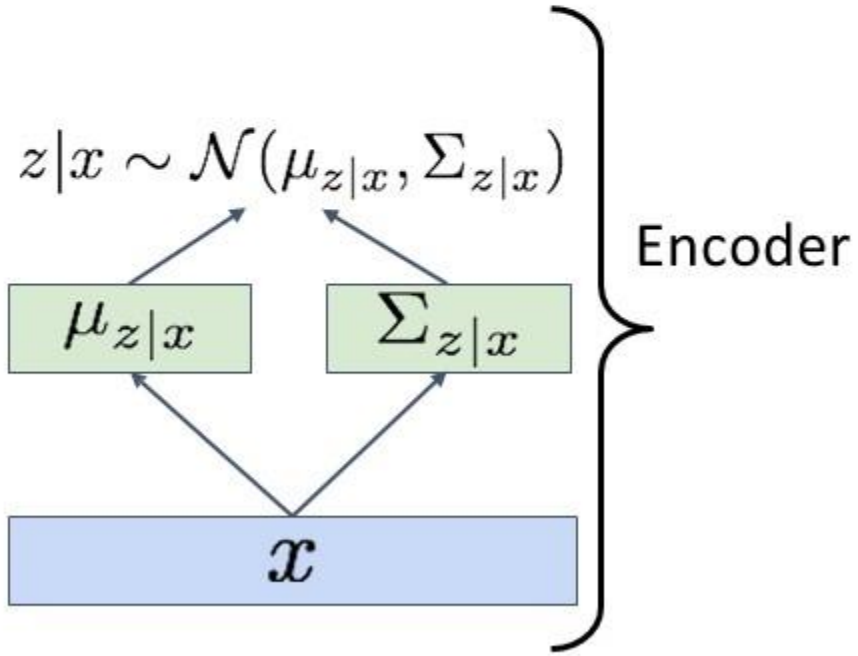
Closed form solution when q_ϕ is diagonal Gaussian and p is unit Gaussian! (Assume z has dimension J)

1. Run input data through **encoder** to get a distribution over latent codes
2. **Encoder output should match the prior p(z)!**

$$-D_{KL}\big(q_\phi(z|x)\|p(z)\big) = \int_Z q_\phi(z|x)\log\frac{p(z)}{q_\phi(z|x)}$$

$$= \int_Z \mathcal{N}(z;\,\mu_{z|x},\Sigma_{z|x})\,\log\frac{\mathcal{N}(z;\,0,I)}{\mathcal{N}(z;\,\mu_{z|x},\Sigma_{z|x})}$$

$$= \frac{1}{2}\sum_{k=1}^{d}\left(1+\log\left(\left(\Sigma_{z|x}\right)_k^2\right)-\left(\mu_{z|x}\right)_k^2-\left(\Sigma_{z|x}\right)_k^2\right)$$



Variational Autoencoders

Train by maximizing the **variational lower bound**

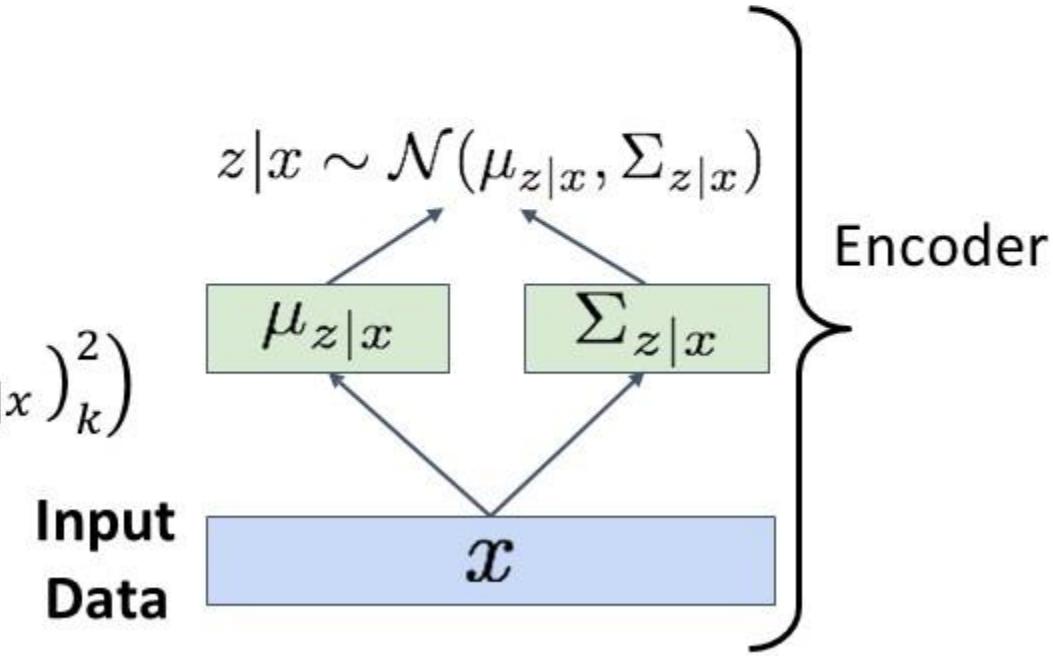
$$\mathbb{E}_{z\sim q_\phi(z/x)}[\log p_\theta(x|z)] - D_{KL}\big(q_\phi(z|x)\|p(z)\big)$$

Closed form solution when $q_)$ is diagonal Gaussian and p is unit Gaussian! (Assume z has dimension J)

1. Run input data through **encoder** to get a distribution over latent codes
2. **Encoder output should match the prior p(z)!**

$$-D_{KL}\big(q_\phi(z|x)\|p(z)\big) = \frac{1}{2}\sum_{k=1}^{d}\left(1+\log\left(\left(\Sigma_{z|x}\right)_k^2\right)-\left(\mu_{z|x}\right)_k^2-\left(\Sigma_{z|x}\right)_k^2\right)$$

Here, d is the dimensionality of the latent space, and the sum is taken over all dimensions of the latent variables.

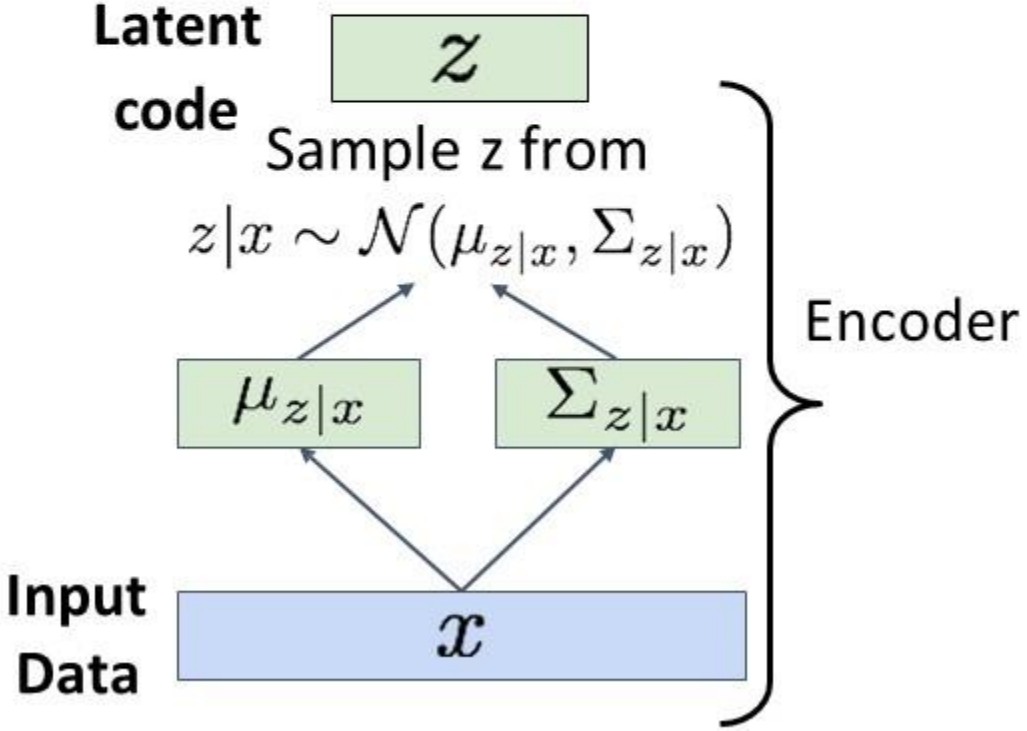


Variational Autoencoders

Train by maximizing the
variational lower bound

$$\mathbb{E}_{z\sim q_\phi(z/x)}[\log p_\theta(x|z)] - D_{KL}\big(q_\phi(z|x)\|p(z)\big)$$

1. Run input data through **encoder** to get a distribution over latent codes
2. **Encoder output should match the prior p(z)!**
3. Sample code z from encoder output

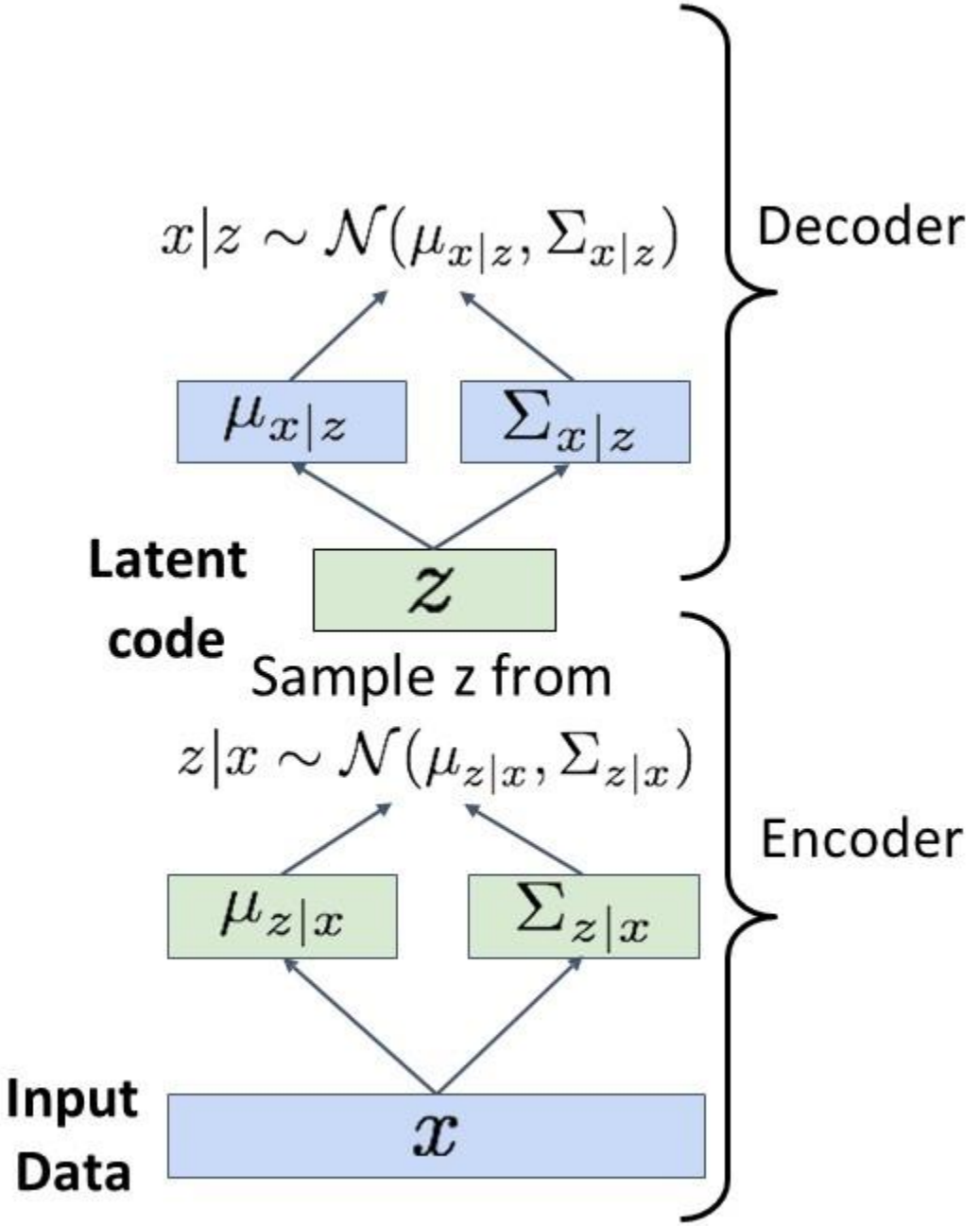


Variational Autoencoders

Train by maximizing the **variational lower bound**

$$\mathbb{E}_{z\sim q_\phi(z/x)}[\log p_\theta(x|z)] - D_{KL}\big(q_\phi(z|x)\|p(z)\big)$$

1. Run input data through **encoder** to get a distribution over latent codes
2. **Encoder output should match the prior p(z)!**
3. Sample code z from encoder output
4. Run sampled code through **decoder** to get a distribution over data samples

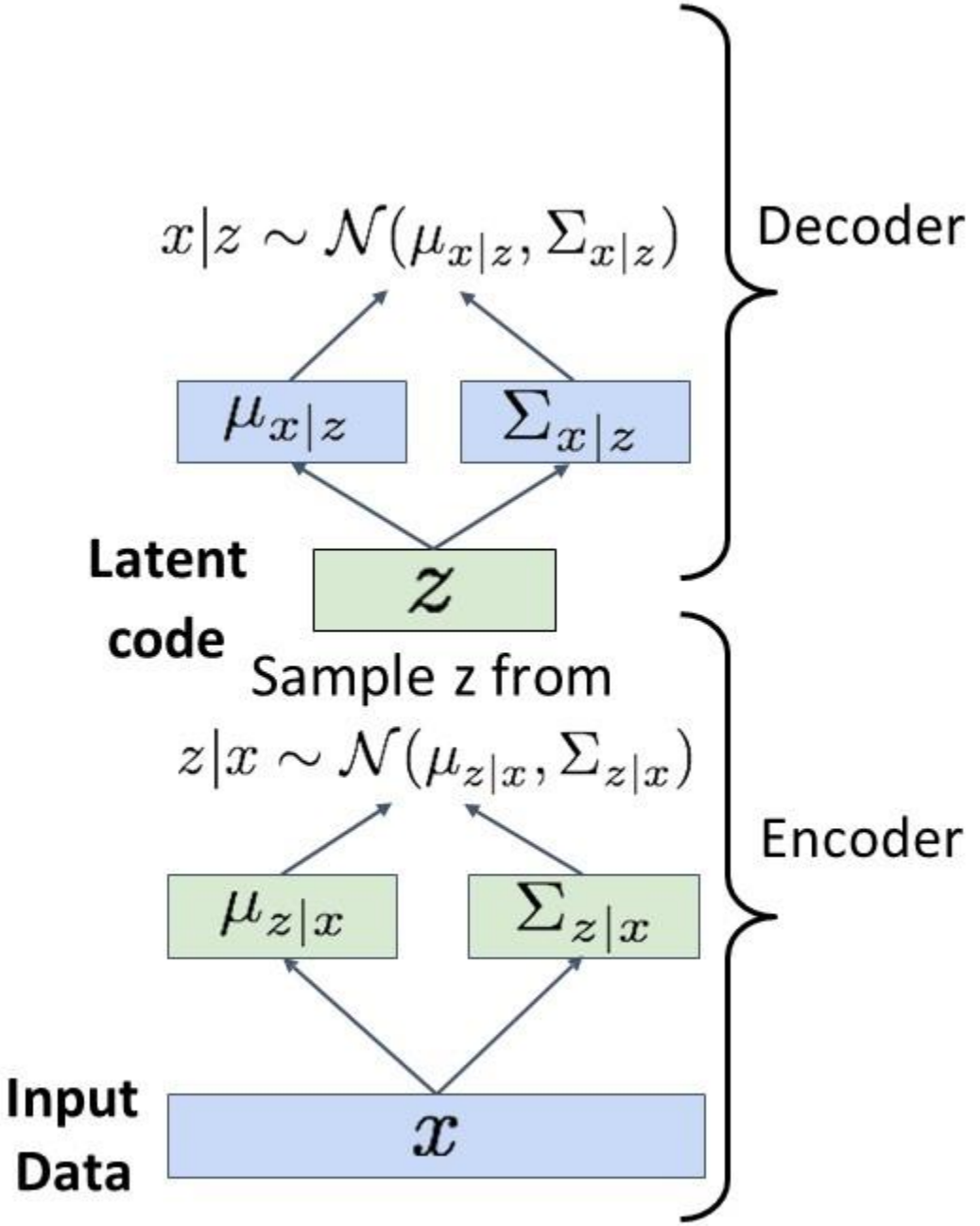


Variational Autoencoders

Train by maximizing the **variational lower bound**

$$\mathbb{E}_{z\sim q_\phi(z/x)}[\log p_\theta(x|z)] - D_{KL}\big(q_\phi(z|x)\|p(z)\big)$$

1. Run input data through **encoder** to get a distribution over latent codes
2. **Encoder output should match the prior p(z)!**
3. Sample code z from encoder output
4. Run sampled code through **decoder** to get a distribution over data samples
5. **Original input data should be likely under the distribution output from (4)!**

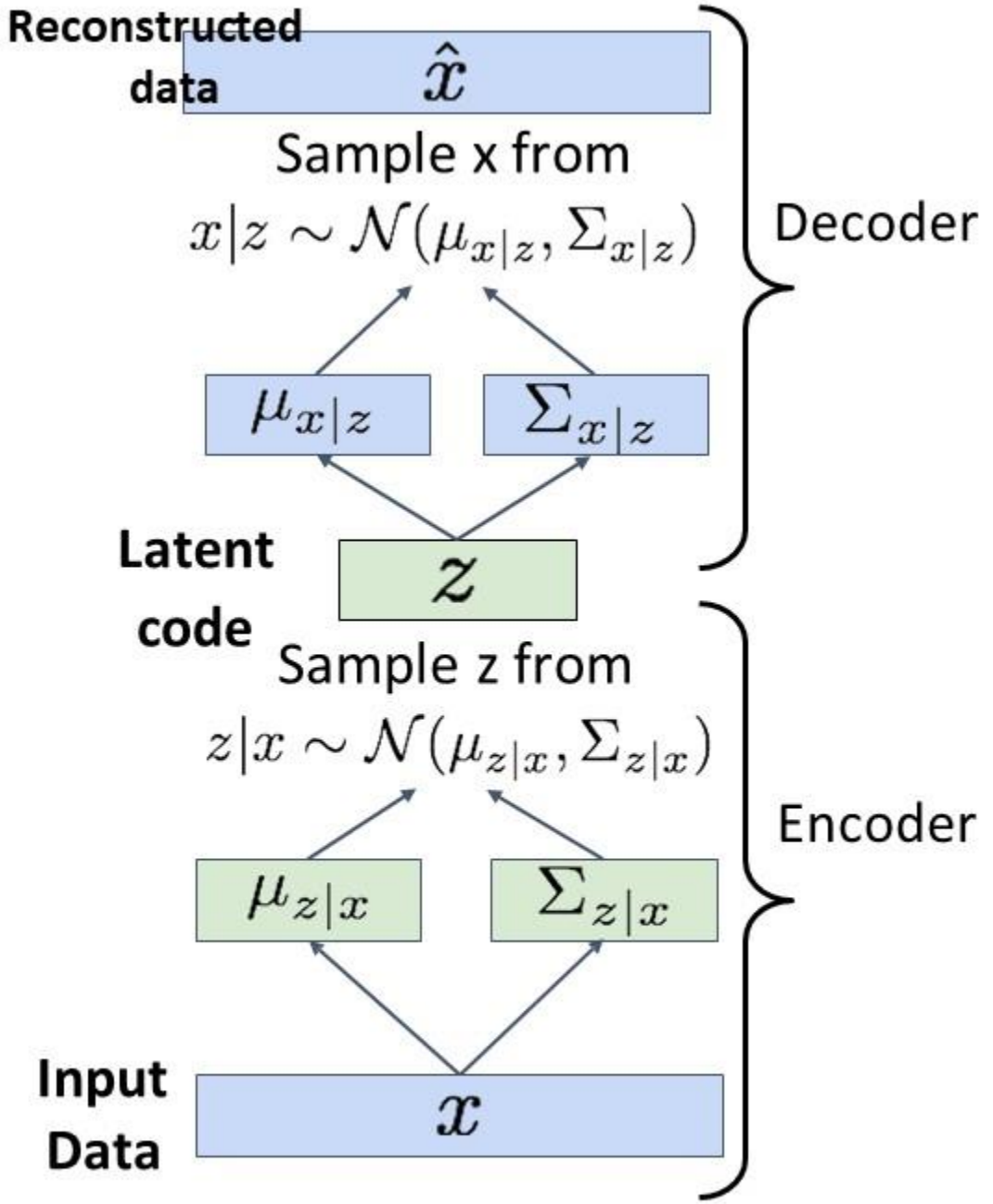


Variational Autoencoders

Train by maximizing the **variational lower bound**

$$\mathbb{E}_{z\sim q_\phi(z/x)}[\log p_\theta(x|z)] - D_{KL}\big(q_\phi(z|x)\|p(z)\big)$$

1. Run input data through **encoder** to get a distribution over latent codes
2. **Encoder output should match the prior p(z)!**
3. Sample code z from encoder output
4. Run sampled code through **decoder** to get a distribution over data samples
5. **Original input data should be likely under the distribution output from (4)!**
6. Can sample a reconstruction from (4)



Variational Autoencoders: Generating Data

After training we can generate new data!

1. Sample z from prior p(z)

Latent code

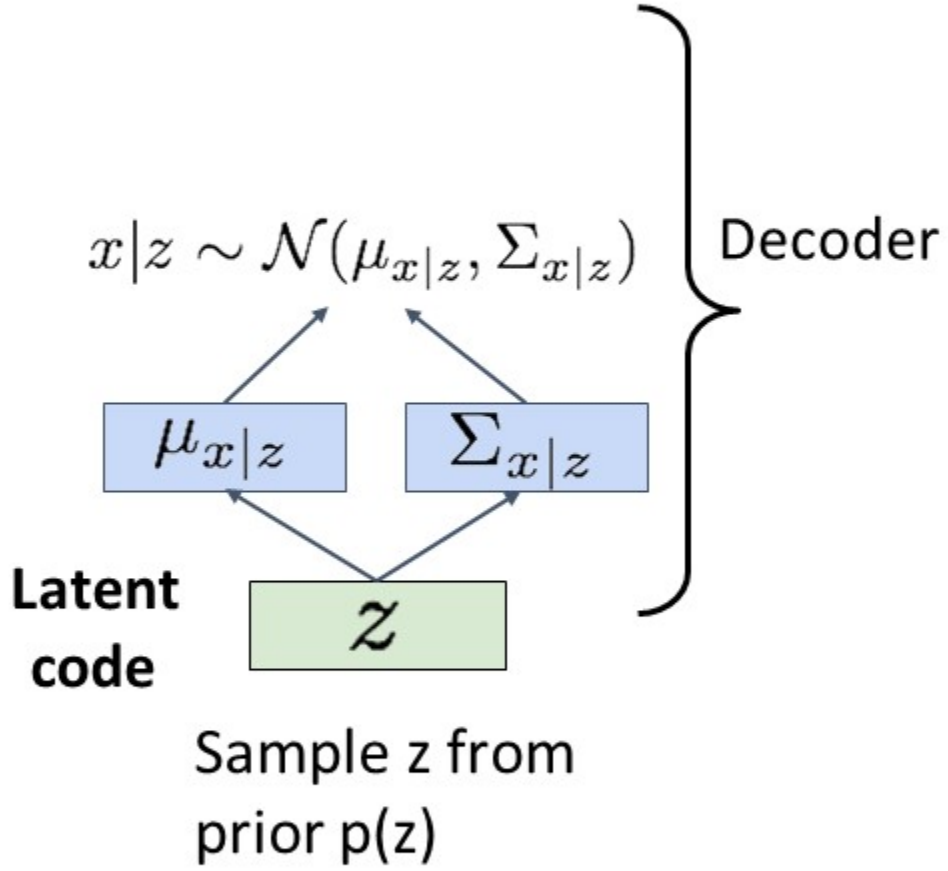
z

Sample z from prior p(z)

Variational Autoencoders: Generating Data

After training we can generate new data!

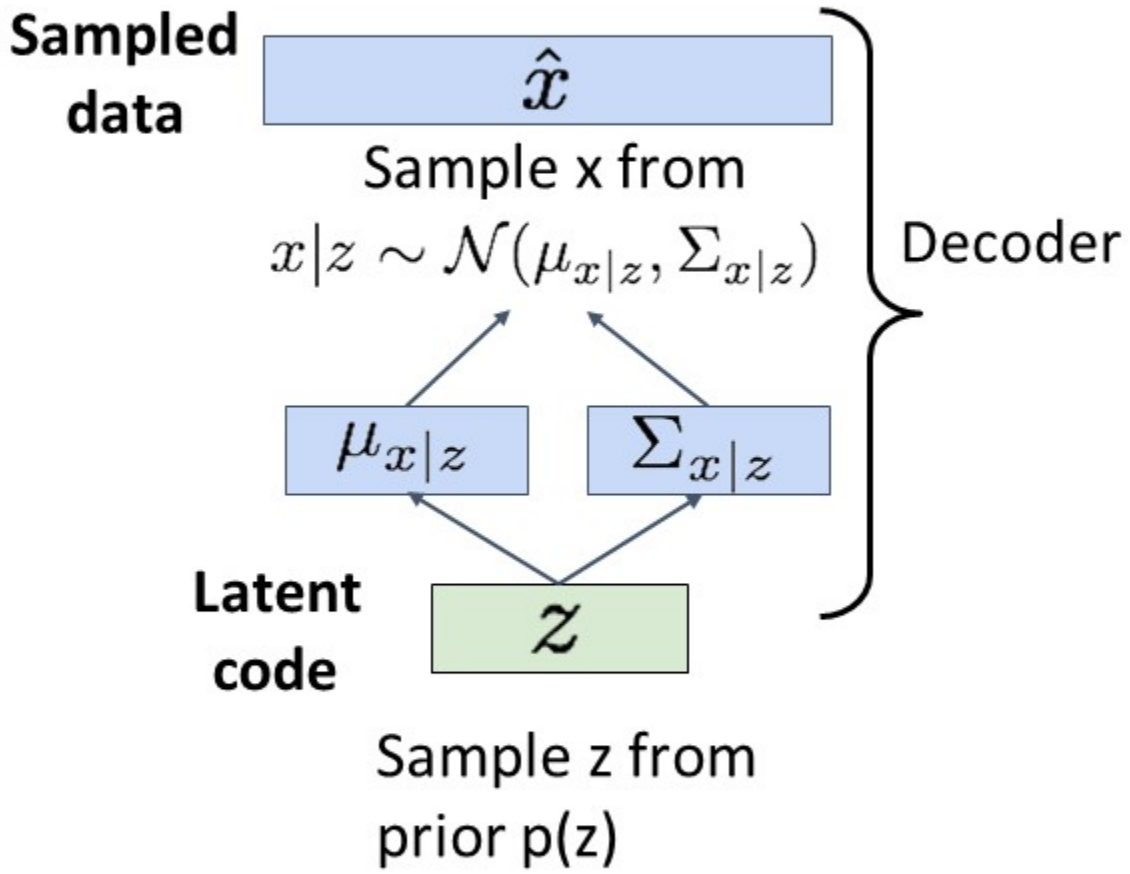
1. Sample z from prior p(z)
2. Run sampled z through decoder to get distribution over data x



Variational Autoencoders: Generating Data

After training we can generate new data!

1. Sample z from prior p(z)
2. Run sampled z through decoder to get distribution over data x
3. Sample from distribution in (2) to generate data



Variational Autoencoders: Generating Data

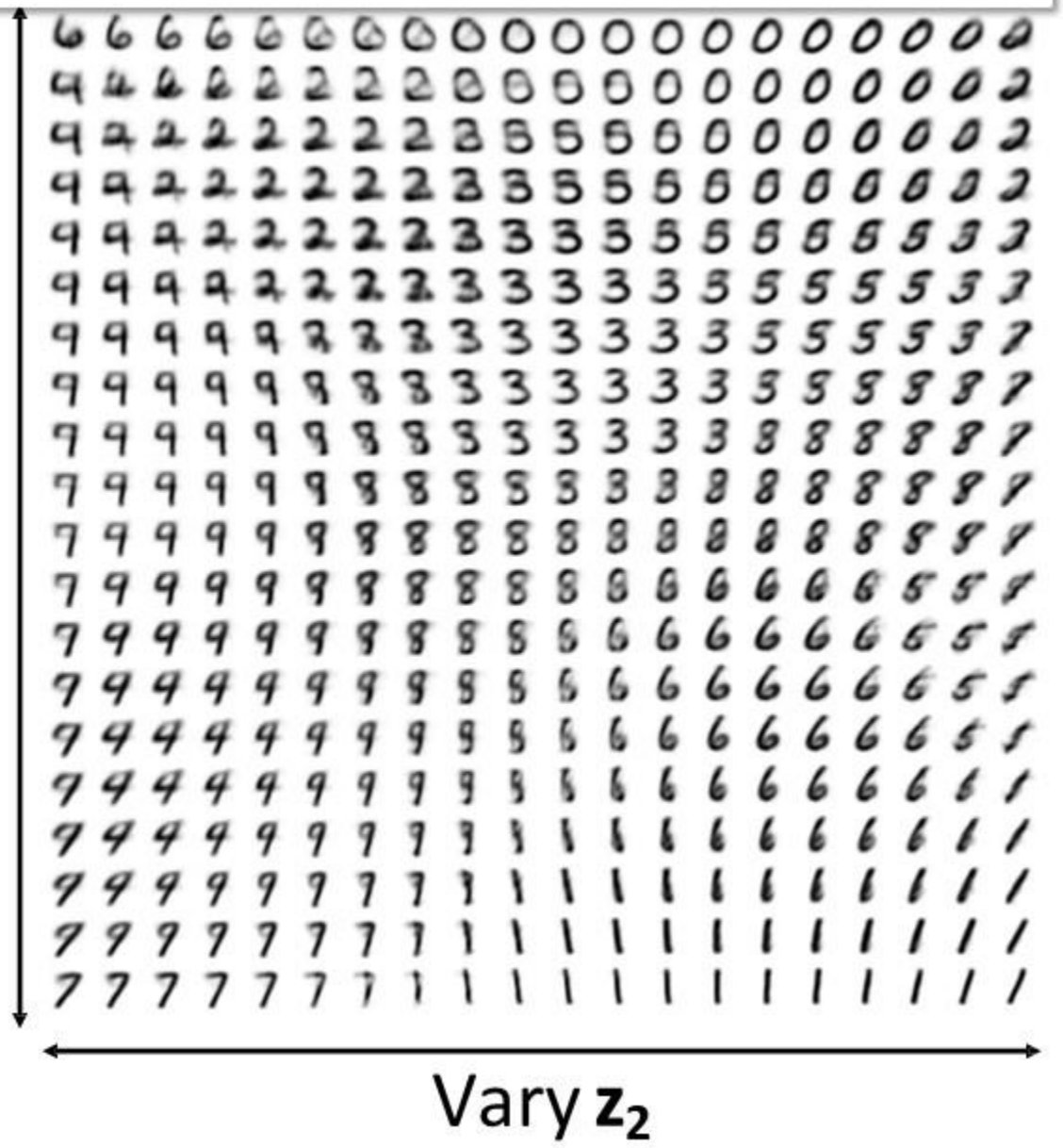
32x32 CIFAR-10

Labeled Faces in the Wild

Variational Autoencoders

The diagonal prior on p(z) causes dimensions of z to be independent

"Disentangling factors of variation"

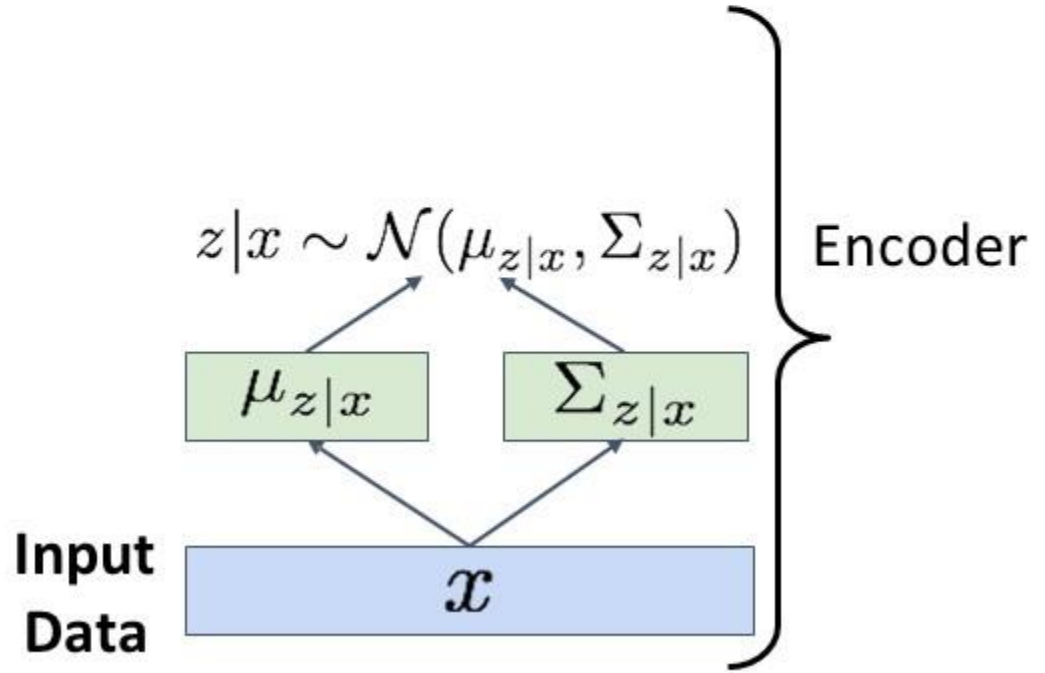


Kingma and Welling, Auto-Encoding Variational Beyes, ICLR 2014

Variational Autoencoders

After training we can **edit images**

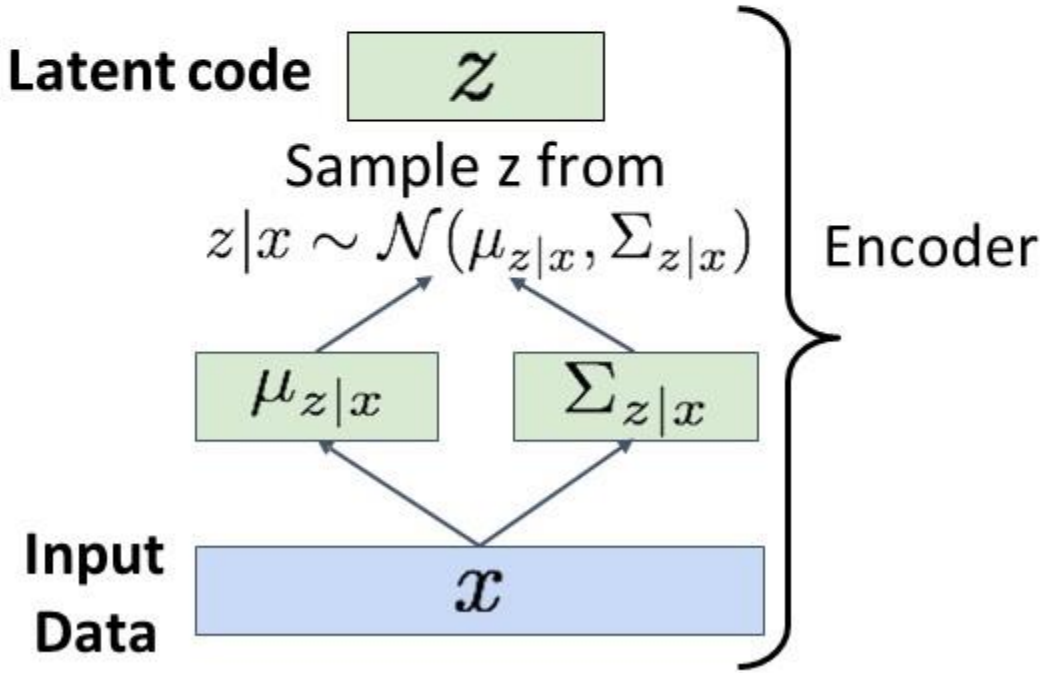
1. Run input data through **encoder** to get a distribution over latent codes



Variational Autoencoders

After training we can **edit images**

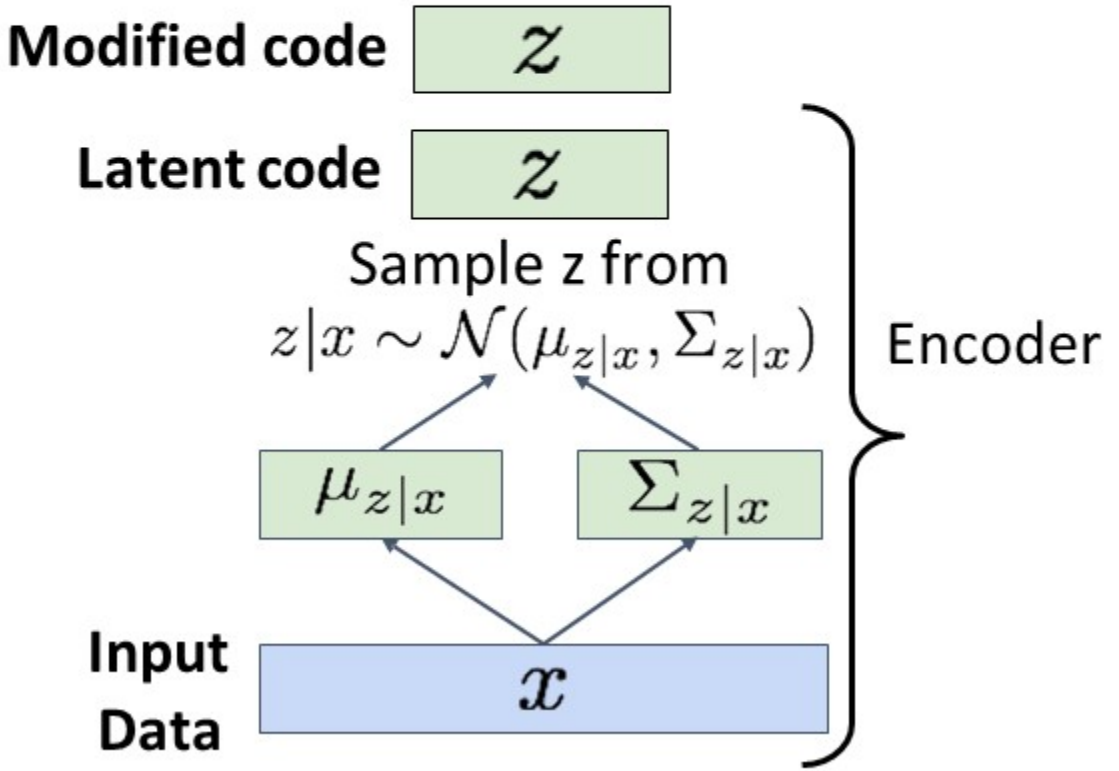
1. Run input data through **encoder** to get a distribution over latent codes
2. Sample code z from encoder output



Variational Autoencoders

After training we can **edit images**

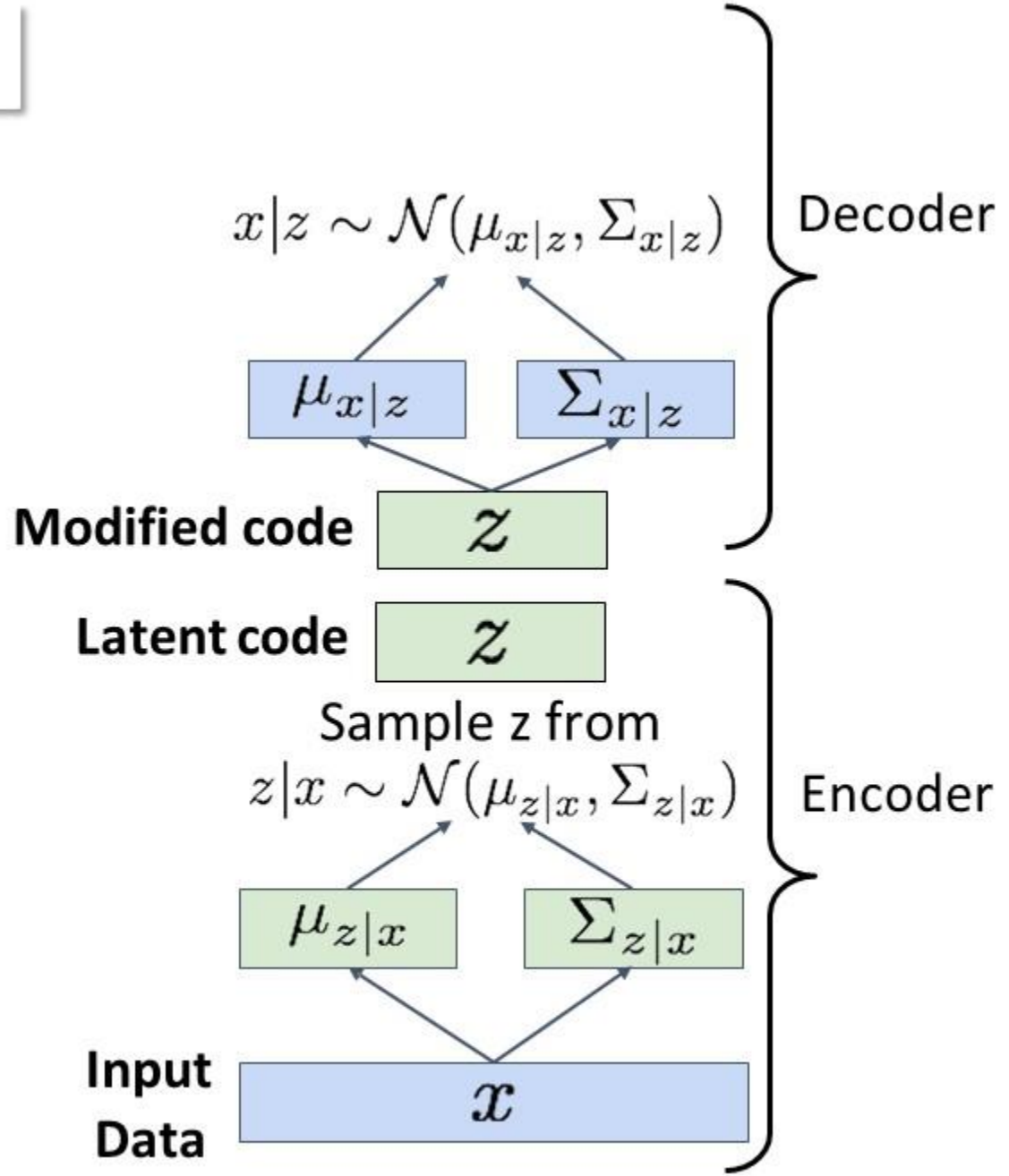
1. Run input data through **encoder** to get a distribution over latent codes
2. Sample code z from encoder output
3. Modify some dimensions of sampled code



Variational Autoencoders

After training we can **edit images**

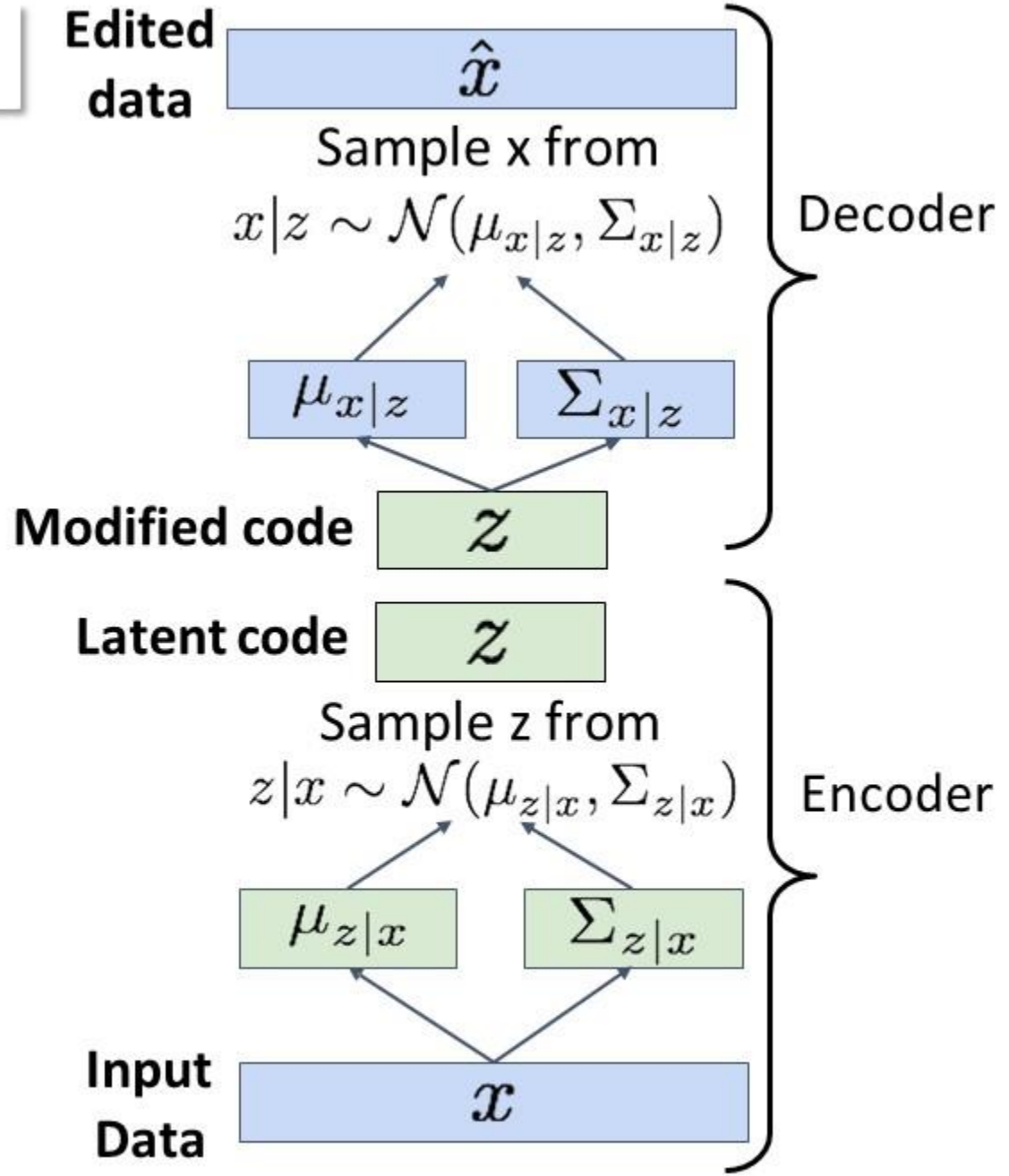
1. Run input data through **encoder** to get a distribution over latent codes
2. Sample code z from encoder output
3. Modify some dimensions of sampled code
4. Run modified z through **decoder** to get a distribution over data sample



Variational Autoencoders

After training we can **edit images**

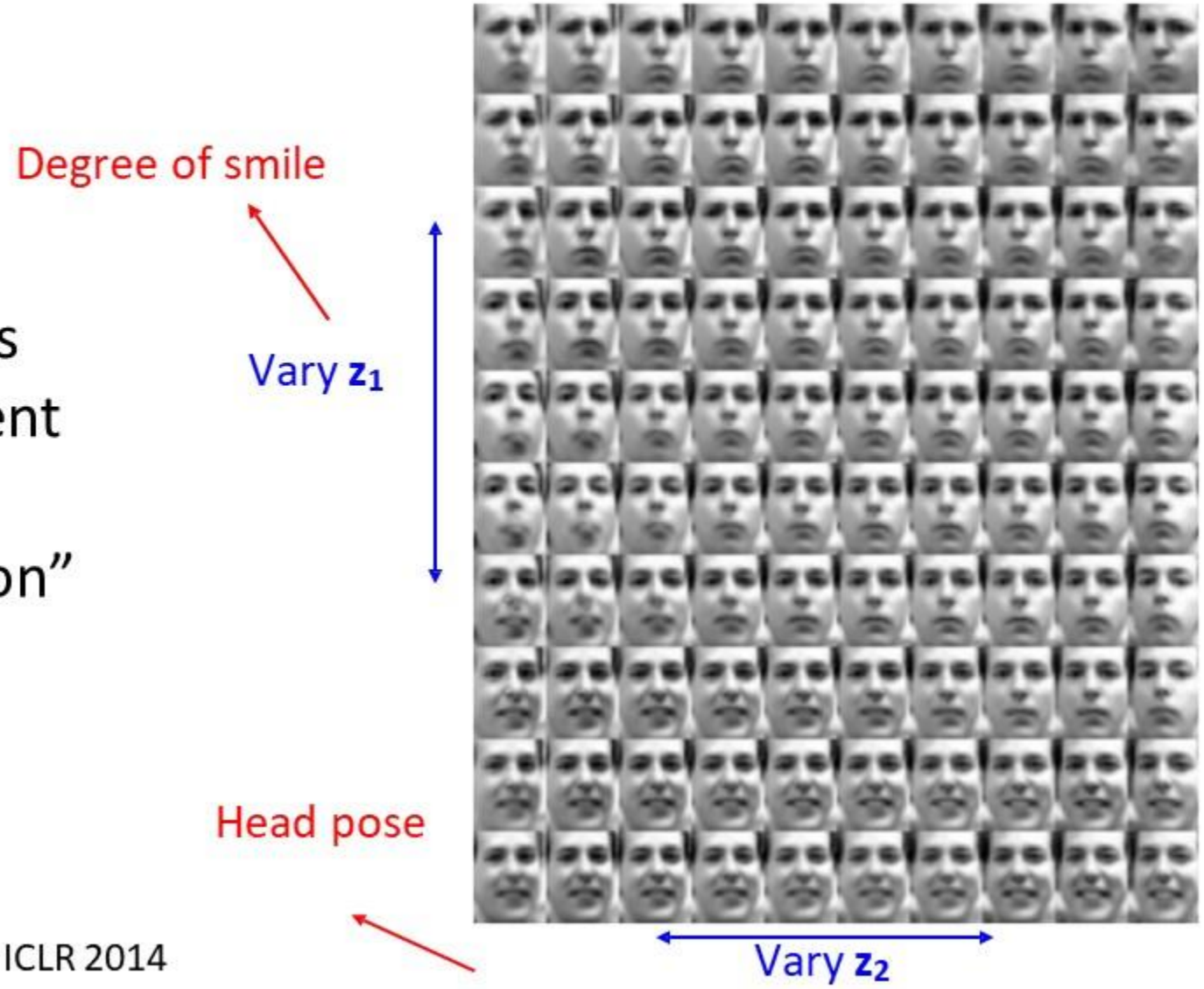
1. Run input data through **encoder** to get a distribution over latent codes
2. Sample code z from encoder output
3. Modify some dimensions of sampled code
4. Run modified z through **decoder** to get a distribution over data samples
5. Sample new data from (4)



Variational Autoencoders

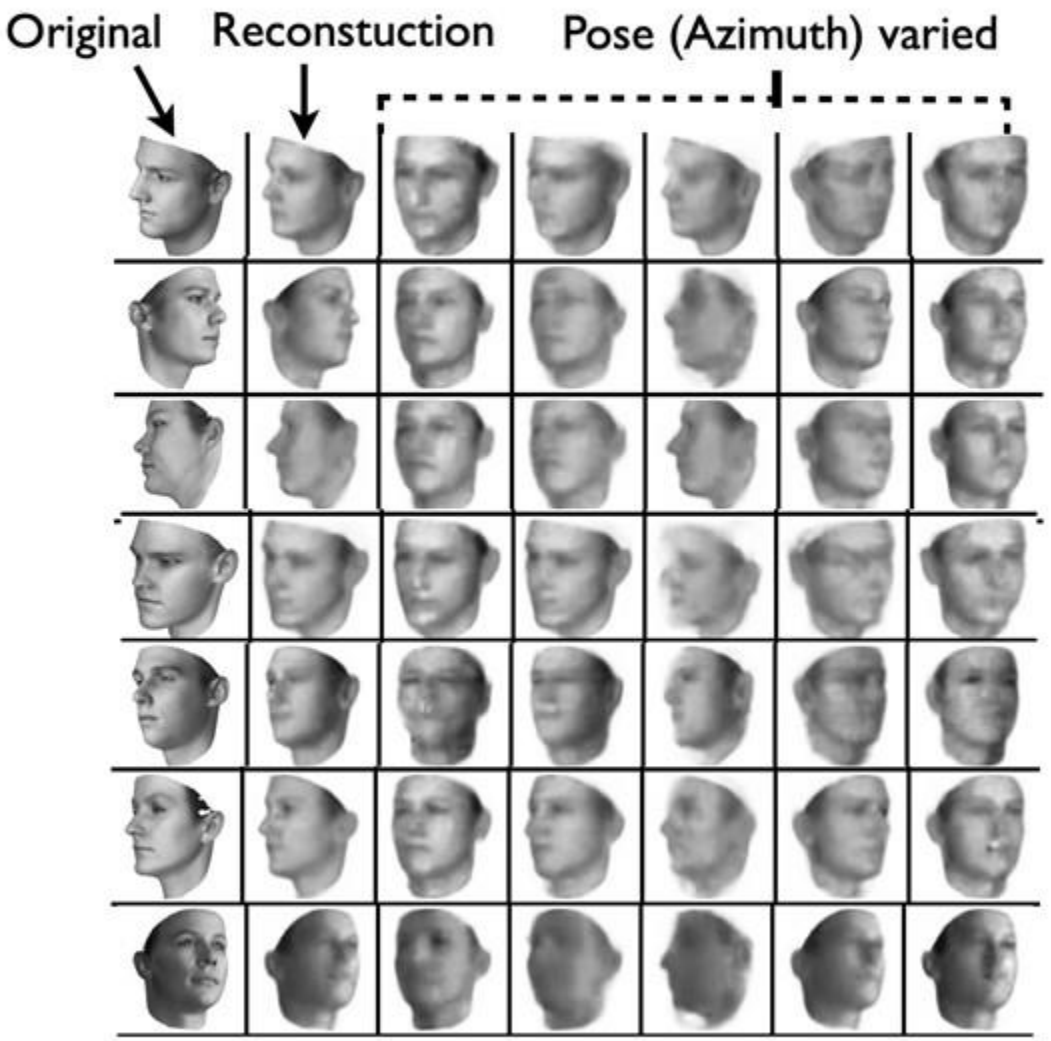
The diagonal prior on p(z) causes dimensions of z to be independent

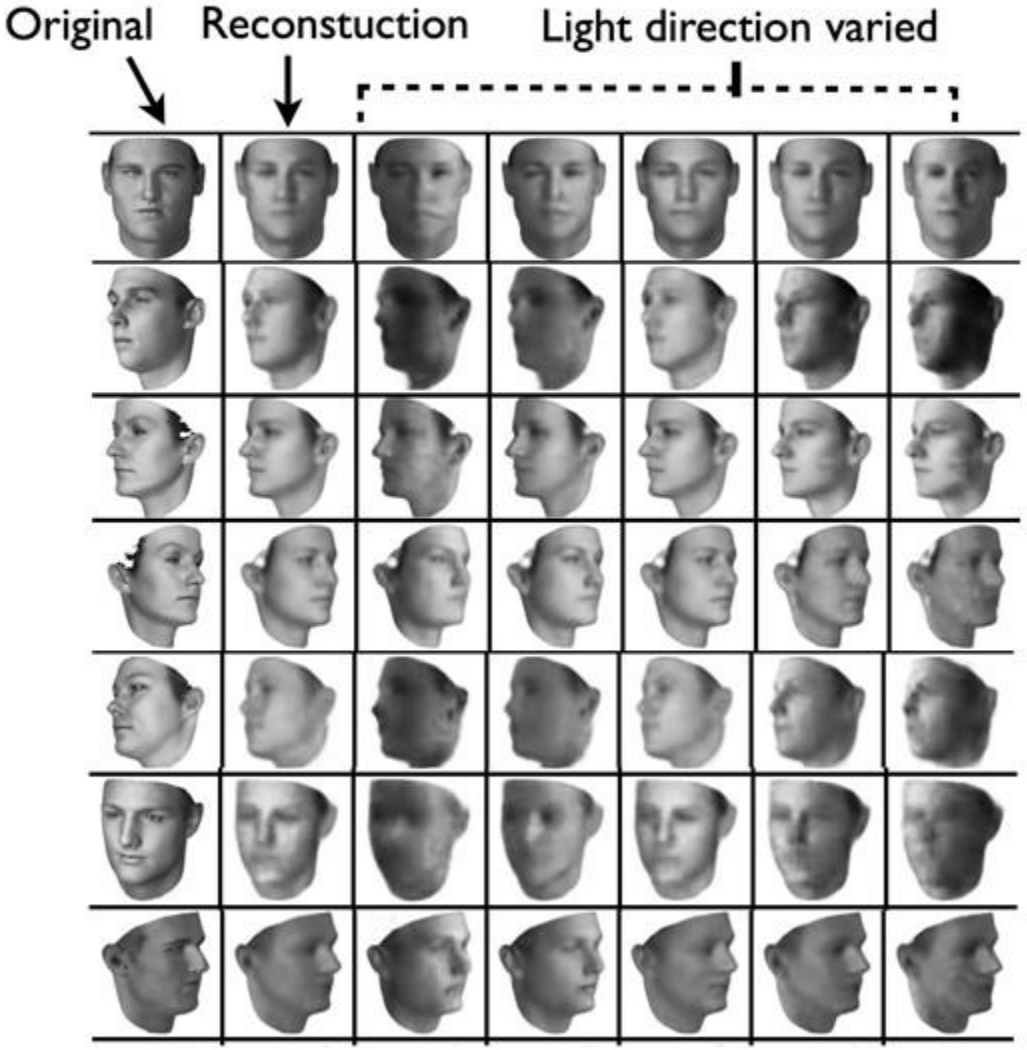
"Disentangling factors of variation"



Kingma and Welling, Auto-Encoding Variational Beyes, ICLR 2014

Variational Autoencoders: Image Editing





Kulkarni et al, "Deep Convolutional Inverse Graphics Networks", NeurIPS 2014

Variational Autoencoder: Summary

Probabilistic spin to traditional autoencoders => allows generating data
Defines an intractable density => derive and optimize a (variational) lower bound

Pros:

- Principled approach to generative models
- Allows inference of $q(z|x)$, can be useful feature representation for other tasks

Cons:

- Maximizes lower bound of likelihood: okay, but not as good evaluation as PixelRNN/PixelCNN
- Samples blurrier and lower quality compared to state-of-the-art (GANs)

Active areas of research:

- More flexible approximations, e.g. richer approximate posterior instead of diagonal Gaussian, e.g., Gaussian Mixture Models (GMMs)
- Incorporating structure in latent variables, e.g., Categorical Distributions